BİLGİSAYAR MÜHENDİSLİĞİ

Teknik Mülakat Hazırlık Notları

İşletim Sistemleri • C Programlama • OOP & C++ • Bilgisayar Ağları

Yapay Zeka ile Soru-Cevap Derlemesi

İçindekiler

Giriş: Asistanın Tanımı ve Çalışma Formatı 4
İşletim Sistemleri 7
Process ve Thread Farkı 7
Multi-thread ve Multi-process 9
PCB (Process Control Block) 11
Zombie ve Orphan Process'ler 13
Concurrency (Eşzamanlılık) 15
Race Condition (Yarış Durumu) 17
Race Condition Oluşturan Koşullar 19
Mutex / Semaphore / Monitor 21
Mutex ve Semafor Farkı 23
Semafor Hangi Problem İçin Bir Çözüm Oluşturmuştur 25
Atomik Değişken 27
Deadlock (Kilitlenme) 29
Starvation (Açlık) 31
IPC (Süreçler Arası İletişim) Yöntemleri 33
C Programlama 35
static Değişken 35
static Fonksiyon 37
struct ve union Farkı 39
enum 41
Pointer'lar (Göstericiler) 43
Makrolar (#define) 46
C Kodunun Executable Olma ve Çalıştırılma Süreci 49
Nesne Yönelimli Programlama (OOP) ve C++ 51
Encapsulation, Abstraction, Inheritance ve Polymorphism 51
Abstract Class ve Interface 53
public, private, protected (Erişim Belirleyiciler) 55
C++'ta struct ve class Farkı 57
C++'ta malloc ve new Farkı 59
Smart Pointers (Akıllı Göstericiler) 61
Casting (Tür Dönüşümü) 63
C++'ta extern 65
C++'ta virtual Kullanımı 67
Egzersiz: Çıktı Ne Olur, Neden? (virtual & Object Slicing) 69
C++ Kodunun Executable Olma ve Çalıştırılma Süreci 71
Veri Yapıları 73
Array, Dinamik Array ve Linked List Karşılaştırması 73
map vs unordered_map (Red-Black Tree vs Hash Map) 75

Heap Veri Yapısı 77
Cache ve Bellek Düzeni Karmaşıklık Analizi 79
Bilgisayar Ağları (Networking) 81
TCP ve UDP 81
Yazılım Geliştirme Süreçleri (Agile & DevOps) 83
Agile (Çevik) Metodoloji 83
CI/CD (Continuous Integration / Continuous Deployment) 85

Giriş: Asistanın Tanımı ve Çalışma Formatı

SORU: *Bilgisayar mühendisi mülakatları için bir hazırlık asistanısın. İşletim sistemleri, Veri yapıları, oop, c programlama gibi konularda uzmansın.*

Harika! Rolü büyük bir memnuniyetle kabul ediyorum. Bilgisayar mühendisliği mülakatlarına giden yolda senin hem zorlayıcı teknik mülakatçın hem de rehberinim.

Seni özellikle FAANG seviyesi şirketlerin veya köklü teknoloji firmalarının teknik mülakatlarına (technical screenings) hazırlamak için buradayım. Konularımızı mülakatlarda en çok karşılaşılan ve adayları en çok zorlayan detaylar üzerinden işleyebiliriz.

Üzerinde Çalışabileceğimiz Temel Konular

- **Veri Yapıları ve Algoritmalar:** Array, Linked List, Stack, Queue, Tree (BST, AVL, Trie, Red-Black), Graph yapıları. Mülakatların kalbi olan zaman ve alan karmaşıklığı analizleri (O(1), O(logn), O(n), O(nlogn) vb.).
- **İşletim Sistemleri:** Process vs. Thread, Concurrency (Eşzamanlılık), Mutex/Semaphore/Monitor kavramları, Deadlock senaryoları, Memory Management (Paging, Virtual Memory, Thrashing) ve CPU Scheduling algoritmaları.
- **Nesne Yönelimli Programlama (OOP):** Sadece Encapsulation, Abstraction, Inheritance ve Polymorphism kavramları değil; aynı zamanda mülakatların vazgeçilmezi olan SOLID prensipleri ve sık kullanılan Tasarım Kalıpları (Singleton, Factory, Observer vb.).
- **C Programlama:** Pointer aritmetiği, fonksiyon göstericileri (function pointers), dinamik bellek yönetimi (malloc, calloc, realloc, free), Struct padding/packing, Bitwise operasyonlar ve tehlikeli bellek hataları (memory leaks, dangling pointers).

Nasıl İlerleyebiliriz?

Bu süreci senin ihtiyaçlarına göre şekillendirebiliriz. İşte birkaç çalışma yöntemi:

- **Gerçekçi Mülakat Simülasyonu:** Sana bir şirketin sorabileceği tarzda bir problem veririm. Sen çözüm yaklaşımını (brute force'tan optimize çözüme doğru) ve kodunu paylaşırsın; ben de bir mülakatçı gibi köşeye sıkıştıran (edge case) sorular sorarım.
- **Kavramsal Derinlemesine İnceleme:** "Polymorphism'in arka planında C++ gibi dillerde vtable (virtual table) nasıl çalışır?" gibi teorik derinliği olan konuları tartışabiliriz.
- **Hata Ayıklama (Debugging) Pratiği:** Sana içinde gizli hatalar olan (örneğin ince bir memory leak veya race condition barındıran) bir kod bloğu veririm ve hatayı bulmanı isterim.

Hedefin nedir? Doğrudan bir mülakat sorusuyla mı başlayalım, yoksa belirli bir konunun üzerinden mi geçelim?

Hemen başlamak için bir alan seçebilirsin:

C'de zorlayıcı bir pointer sorusu sor
› Veri yapıları mülakat simülasyonu başlat
› İşletim sistemleri teori sorusu sor

SORU: *Sorduğum sorular için öncelikle mülakatta cevap verecek şekilde bir şekilde açıkla. Sonra tüm konuları detaylı şekilde arka planlarını da anlat*

Anlaştık. Bu, teknik mülakatlara hazırlanmak için kesinlikle en iyi stratejidir. Mülakatçılar ilk başta kısa, net ve doğrudan soruyu yanıtlayan "asansör konuşması" (elevator pitch) tarzı bir cevap duymak isterler. Ardından "Peki bu arka planda nasıl çalışır?" dediklerinde o derin teknik bilgiyi göstermen gerekir.

Bundan sonraki tüm sorularında bu formatı kullanacağım:

- **Mülakat Cevabı:** Doğrudan, net ve mülakatçıya 1-2 dakikada verilebilecek ideal yanıt.
- **Detaylı Arka Plan:** Konunun mimarisi, bellek yönetimi, işletim sistemi seviyesindeki karşılığı ve potansiyel tuzakları.

Sistemin nasıl çalışacağını sana göstermek için işletim sistemleri mülakatlarının en klasik ve eleyici sorularından biriyle bir örnek yapalım.

Örnek Soru: Process (Süreç) ve Thread (İş Parçacığı) arasındaki fark nedir?

1. Mülakat Cevabı (Örnek Yanıtın)

> *"Temel fark bellek izolasyonudur. Process, işletim sisteminde bağımsız olarak çalışan, kendi bellek alanına (heap, data, code) sahip olan ağır siklet (heavyweight) bir yürütme birimidir. İki process birbirinin belleğine doğrudan erişemez, iletişim için IPC (Inter-Process Communication) mekanizmalarına ihtiyaç duyarlar.*
>
> *Thread ise bir process'in içinde çalışan hafif siklet (lightweight) bir yürütme birimidir. Aynı process içindeki thread'ler; code, data ve heap alanlarını ortaklaşa kullanırken, her birinin kendine ait bağımsız bir Stack (yığın) alanı ve Register (yazmaç) seti vardır. Thread'ler arası bağlam değişimi (context switch) çok daha hızlıdır, ancak paylaşılan bellek nedeniyle race condition (yarış durumu) gibi eşzamanlılık (concurrency) sorunlarına açıktırlar ve Mutex gibi senkronizasyon araçları gerektirirler."*

2. Detaylı Arka Plan

Bu konunun derinine indiğimizde işin donanım ve işletim sistemi çekirdeği (kernel) boyutu ortaya çıkar.

Bellek Düzeni (Memory Layout)

Bir C programı çalıştırıldığında (Process olduğunda) RAM'de sanal bellek (virtual memory) bloklarına ayrılır:

- **Text Segment:** Derlenmiş makine kodu. (Thread'ler paylaşır)
- **Data/BSS Segment:** Global ve statik değişkenler. (Thread'ler paylaşır)
- **Heap:** Dinamik olarak tahsis edilen bellek (malloc, new). (Thread'ler paylaşır)
- **Stack:** Yerel değişkenler ve fonksiyon çağrı geçmişi. (Her thread'in kendi stack'i vardır!)

Neden her thread'in kendi stack'i olmak zorunda? Çünkü stack, bir fonksiyonun nerede çalıştığını, yerel değişkenlerini ve fonksiyon bitince nereye döneceğini (return address) tutar. Eğer iki thread aynı stack'i paylaşsaydı, birbirlerinin fonksiyon çağrılarını ezer ve programın çökmesine (segmentation fault) neden olurlardı.

Bağlam Değişimi (Context Switch) Neden Süreçlerde Daha Yavaştır?

Mülakatçı "Peki thread'ler neden process'lerden daha hızlı geçiş yapar?" diye sorarsa vereceğin cevap TLB (Translation Lookaside Buffer) olmalıdır.

İki process arasında geçiş yapıldığında, işletim sistemi sayfa tablosunu (page table) değiştirmek zorundadır. Bu durum, CPU içindeki bellek adres çeviri önbelleği olan TLB'nin sıfırlanmasına (TLB flush) neden olur. TLB sıfırlandığında sistem bir süreliğine belleğe erişimde yavaşlar (cache miss). Ancak aynı process içindeki iki thread arasında geçiş yapılırken sayfa tablosu değişmez, TLB aynen kalır. Sadece CPU register'ları ve Stack Pointer (SP) değiştirilir.

Gerçek Hayat Mimarisi

Özellik	Process	Thread
İzolasyon	Yüksek (Biri çökerse diğeri etkilenmez)	Düşük (Biri çökerse tüm süreç çöker)
Yaratılma Maliyeti	Yüksek (fork() veya exec())	Düşük (pthread_create())
Örnek Kullanım	Chrome sekmeleri (Her sekme ayrı process)	Web sunucusunda gelen istekleri işleme

Formatımız bu şekilde olacak. Şimdi kontrol sende. Hangi konudan gerçek bir soruyla seni test etmemi istersin?

İlk konunu seç, soruyu sorayım ve cevabını bekleyeyim:

› *C Programlama ve Bellek Yönetimi*

› *Veri Yapıları ve Algoritmalar*

› *Nesne Yönelimli Programlama (OOP)*

İşletim Sistemleri

Process ve Thread Farkı

> **SORU:** *Process ve thread farkı nedir?*

Önceki mesajımda bu soruyu sistemimizin nasıl çalışacağını göstermek için örnek olarak kullanmıştım. Madem doğrudan bu klasik ve çok önemli soruyla başladık; mülakatta bu soruyu aldığında cevabını nasıl yapılandırman gerektiğine ve arka planda mülakatçının senden duymayı beklediği o "kilit kelimelere" (TLB, Copy-on-Write, Isolation) odaklanarak çok daha kapsamlı bir yanıt hazırlayalım.

1. Mülakat Cevabı

> *"Temel fark, kaynak izolasyonu ve bellek paylaşımıdır.*
>
> *Bir Process (Süreç), işletim sistemi tarafından kaynak tahsis edilen, tamamen izole edilmiş bir yürütme birimidir. Her process'in kendi bağımsız bellek alanı (Code, Data, Heap ve Stack) vardır. Bu yüksek izolasyon, bir process çöktüğünde diğerlerinin etkilenmemesini sağlar. Ancak bu durum, process'ler arası iletişimi (IPC) ve bağlam değişimini (context switch) yavaş ve maliyetli hale getirir.*
>
> *Bir Thread (İş Parçacığı) ise process içinde yaşayan daha hafif bir yürütme birimidir. Aynı process içindeki thread'ler; Code, Data ve Heap alanlarını ortaklaşa kullanır. Sadece Stack (Yığın) ve Register (Yazmaç) setleri kendilerine özeldir. Belleği paylaştıkları için yaratılmaları ve aralarında geçiş yapılması çok daha hızlıdır. Ancak bu paylaşım, Race Condition (yarış durumu) gibi tehlikelere yol açar ve Mutex/Semaphore gibi senkronizasyon mekanizmalarının kullanılmasını zorunlu kılar. Kısacası process'ler güvenli ve maliyetliyken, thread'ler hızlı ama yönetimi risklidir."*

2. Detaylı Arka Plan

Mülakatçı ilk cevabından tatmin olduktan sonra muhtemelen "Neden daha hızlı?", "Bellek nasıl yönetiliyor?" veya "İşletim sistemi bunları nasıl ayırıyor?" gibi derinlemesine sorular soracaktır. İşte bilmen gereken teknik altyapı:

Bellek Mimarisi ve İzolasyon

Bir program çalıştırıldığında işletim sistemi ona sanal bellek (virtual memory) tahsis eder.

- **Process Yaklaşımı:** İki farklı process, bellek adreslerinde "0x00400000" adresini okumaya çalışırsa, Page Table (Sayfa Tablosu) sayesinde fiziksel RAM'de tamamen farklı yerlere giderler. Birbirlerinin verisini göremezler.
- **Thread Yaklaşımı:** İki thread aynı adresi (örneğin Heap'teki bir objeyi) okursa, fiziksel RAM'de birebir aynı veriye ulaşırlar.

Mülakat Tuzağı: "Thread'ler her şeyi paylaşır mı?" sorusuna "Evet" deme. Stack (yerel değişkenler, fonksiyon dönüş adresleri) ve CPU Register'ları (Program Counter gibi) her thread için kesinlikle bağımsızdır. Aksi takdirde aynı anda iki farklı fonksiyonu çalıştıramazlardı.

Bağlam Değişimi (Context Switch) Performansı

Mülakatların en eleyici noktalarından biri şudur: Thread'ler arası geçiş neden process'ler arası geçişten daha hızlıdır?

- **Page Table Değişimi:** Process'ler arası geçişte işletim sistemi donanımsal Sayfa Tablosunu (x86 mimarisinde CR3 register'ı) değiştirmek zorundadır. Thread'lerde bellek ortak olduğu için buna gerek yoktur.

- **TLB (Translation Lookaside Buffer) Flush:** Page table değiştiğinde, işlemci içindeki bellek adreslerini önbellekleyen donanımsal yapı olan TLB sıfırlanır (flush). TLB sıfırlandığında sistem bir süreliğine belleğe yavaş erişir (Cache Miss). Thread geçişlerinde TLB sıfırlanmaz, bu da devasa bir performans avantajı sağlar.

Yaratılma Maliyeti ve Copy-on-Write (CoW)

- **Process Yaratmak (fork()):** Unix/Linux sistemlerde yeni bir process yaratıldığında işletim sistemi bellek alanının bir kopyasını çıkarır. Modern işletim sistemleri bunu optimize etmek için Copy-on-Write (CoW) kullanır. Yani process'ler veriyi değiştirene kadar fiziksel belleği paylaşırlar, yazma işlemi yapıldığında bellek kopyalanır. Yine de OS tarafındaki veri yapıları (PCB - Process Control Block) ağırdır.
- **Thread Yaratmak (pthread_create()):** İşletim sistemi sadece yeni bir Stack tahsis eder ve Thread Control Block (TCB) oluşturur. Çok daha az sistem çağrısı (system call) ve maliyet gerektirir.

Hata Toleransı (Fault Tolerance)

Neden hala multi-thread yerine multi-process kullanıyoruz? (Örneğin Google Chrome neden her sekme için ayrı bir process açar?)

Cevap güvenlik ve dayanıklılıktır. Bir process içindeki thread'lerden biri yanlış bir bellek adresine erişip Segmentation Fault hatası alırsa, işletim sistemi SIGSEGV sinyalini process'in bütününe gönderir. Yani bir thread'in hatası tüm process'i ve diğer tüm thread'leri öldürür. Multi-process mimarilerde ise bir process çökerse, sadece o process ölür, sistemin geri kalanı çalışmaya devam eder.

Multi-thread ve Multi-process

> **SORU:** *Multi-thread vs multi-process*

Daha önce Process (Süreç) ve Thread (İş Parçacığı) tanımlarına değinmiştik. Ancak mülakatçı soruyu "Hangi durumda Multi-process, hangi durumda Multi-thread mimarisi kurmayı tercih edersin?" şeklinde sorarsa, senden bir Sistem Tasarımı (System Design) analizi bekliyordur.

İşte bu mimari seçimi mülakatçıya nasıl satacağın:

1. Mülakat Cevabı

> *"Bu iki mimari arasındaki seçim; performans, güvenlik ve iletişim maliyeti arasındaki dengeye (trade-off) bağlıdır.*
>
> *Multi-thread mimarisini, görevlerin yoğun bir şekilde ortak veriye erişmesi gereken, yüksek performanslı ve hızlı bağlam değişimi (context switch) gerektiren durumlarda tercih ederim (örneğin bir oyun motoru veya yoğun I/O yapan bir web sunucusu). Belleği paylaştıkları için iletişim çok hızlıdır ancak 'Race Condition' risklerini yönetmek zorundadır.*
>
> *Multi-process mimarisini ise güvenlik, izolasyon ve hata toleransının (fault tolerance) kritik olduğu durumlarda kullanırım. Bellekleri izole olduğu için bir süreç çöktüğünde (örneğin 'Segmentation Fault') diğerleri etkilenmez. Google Chrome'un her sekme için ayrı bir süreç (process) açması buna en iyi örnektir. Dezavantajı ise yüksek bellek tüketimi ve süreçler arası iletişimin (IPC) yavaş ve zahmetli olmasıdır."*

2. Detaylı Arka Plan

Mülakatçı cevabını aldıktan sonra, "Peki bu iki mimari birbiriyle nasıl haberleşir?" veya "Global Interpreter Lock (GIL) gibi dillerin kısıtlamaları bu seçimi nasıl etkiler?" gibi detaylara inecektir.

Haberleşme ve İletişim Maliyeti

En büyük mimari fark veriyi nasıl paylaştıklarıdır.

- **Multi-thread Haberleşmesi:** İki thread aynı process içinde yaşadıkları için Heap ve Data segmentleri ortaktır. Thread A, global bir değişkene yazar; Thread B anında onu okur. İletişim maliyeti neredeyse sıfırdır. Tek yapman gereken bu erişimi bir Mutex ile korumaktır.
- **Multi-process Haberleşmesi (IPC):** İki process'in bellek alanları fiziksel olarak izole edildiği için, birbirlerinin değişkenlerini doğrudan okuyamazlar. İletişim kurmak için İşletim Sisteminin sunduğu IPC (Inter-Process Communication) araçlarını kullanmak zorundadırlar.
- **IPC Yöntemleri:** Pipes (Borular), Message Queues (Mesaj Kuyrukları), Sockets (Ağ Soketleri - mikroservisler gibi) veya Shared Memory (Paylaşılan Bellek). Bunların hepsi İşletim Sistemi çekirdeğine (kernel space) inmeyi gerektirdiği için system call (sistem çağrısı) maliyeti yaratır ve yavaştır.

Hata Toleransı ve Güvenlik (Neden Multi-process?)

Eğer Multi-thread bu kadar hızlıysa neden her şeyi böyle yazmıyoruz? Çünkü çok kırılgandır.

C/C++ gibi bir dilde yazdığın multi-thread bir uygulamada, thread'lerden sadece bir tanesi yetkisi olmayan bir bellek adresine (örneğin NULL pointer) erişirse, işletim sistemi SIGSEGV (Segmentation Fault) sinyali üretir. Bu sinyal thread'e değil, process'in tamamına gönderilir. Yani bir hatalı thread, aynı uygulamada sorunsuz çalışan diğer 100 thread'i de öldürür.

Eğer yazdığın sistem bir bankacılık uygulaması veya izole çalışması gereken bir veritabanı motoruysa (örneğin PostgreSQL), her bağlantı için genellikle ayrı bir process açılır. Biri çökerse, sadece o müşterinin işlemi patlar; sistemin geri kalanı çalışmaya devam eder.

Programlama Dili Kısıtlamaları (Python ve Node.js İstisnaları)

Mülakatlarda bu konu genellikle modern dillerin limitleriyle birleştirilir:

- **Python ve GIL (Global Interpreter Lock):** Python (CPython) mülakatında multi-thread sorulursa cevap çok nettir: Python'da CPU-bound (işlemciyi yoran matematiksel hesaplamalar vs.) işler için multi-thread KULLANILMAZ. Çünkü GIL adı verilen bir kilit, aynı anda sadece bir thread'in Python byte kodunu çalıştırmasına izin verir. Gerçek paralellik elde etmek için Python'da multiprocessing kütüphanesi ile Multi-process mimarisi kullanmak zorundasındır.
- **Node.js:** Node.js doğası gereği Single-Thread (Tek iş parçacıklı) ve Asenkron çalışır. İşlemciyi kilitleyen ağır bir şifreleme (kriptografi) işlemi yapman gerekirse, ana thread (Event Loop) kilitlenir. Bu yüzden Node.js'te ağır hesaplamalar worker_threads (Multi-thread) veya child_process (Multi-process) mimarileriyle ana sistemin dışına itilir.

Hızlı Karşılaştırma Tablosu

Kriter	Multi-Thread	Multi-Process
Yaratılma Maliyeti	Çok düşük (Sadece Stack tahsisi)	Yüksek (Fork/Copy-on-write işlemleri)
İletişim (Haberleşme)	Çok hızlı (Shared Memory ile doğrudan)	Yavaş (IPC - Pipe, Socket, Queue)
Hata İzolasyonu	Yok (Biri çökerse hepsi çöker)	Yüksek (Biri çökerse diğerleri etkilenmez)
Senkronizasyon Riski	Çok yüksek (Race condition, Deadlock)	Düşük (Veri paylaşımı sınırlı olduğu için)

PCB (Process Control Block)

> **SORU:** *PCB ne zaman oluşturulur ve kaydedilir? Ayrıca ne tür bilgiler içerir?*

1. Mülakat Cevabı

> *"PCB (Process Control Block - Süreç Kontrol Bloğu), işletim sisteminin bir süreci (process) yönetmek ve takip etmek için kullandığı, o sürece ait tüm kritik bilgileri barındıran çekirdek (kernel) seviyesindeki bir veri yapısıdır.*
>
> *Ne Zaman Oluşturulur? İşletim sistemi yeni bir süreç yaratırken (örneğin Unix/Linux'ta fork() çağrısıyla) oluşturulur.*
>
> *Ne Zaman Kaydedilir (Güncellenir)? Sürecin durumu her değiştiğinde güncellenir. Ancak en kritik 'kaydetme' işlemi Bağlam Değişimi (Context Switch) sırasında olur. İşlemci bir süreci durdurup diğerine geçerken, durdurulan sürecin o anki işlemci durumu (Register'lar, Program Counter vb.) daha sonra kaldığı yerden devam edebilmesi için PCB'sine kaydedilir.*
>
> *Ne Tür Bilgiler İçerir? Temel olarak süreci tanımlayan kimlik bilgileri (PID), sürecin durumu (Ready, Running, Blocked), bir sonraki çalıştırılacak komutun adresi (Program Counter), CPU yazmaç (Register) durumları, bellek yönetim bilgileri (Sayfa tablosu işaretçileri) ve açık dosyaların listesi (File Descriptors) gibi bilgileri içerir."*

2. Detaylı Arka Plan

PCB konusu, mülakatlarda işletim sistemi mimarisinin kalbine inmek için bir giriş kapısıdır. Mülakatçı bu soruyu genellikle "Peki bu PCB nerede tutuluyor?" veya "Zombie Process nedir?" gibi sorularla devam ettirir.

PCB'nin Anatomisi (İçerdiği Bilgiler)

Bir süreci ayağa kaldırmak ve yönetmek için işletim sisteminin devasa bir kayıt defteri tutması gerekir. C dilindeki dev bir struct gibi düşünebileceğin PCB'yi mülakatta kategorilere ayırarak anlatmak büyük puan getirir:

- **Süreç Kimliği (Process Identification):** Sürecin benzersiz numarası (PID), onu yaratan ebeveyn sürecin numarası (PPID) ve süreci çalıştıran kullanıcının kimliği (UID).
- **İşlemci Durumu (CPU State / Hardware Context):** Süreç CPU'dan atıldığında donanımın son durumu buraya yazılır.
- Program Counter (PC): Süreç tekrar çalışmaya başladığında kodun hangi satırından devam edeceğini tutar.
- Stack Pointer (SP): Sürecin o anki fonksiyon çağrı yığınının (stack) tepe noktasını gösterir.
- Genel Amaçlı Yazmaçlar (Registers): CPU içindeki geçici hesaplama verileri.
- **Süreç Kontrol ve Çizelgeleme (Process Control & Scheduling):** Sürecin o anki durumu (New, Ready, Running, Blocked, Terminated) ve CPU Scheduler'ın (Çizelgeleyici) kullanacağı öncelik (Priority) değeri.
- **Bellek Yönetim Bilgileri (Memory Management):** Bu sürecin sanal belleği, fiziksel RAM'de nerelere denk geliyor? Bunu gösteren Sayfa Tablosunun (Page Table) başlangıç adresi (x86 mimarisinde CR3 register'ının değeri) burada tutulur.
- **G/Ç ve Dosya Bilgileri (I/O Status):** Sürecin açtığı dosyaların listesi (File Descriptor Table), kullandığı ağ soketleri ve tahsis edilen donanım aygıtları.

PCB Nerede Yaşar? (Mülakat Tuzağı)

Mülakatçı sorar: "PCB, sürecin kendi bellek alanı içinde mi (örneğin Heap'inde mi) tutulur?"

Kritik Cevap: Kesinlikle hayır! PCB, doğrudan Çekirdek Belleğinde (Kernel Space) tutulur. Eğer PCB kullanıcının erişebildiği User Space'te (sürecin kendi belleğinde) tutulsaydı, kötü niyetli bir program basit bir pointer (gösterici) manipülasyonu ile kendi Priority (öncelik) değerini en yükseğe çıkarabilir veya başka bir sürecin PID'sini taklit edebilirdi. Bu yüzden süreçler kendi PCB'lerine doğrudan dokunamazlar, sadece System Call'lar aracılığıyla işletim sisteminden işlem yapmasını isteyebilirler.

Bağlam Değişimi (Context Switch) ve PCB'nin Rolü

İşletim sisteminde bir Timer Interrupt (zamanlayıcı kesmesi) olduğunda ve CPU bir süreci (Süreç A) durdurup diğerini (Süreç B) almaya karar verdiğinde şu adımlar yaşanır:

- CPU'nun o anki Program Counter, Stack Pointer ve tüm Register değerleri Süreç A'nın PCB'sine yazılır (Kaydetme).
- Süreç A'nın durumu "Running" (Çalışıyor) halinden "Ready" (Hazır) veya "Blocked" (Bloklanmış) haline çekilir.
- İşletim sistemi, çalıştırılacak sıradaki Süreç B'nin PCB'sini okur.
- Süreç B'nin PCB'sindeki Program Counter ve Register değerleri fiziksel CPU'ya yüklenir (Load). Süreç B "Running" durumuna geçer ve CPU çalışmaya devam eder.

Bu işlemin tamamı saf bir genel giderdir (overhead). Bu yüzden Context Switch ne kadar az olursa sistem o kadar performanslı çalışır.

Mülakat Bonusu: Zombie (Zombi) Process Nedir?

Bir süreç işini bitirip exit() çağrısı yaptığında (Terminated durumu), işletim sistemi o sürecin belleğini, açtığı dosyaları ve kullandığı kaynakları tamamen temizler (deallocate).

Ancak PCB'sini hemen silmez!

Çünkü süreci yaratan ebeveyn (parent) sürecin, çocuğunun neden öldüğünü (hata koduyla mı yoksa başarıyla mı bittiğini) okuması gerekir (Bunu Unix'te wait() fonksiyonu ile yapar). Süreç ölmüş, belleği boşaltılmış ama PCB'si ebeveyni tarafından okunmayı beklediği için sistemde (Process Table'da) hala bir satır işgal ediyorsa, buna Zombie Process denir. Eğer ebeveyn süreç wait() çağırmayı unutursa, zombi süreçler sistemin PID havuzunu tüketip sistemi kilitleyebilir.

Zombie ve Orphan Process'ler

> **SORU:** *zombie ve orphan processlerini oluşma sebepleri ile beraber açıkla. Önlem olarak neler yapmalıyız anlat*

1. Mülakat Cevabı

> *"Zombie (Zombi) ve Orphan (Yetim) süreçler, Linux/Unix işletim sistemlerindeki ebeveyn-çocuk (parent-child) süreç ilişkisinin kopması sonucu ortaya çıkan iki farklı durumdur.*
>
> *Zombie Process (Zombi Süreç):*
>
> *Çalışmasını tamamlamış (exit yapmış) ancak ebeveyn süreci tarafından son durumu (wait çağrısıyla) henüz okunmamış olan süreçlerdir.*
>
> *Oluşma Sebebi: Ebeveyn sürecin, çocuğunun bitiş durumunu okumayı unutması veya meşgul olmasıdır.*
>
> *Tehlikesi ve Önlem: Zombiler RAM veya CPU harcamazlar, bellekleri çoktan temizlenmiştir. Ancak Süreç Tablosunda (Process Table) bir satır ve PID (Süreç ID) işgal ederler. Çok fazla zombi oluşursa sistem yeni PID üretemez ve çöker. Önlemek için ebeveyn süreç mutlaka wait() çağrısı yapmalı veya asenkron olarak SIGCHLD sinyalini dinleyerek çocuklarını temizlemelidir (reaping).*
>
> *Orphan Process (Yetim Süreç):*
>
> *Kendisinden önce ebeveyni ölmüş (veya sonlanmış) olan, hala çalışmaya devam eden çocuk süreçlerdir.*
>
> *Oluşma Sebebi: Ebeveyn sürecin çökmesi veya çocuğunu beklemeden çalışmasını bitirip kapanmasıdır.*
>
> *Tehlikesi ve Önlem: İşletim sistemleri yetim süreçleri tehlikeli bir durum olarak görmez. Bir süreç yetim kaldığında, sistemin en tepesindeki init (PID 1) veya modern sistemlerde systemd süreci tarafından otomatik olarak evlat edinilir (adopted). İşleri bittiğinde init süreci onlar için wait() çağrısı yapar ve zombi olmalarını engeller. Hatta arka planda çalışan servisler (Daemon'lar) bilerek yetim bırakılarak terminalden koparılır."*

2. Detaylı Arka Plan

Bu konu OS mülakatlarının en klasik senaryo sorularından biridir. Mülakatçı tanımları geçtikten sonra sana sistem yöneticisi veya C programcısı şapkası taktırıp "Bu sorunu nasıl çözersin?" diyecektir.

Zombie Process Derinlemesine Analiz

Mülakat Tuzağı: "Sistemde 100 tane Zombi süreç var. Terminali açtın ve kill -9 <Zombi_PID> yazdın. Zombi ölür mü?"

Kritik Cevap: "Hayır, ölmez. Çünkü bir ölüyü tekrar öldüremezsiniz."

Zombi süreç zaten exit() yapmıştır, CPU'da çalışmıyordur, bu yüzden kill sinyallerine cevap vermez. Sistemde tutulmasının tek sebebi, ebeveyninin "Benim çocuğum başarılı mı oldu (exit 0) yoksa hata koduyla mı patladı?" sorusunun cevabını tutan PCB'deki (Süreç Kontrol Bloğu) o ufacık bilgidir.

Zombilerden Kurtulma Yöntemleri (Kod Seviyesi):

- **Senkron Bekleme:** Ebeveyn kodunun içine wait() veya waitpid() yazmak. (Sorun: Ebeveyn bu satırda bloke olur, çocuk bitene kadar başka iş yapamaz).
- **Asenkron Bekleme (En İyi Pratik):** İşletim sistemi, bir çocuk öldüğünde ebeveyne SIGCHLD adında bir sinyal gönderir. Ebeveyn koda bir sinyal yakalayıcı (Signal Handler) yazarak, sadece bu sinyal geldiğinde arka planda wait() çağırıp çocuğu temizler.
- **Double Fork (İki Kere Çatallanma) Hilesi:** Ebeveyn bir çocuk yaratır. O çocuk anında bir torun yaratıp kendi kendini öldürür. Torun asıl işi yapar. Aradaki çocuk öldüğü için torun Yetim (Orphan)

kalır ve init sürecine devredilir. Böylece ebeveyn torunla ilgilenmek zorunda kalmaz, zombi riski sıfırlanır.

Zombilerden Kurtulma Yöntemleri (Sistem Yöneticisi Seviyesi):

Eğer kod senin değilse ve sistemde zombiler birikiyorsa, o zombiyi yaratan ebeveyn süreci bulup onu öldürmelisin (kill -9 <Parent_PID>). Ebeveyn ölünce, zombi çocuk Yetim statüsüne düşer, init sürecine geçer ve init onu saniyeler içinde temizler.

Orphan Process Derinlemesine Analiz ve Daemon'lar

Mülakatçı sorar: "Bir yetim süreç sistem için zararlı mıdır?"

Çoğu zaman hayır. İşletim sisteminin tasarımı bununla başa çıkmak üzerine kuruludur (PID 1'in evlat edinme mekanizması). Hatta biz programcılar, arka planda sonsuza kadar çalışacak bir servis (örneğin bir Web Sunucusu veya Veritabanı hizmeti) yazarken süreci bilerek yetim bırakırız.

Daemon (Arka Plan Servisi) Yaratmak:

Bir terminal açtın ve bir komut çalıştırdın. Eğer terminali kapatırsan, terminal ebeveyn olduğu için çocuk sürecine de SIGHUP (kapanma) sinyali gider ve programın kapanır.

Eğer yazdığın programın terminalden bağımsız, arka planda bir hayalet gibi yaşamasını istiyorsan programı Daemon yapmalısın. Bunun ilk adımı şudur: Program fork() ile bir çocuk yaratır ve ebeveyn hemen exit() ile intihar eder. Çocuk yetim kalıp init'e bağlanır ve artık senin terminaline bağlı olmadığı için arka planda özgürce çalışmaya devam eder.

Concurrency (Eşzamanlılık)

> **SORU:** *Concurrency kavramını anlatabilir misiniz*

1. Mülakat Cevabı

> *"Concurrency (Eşzamanlılık), bir sistemin aynı zaman dilimi içerisinde birden fazla görevi (task) yönetebilme ve ilerletebilme yeteneğidir. Yazılımın bağımsız çalışabilen parçalara bölünerek yapılandırılmasıdır.*
>
> *Mülakatlarda sıkça karıştırılan Parallelism (Paralellik) ile arasındaki temel fark şudur: Paralellik, birden fazla işin fiziksel olarak tam aynı anda (farklı CPU çekirdeklerinde) çalışmasıdır. Concurrency ise işlerin aynı anda çalışması zorunluluğunu taşımaz; tek çekirdekli bir işlemcide bile görevler arasında çok hızlı geçişler (context switch) yapılarak kullanıcıya işler aynı anda yapılıyormuş illüzyonu sunulur. Kısacası; Concurrency aynı anda birçok işle ilgilenmek (dealing with), Paralellik ise aynı anda birçok işi yapmaktır (doing)."*

2. Detaylı Arka Plan

Mülakatçı temel tanımı aldıktan sonra, "Peki tek çekirdekli bir sistemde eşzamanlılığı nasıl sağlıyoruz?" veya "Eşzamanlı programlamanın getirdiği riskler nelerdir?" gibi derinleştirici sorular soracaktır.

İşletim Sistemi Concurrency'i Nasıl Sağlar? (Time-Slicing)

Tek çekirdekli bir CPU aynı anda fiziksel olarak sadece 1 makine komutunu işleyebilir. İşletim sistemi concurrency'i donanımsal zamanlayıcılar (hardware timer) ve bir CPU Scheduler (İşlemci Zamanlayıcı) ile sağlar.

- **Timer Interrupt (Zamanlayıcı Kesmesi):** Sistem saati her birkaç milisaniyede bir donanımsal kesme üretir.
- **Zaman Dilimleme (Time-Slicing):** İşletim sistemi bu kesmeyi yakalar, o an çalışan Thread'i durdurur (preemption).
- **Context Switch:** Durdurulan Thread'in CPU yazmaçlarındaki (register) son durumu belleğe (TCB - Thread Control Block) kaydedilir. Sıradaki Thread'in durumu CPU'ya yüklenir ve çalışmaya başlar.

Bu işlem saniyede yüzlerce kez olduğu için biz bilgisayarda Spotify dinlerken aynı anda kod derleyebildiğimizi sanırız.

Concurrency vs. Parallelism Karşılaştırması

Bu ayrım, dağıtık sistemler ve OS mülakatlarının en klasik tuzaklarından biridir. Golang'in yaratıcılarından Rob Pike'ın mottosunu hatırlamak hayat kurtarır.

Özellik	Concurrency (Eşzamanlılık)	Parallelism (Paralellik)
Odak Noktası	Programın yapısı ve tasarımı (Structure)	Programın çalıştırılması (Execution)
Donanım İhtiyacı	Tek çekirdek (Single-core) yeterlidir	Çoklu çekirdek (Multi-core) zorunludur
Nasıl Çalışır?	Görevler CPU'yu sırayla ve bölüşerek kullanır (Interleaving)	Görevler farklı donanımlarda aynı anda işlenir

Eşzamanlılığın Karanlık Yüzü: Senkronizasyon Sorunları

Concurrency bedavaya gelmez. İşleri parçalara ayırıp belleği paylaştırdığınız an (özellikle Thread'ler ile), mülakatçıların sizden çözmenizi bekleyeceği şu klasik sorunlar ortaya çıkar:

- **Race Condition (Yarış Durumu):** İki veya daha fazla thread'in paylaşılan bir veriyi (örneğin banka hesabı bakiyesi) aynı anda okuyup değiştirmeye çalışmasıdır. İşlemlerin sırasına bağlı olarak sonuç yanlış çıkar.
- Çözüm: Ortak belleğe erişilen kod bloklarına Critical Section denir. Bu bölgeler Mutex (Mutual Exclusion) veya Semaphore gibi kilit (lock) mekanizmalarıyla korunmalıdır.
- **Deadlock (Kilitlenme):** İki thread'in de çalışmak için birbirlerinin elindeki kaynağı (kilidi) sonsuza kadar beklemesi durumudur (A, B'yi bekler; B, A'yı bekler).
- **Starvation (Açlık):** Düşük öncelikli bir thread'in, CPU sürekli yüksek öncelikli thread'lere tahsis edildiği için asla çalışma sırası bulamamasıdır.

Race Condition (Yarış Durumu)

> **SORU:** *race condition nedir anlatır mısın ve tüm çözüm yöntemlerini sıralar mısın*

1. Mülakat Cevabı

> *"Race Condition (Yarış Durumu), çoklu iş parçacığı (multi-threading) ortamında, birden fazla thread'in paylaşılan bir veriye aynı anda erişip onu değiştirmeye çalışması sonucu ortaya çıkan mantıksal bir hatadır. İşlemlerin sonucu, işletim sisteminin bu thread'leri CPU'da hangi sırayla çalıştırdığına (zamanlamasına) bağlı olarak rastgele ve yanlış çıkar.*
>
> *Bu sorunu çözmek için temel strateji, paylaşılan veriye erişilen kod bloğunu (Critical Section) koruma altına almaktır. Başlıca çözüm yöntemleri şunlardır:*
>
> *Kilitleme (Locking) Mekanizmaları: Mutex, Semaphore ve Monitor (Java'daki synchronized gibi) kullanarak kod bloğuna aynı anda sadece tek bir thread'in girmesini sağlamak.*
>
> *Atomik Operasyonlar (Lock-Free Programlama): Donanım destekli CAS (Compare-And-Swap) komutları kullanarak, değişkenin okunması ve güncellenmesi işlemini bölünemez tek bir adıma indirmek.*
>
> *Mimari Çözümler: Veriyi paylaşmak yerine iletişim kurmak (Go kanalları, Actor modeli), Thread-Local Storage (her thread'e verinin kopyasını vermek) veya Immutability (veriyi tamamen değiştirilemez yapmak)."*

2. Detaylı Arka Plan

Mülakatçı senden bu sorunu kod seviyesinde kanıtlamanı isteyecektir. "Neden bakiye++ işlemi tehlikelidir?" sorusu bu konunun kalbidir.

Race Condition Nasıl Oluşur? (Banka Örneği)

Yüksek seviyeli bir dilde yazdığın tek satırlık bakiye++ (veya bakiye = bakiye + 1) kodu, işlemci (CPU) seviyesinde 3 ayrı makine komutuna (Assembly) bölünür:

- **READ:** Bellekteki mevcut bakiyeyi CPU yazmacına (register) oku.
- **MODIFY:** Yazmaçtaki değeri 1 artır.
- **WRITE:** Yeni değeri belleğe geri yaz.

Diyelim ki hesapta 100 TL var ve iki thread aynı anda bakiye++ yapmak istiyor. İdeal sonuç 102 olmalıdır, ancak zamanlama (Context Switch) şu şekilde gerçekleşirse felaket olur:

- Thread 1 (READ): Bakiyeyi okur (100).
- (İşletim sistemi Thread 1'i durdurur, Thread 2'ye geçer)
- Thread 2 (READ): Bakiyeyi okur (100).
- Thread 2 (MODIFY & WRITE): Değeri 1 artırır (101) ve belleğe yazar. Bakiye 101 oldu.
- (İşletim sistemi tekrar Thread 1'e döner)
- Thread 1 (MODIFY & WRITE): Kendi yazmacında kalan 100 değerini 1 artırır (101) ve belleğe yazar.

Sonuç: Bakiye 101 oldu. İşlemlerden biri kayboldu (Lost Update)!

Tüm Çözüm Yöntemleri (Junior'dan Senior'a Doğru)

Mülakatlarda sadece "Mutex kullanırım" demek yetmez. Projenin ihtiyacına göre diğer mimari yöntemleri de sayabilmek seni öne çıkarır.

1. Kilitleme Mekanizmaları (Pessimistic - Kötümser Yaklaşım)

Bu yöntemler, "Kesin bir yarış olacak, ben şimdiden kapıyı kilitleyeyim" mantığıyla çalışır.

- **Mutex (Mutual Exclusion):** Sadece tek bir thread'in paylaşılan veriyi değiştirmesine izin verir. Yukarıdaki 3 adımlık işlemi (Read-Modify-Write) tek bir blok haline (Critical Section) getirir.
- **Monitor / synchronized:** OOP dillerinde Mutex'in dile entegre edilmiş, kullanımı daha güvenli halidir.
- **Read/Write Locks:** Eğer veriyi 100 thread okuyor ama sadece 1 thread değiştiriyorsa, standart Mutex performansı öldürür. R/W kilitleri, aynı anda birden fazla okuyucuya izin verirken, yazma işlemi yapılacağı zaman herkesi durdurur (Okuma ağırlıklı sistemler için mükemmeldir).

2. Donanım Seviyesi Çözümler (Lock-Free & Optimistic Yaklaşım)

Kilitleme mekanizmaları Context Switch maliyeti yarattığı için performansı düşürür. Yüksek performanslı sistemlerde (örneğin işletim sistemi çekirdekleri veya oyun motorları) kilit kullanılmaz.

- **Atomic Types / İşlemler:** C++'taki std::atomic<int> veya Java'daki AtomicInteger gibi yapılar kullanılır.
- **Nasıl Çalışır? (CAS - Compare And Swap):** Donanım (CPU), bu artırma işlemini 3 parçada değil, donanımsal düzeyde bölünemez (atomic) tek bir komut olarak işler. Bir thread güncellemeyi yaparken değeri kontrol eder; eğer başka biri değiştirmişse işlemi kilit beklemeden baştan dener.

3. Mimari ve Tasarımsal Çözümler (Modern Yaklaşımlar)

En iyi kilit, hiç ihtiyaç duyulmayan kilittir. Veriyi paylaştırmazsan, yarış da olmaz.

- **Immutability (Değişmezlik):** Fonksiyonel programlama dillerinin (Haskell, Scala) veya modern yapıların temelidir. Bir nesne yaratıldıktan sonra asla değiştirilemez. Eğer değer değişecekse, verinin güncel halini barındıran tamamen yeni bir kopya yaratılır. Veri değiştirilemediği için Race Condition oluşması imkansızdır.
- **Thread-Local Storage (TLS):** Paylaşılan tek bir global sayaç tutmak yerine, her thread'e kendi sayacını verirsiniz. Thread'ler kendi sayaçlarını kilit olmadan (çok hızlı) günceller. Bütün thread'lerin işi bittiğinde, ana thread (main) bu yerel sayaçların sonucunu toplar. (Özellikle MapReduce mantığında kullanılır).
- **Message Passing (Mesajlaşma):** Bellek paylaşmak yerine, thread'ler birbirlerine mesaj gönderir. Golang'in meşhur mottosu buradan gelir: "Do not communicate by sharing memory; instead, share memory by communicating." Go kanalları (Channels) veya Erlang'in Actor Modeli bu mimariye örnektir; verinin sahibi tek bir "Actor" veya kanaldır, işlemler sıraya girer.

Race Condition Oluşturan Koşullar

> **SORU:** *Race condition oluşturan koşullar nelerdir*

1. Mülakat Cevabı

> *"Bir sistemde Race Condition (Yarış Durumu) oluşabilmesi için üç temel koşulun aynı anda gerçekleşmesi gerekir:*
>
> *Paylaşılan Kaynak (Shared Resource): Birden fazla thread veya process'in aynı bellek alanına (global bir değişken, heap'teki bir nesne, bir dosya veya donanım aygıtı) erişiyor olması gerekir.*
>
> *Yazma/Değiştirme İşlemi (Mutation): Kaynağa erişen thread'lerden en az birinin o kaynağın değerini değiştiriyor (yazıyor) olması gerekir. (Eğer tüm thread'ler sadece 'okuma' yapıyorsa yarış durumu oluşmaz).*
>
> *Atomik Olmayan İşlemler (Non-Atomic Execution): Paylaşılan kaynağı okuma, değiştirme ve geri yazma işlemlerinin tek ve bölünemez bir adımda (atomik olarak) yapılmaması; araya işletim sisteminin Context Switch (bağlam değişimi) sokabilecek genişlikte bir Kritik Bölge (Critical Section) bırakılmasıdır."*

2. Detaylı Arka Plan

Mülakatçı bu 3 şartı duyduğunda konuya tam olarak hakim olduğunu anlar. Bu aşamadan sonra senden bu koşulları kırmanın yollarını (yani çözümleri) mimari açılardan tartışmanı isteyecektir.

Bu üç koşulun mantığını günlük hayat senaryoları ve bilgisayar mimarisi üzerinden inceleyelim.

1. Paylaşılan Kaynak Koşulu (Neden Lokal Değişkenlerde Hata Çıkmaz?)

Mülakatlarda adaylara şu kod verilir ve "Burada Race Condition var mı?" diye sorulur:

```c
void hesapla() {
    int lokal_sayac = 0; // Stack (Yığın) üzerinde yaşar
    lokal_sayac++;
}
```

Cevap: Kesinlikle hayır. İsterse 10.000 thread aynı anda bu fonksiyonu çağırsın, Race Condition oluşmaz.

Çünkü işletim sistemi, yaratılan her thread için bağımsız bir Stack (Yığın) alanı tahsis eder. lokal_sayac değişkeni Heap'te (ortak alan) veya BSS'te (global alan) değil, her thread'in kendi Stack'inde yaratılır. Paylaşım (1. koşul) sağlanmadığı için yarış da olmaz. Bu duruma Thread-Safe (İş parçacığı güvenli) Kod denir.

2. Yazma (Mutation) Koşulu ve Immutability

"Sadece okuma yapılıyorsa kilit (Mutex) kullanmalı mıyım?" sorusu mülakatlarda çok sık gelir.

Diyelim ki sistemde bir ayar dosyası veya "Pi sayısı" gibi global bir değişken var. 100 farklı thread aynı anda bu değişkeni okuyup hesaplama yapıyor.

Eğer hiçbir thread bu değeri değiştirmiyorsa (ikinci koşul sağlanmıyorsa), Race Condition oluşması fiziksel olarak imkansızdır. Kilit (Mutex) kullanmak sadece performansı düşürür.

Modern Mimari Çözümü: Fonksiyonel programlama dilleri (Haskell, Elixir vb.) veya modern yazılım prensipleri Immutability (Değişmezlik) kavramını savunur. "Bir nesneyi yarat, ama yaratıldıktan sonra değerini asla değiştirtme." Değiştirme (Mutation) koşulu kasten yok edildiği için sistem doğal olarak Thread-Safe hale gelir.

3. Atomik Olmayan İşlem Koşulu (Zamanlamanın Rolü)

Bir işlemin "Atomik" olması, onun sistem tarafından kesinlikle bölünemez tek bir adımda yapılması demektir.

Daha önce incelediğimiz meşhur bakiye++ işlemi atomik değildir çünkü donanım (CPU) seviyesinde 3 adıma bölünür (Read -> Modify -> Write).

Eğer İşletim Sistemi (CPU Scheduler), Thread 1 tam Read işlemini yapıp Modify aşamasına geçerken araya girip "Senin süren bitti (Context Switch), şimdi Thread 2 çalışacak" demezse, Race Condition gizli kalır. Zaten Race Condition'ın mülakatlarda bu kadar üzerine düşülmesinin sebebi budur: Hata her zaman oluşmaz. Kodu kendi bilgisayarında 100 kere çalıştırırsın doğru çıkar, canlı sunucuya (Production) atarsın, ayda 1 kez zamanlama (Context Switch) tam o 3 adımın arasına denk gelir ve hesap bakiyesi bozulur.

Bu koşulu kırma yöntemleri:

- **Kötümser (Pessimistic) Yaklaşım:** Bu 3 adımlık işlemi bir Mutex ile sararsın. İşletim sistemi araya girip Context Switch yapsa bile, diğer thread kilidi göreceği için o bloğa giremez.
- **İyimser (Optimistic) / Donanım Yaklaşımı:** İşlemciye (CPU) özel komutlar (Compare-And-Swap) göndererek, "Bu üç işlemi bana tek bir donanım clock cycle'ında (saat vuruşu) atomik olarak yap, araya işletim sistemi bile giremesin" dersin. (Lock-free programlama).

C'de Thread-Safe Singleton Pattern

› Deadlock Şartları Nelerdir?

› Veri Yapıları: Big O Notation

Mutex / Semaphore / Monitor

> **SORU:** *Mutex/Semaphore/Monitor kavramlarını açıklayabilir misiniz?*

1. Mülakat Cevabı

> *"Bu üç kavram, eşzamanlı (concurrent) programlamada paylaşılan kaynaklara güvenli erişim sağlamak için kullanılan senkronizasyon mekanizmalarıdır.*
>
> *Mutex (Mutual Exclusion - Karşılıklı Dışlama): Temelde paylaşılan bir kaynağa aynı anda sadece tek bir thread'in erişmesini sağlayan bir 'kilit' (lock) mekanizmasıdır. Tuvalet anahtarı gibi düşünebiliriz; anahtarı alan içeri girer, işi bitince anahtarı bırakır. Kilidi kim aldıysa, sadece o bırakabilir (Ownership).*
>
> *Semaphore (Semafor): Sinyalleşme (signaling) mekanizmasıdır. İçinde bir sayaç (counter) tutar ve bu sayaç kadar thread'in bir kaynağa erişmesine izin verir. Örneğin 3 değerine sahip bir semafor, bir havuza aynı anda en fazla 3 kişinin girmesine izin veren bir görevli gibidir. Mutex'ten en büyük farkı sahiplik (ownership) kavramının olmamasıdır; bir thread semaforun değerini düşürürken (wait), başka bir thread artırabilir (signal/post).*
>
> *Monitor (Monitör): Mutex ve Koşul Değişkenlerini (Condition Variables) birleştiren, genellikle nesne yönelimli dillerde (Java'daki synchronized gibi) dilin kendisine entegre edilmiş, kullanımı Mutex ve Semaphore'a göre çok daha güvenli ve yüksek seviyeli bir senkronizasyon yapısıdır."*

2. Detaylı Arka Plan

Mülakatlarda sadece bu tanımları yapmak yetmez. Adaylar genellikle Mutex ve Binary Semaphore'u birbirine karıştırır. Mülakatçı seni buradan vurmaya çalışacaktır.

Mutex (Mutual Exclusion)

Mutex, Critical Section (Kritik Bölge - paylaşılan verinin güncellendiği kod bloğu) koruması için yaratılmıştır.

- **Nasıl Çalışır:** Thread A, lock() çağrısı yapar ve Mutex'i alır. Thread B lock() çağrısı yaptığında Mutex'in başkası tarafından alındığını görür ve Mutex serbest kalana kadar işletim sistemi tarafından uyutulur (Blocked state). Thread A işini bitirince unlock() yapar ve Thread B uyandırılır.
- **En Önemli Kural (Mülakat Sorusu):** Mutex'te "Sahiplik" (Ownership) vardır. Yani Mutex'i hangi thread kilitlediyse, kilidi açma (unlock) yetkisi yalnızca ona aittir. Başka bir thread onun kilidini açamaz.

Semaphore (Semafor)

Semafor, Hollandalı bilgisayar bilimcisi Edsger Dijkstra tarafından icat edilmiştir. İki temel atomik operasyonu vardır: Wait() (veya P()) ve Signal() (veya V()).

- **Wait() / P() / down():** Semaforun sayacını 1 azaltır. Eğer sayaç 0 ise, thread uyutulur.
- **Signal() / V() / up():** Semaforun sayacını 1 artırır. Eğer uyuyan thread'ler varsa, birini uyandırır.
- **Kullanım Alanı:** Semaforlar daha çok Thread'ler arası sinyalleşme (örneğin Üretici-Tüketici / Producer-Consumer problemi) veya sınırlı kaynakların (örneğin sistemde sadece 5 adet bulunan veri tabanı bağlantısı) yönetimi için kullanılır.

Mülakat Tuzağı: Mutex vs. Binary Semaphore

Mülakatçı sorar: "Değeri maksimum 1 olabilen bir Binary Semaphore (İkili Semafor) ile Mutex aynı şey midir?"

Cevap: KESİNLİKLE HAYIR.

Görünüşte ikisi de 0 ve 1 değerlerini alır ve kritik bölgeyi koruyabilir. Ancak kritik fark Sahiplik (Ownership) kavramındadır.

- Mutex'te kilidi alan açmak zorundadır.
- Binary Semaphore'da ise bir thread Wait() yaparak sayacı 0'a çekip kritik bölgeye girdiğinde, sistemdeki tamamen alakasız başka bir thread Signal() çağrısı yaparak semaforu tekrar 1 yapabilir ve başka thread'lerin içeri girmesine sebep olabilir. Semaforlar sinyalleşme için tasarlandığından bu duruma izin verir, Mutex ise buna izin vermez ve hata fırlatır.

Monitor (Monitör)

Mutex ve Semaphore güçlüdür ama kullanımları tehlikelidir. Programcı bir Mutex'i kilitler (lock) ama hata yapıp açmayı (unlock) unutursa veya bir exception (istisna) fırlatılıp kod unlock satırına hiç ulaşamazsa sistem Deadlock'a (kilitlenmeye) gider.

Monitor, bu sorunu çözmek için geliştirilmiş yüksek seviyeli (high-level) bir yapıdır. Genellikle derleyiciler (compiler) veya dilin çalışma zamanı (runtime) tarafından desteklenir.

- Bir sınıfın veya metodun Monitor olarak tanımlandığını düşün. Bu sınıfa aynı anda sadece tek bir thread girebilir.
- Giriş ve çıkışlardaki kilit (lock) ve kilit açma (unlock) işlemleri geliştirici tarafından değil, dilin kendisi tarafından otomatik yapılır.
- İçerisinde Condition Variables (Koşul Değişkenleri) barındırır. Bu değişkenler sayesinde bir thread Monitor içine girdikten sonra belirli bir koşulun sağlanmasını beklemek üzere kendini uyutabilir (örneğin kuyruk boşsa veri gelmesini beklemek).

Gerçek Hayat Örneği: Java'daki synchronized anahtar kelimesi. Bir metodu synchronized yaptığında arka planda o nesne için bir Monitor mekanizması kurulur, lock/unlock işlemleriyle senin uğraşmana gerek kalmaz.

Mutex ve Semafor Farkı

> **SORU:** *mutex ve semafor farkı nedir ?*

1. Mülakat Cevabı

> *"Mutex ve Semaphore arasındaki en temel ve mülakat kazandıran fark Sahiplik (Ownership) kavramıdır.*
>
> *Mutex (Mutual Exclusion - Karşılıklı Dışlama): Bir paylaşılan kaynağa aynı anda sadece tek bir thread'in girmesini sağlayan bir 'kilit' (lock) mekanizmasıdır. Mutex'in katı bir kuralı vardır: Kilidi hangi thread aldıysa (lock), o kilidi açma yetkisi (unlock) SADECE o thread'e aittir.*
>
> *Semaphore (Semafor): Sınırlı sayıdaki kaynakların yönetimini sağlayan ve içinde bir sayaç (counter) barındıran bir 'sinyalleşme' (signaling) mekanizmasıdır. Semaforlarda sahiplik (ownership) kavramı yoktur. Bir thread semaforun sayacını düşürüp (wait/kilitlerken), sistemdeki tamamen başka bir thread semaforun sayacını artırıp (signal/kilidi açabilir) diğer thread'lere yol verebilir."*

2. Detaylı Arka Plan

Bu soru, eşzamanlı (concurrent) programlama mülakatlarının en klasik "tuzak" sorusudur. Mülakatçı yukarıdaki cevabı aldıktan sonra hemen o meşhur tuzağı kuracaktır.

Mülakat Tuzağı: Binary Semaphore vs. Mutex

Mülakatçı sorar: "Maksimum değeri 1 olan bir Semafor (Binary Semaphore) ile Mutex tamamen aynı şey değil midir?"

Cevap: Kesinlikle hayır.

Görünüşte ikisi de 0 ve 1 değerlerini alarak sadece 1 thread'in içeri girmesine izin verir (örneğin bir banka bakiyesini güncellerken). Ancak mimari amaçları tamamen farklıdır.

- **Mutex Koruma İçindir:** Diyelim ki bir tuvalettesin (Critical Section). İçeri girdin ve kapıyı kilitledin (Mutex Lock). Dışarıdaki birinin elinde sihirli bir düğme olup senin kapını sen içerideyken açması güvenli midir? Hayır. Mutex buna izin vermez. İşin bitince kapıyı sen açmalısın.
- **Semaphore Sinyal İçindir:** Telefonundan müzik dinlediğini düşün. Şarkı İndirici Thread ve Şarkı Çalıcı Thread var. Çalıcı thread'in çalışmak için şarkının inmesini beklemesi (Wait) lazım. İndirici thread şarkıyı indirdiğinde, Çalıcı thread'e "Hadi başla" diye sinyal gönderir (Signal). Yani kilidi bir thread koydu, başka bir thread açtı. Binary Semaphore işte bu sinyalleşme için yaratılmıştır.

Eğer Mutex Yerine Binary Semaphore Kullanırsak Ne Olur?

Bir veriyi güncellemek için yanlışlıkla Mutex yerine Binary Semaphore kullanırsan şu korkunç senaryo yaşanır:

- Thread A gelir, Semaforu kilitler (Değeri 0 yapar) ve banka bakiyesini güncellemeye başlar.
- Tam o sırada sistemdeki tamamen alakasız veya kötü yazılmış bir Thread B gelir.
- Thread B yanlışlıkla Semaforun Signal() (kilidi açma) fonksiyonunu çağırır.
- Semaforun değeri tekrar 1 olur!
- O sırada bekleyen Thread C, semaforu 1 gördüğü için içeri dalar ve Thread A daha işini bitirmeden aynı banka bakiyesini değiştirmeye başlar.

Sonuç: Veri tabanı bozulur (Race Condition). Eğer bunu Mutex ile yapsaydın, Thread B başkasının Mutex'ini açmaya çalıştığı an işletim sistemi hata (exception) fırlatırdı ve veriyi korurdu.

Karşılaştırma Özeti

Özellik	Mutex	Semaphore
Temel Amacı	Ortak veriyi korumak (Locking mekanizması)	Thread'ler arası sinyalleşme (Signaling mekanizması)
Sahiplik (Ownership)	VAR. (Kim kilitlediyse o açar)	YOK. (Herkes artırıp azaltabilir)
Kapasite	Aynı anda sadece 1 thread	Sayaç değeri kadar (Örn: N adet thread)
Gerçek Hayat Örneği	Umumi tuvalet anahtarı	İçeriye en fazla 50 araba alan otopark görevlisi

Producer-Consumer problemi çözümü

› *Deadlock (Kilitlenme) senaryosu oluştur*

› *Thread-Safe Singleton Pattern*

Semafor Hangi Problem İçin Bir Çözüm Oluşturmuştur

> **SORU:** *semafor hangi problem için bir çözüm oluşturmuştur*

1. Mülakat Cevabı

> *"Semafor (Semaphore), 1965 yılında ünlü bilgisayar bilimcisi Edsger Dijkstra tarafından, eşzamanlı (concurrent) programlamadaki iki temel problemi çözmek için icat edilmiştir: Sınırlı kaynakların yönetimi ve Süreçler/İş Parçacıkları arası senkronizasyon (sinyalleşme).*
>
> *Semaforun çözdüğü temel problemler şunlardır:*
>
> *Kaynak Havuzu Problemi (Counting Semaphore): Mutex sadece tek bir kaynağı (1 veya 0) korurken, semaforlar 'sistemde aynı kaynaktan N tane var' problemini çözer. Örneğin, bir veritabanı bağlantı havuzunda (connection pool) tam 5 adet boş bağlantı varsa, 5 değerine sahip bir semafor bu havuzun güvenli bir şekilde kullanılmasını ve limitin aşılmamasını sağlar.*
>
> *Sinyalleşme ve İşlem Sırası (Signaling/Synchronization): Bir sürecin çalışmaya başlamak için başka bir sürecin işini bitirmesini beklemesi gereken durumları (Örneğin; Üretici-Tüketici problemi) çözer.*
>
> *Busy-Waiting (Meşgul Bekleme) İsrafını Önleme: Semaforlar icat edilmeden önce, bir kaynağı bekleyen süreçler while(kilitli_mi) {} şeklinde sonsuz bir döngüde dönerek CPU'yu %100 meşgul ederdi. Semaforlar, bekleyen süreçleri işletim sistemi seviyesinde uyutarak (Blocking/Sleep) CPU israfını ortadan kaldırmıştır."*

2. Detaylı Arka Plan

Mülakatçı sen Dijkstra'nın adını andığında ve "Busy-Waiting" kavramından bahsettiğinde doğru yolda olduğunu anlayacaktır. Ancak teknik mülakatlarda bu konuyu akademik bir problem üzerinden kanıtlaman beklenir.

Semaforların neyi çözdüğünü anlamak için, "Semafor olmasaydı ne olurdu?" sorusuna bakmalıyız.

Semafor Öncesi Dönem: Spinlock (Meşgul Bekleme) Problemi

Semaforlar (ve Mutex'ler) standartlaşmadan önce, bir thread başka bir thread'in işini bitirmesini beklemek için şu tarz bir kod yazardı:

```c
// Thread B (Tüketici)
while(veri_hazir_degil) {
    // Hiçbir şey yapma, sadece bekle. (Busy Waiting)
}
veriyi_isle();
```

Bu koda Spinlock (Kendi etrafında dönen kilit) denir. Thread B, veri_hazir_degil değişkeni değişene kadar CPU'da sürekli döner. Eğer işletim sistemi Thread B'ye 100 milisaniye çalışma süresi (Time-slice) verdiyse, Thread B bu 100 milisaniyeyi saniyede milyonlarca kez "Veri hazır mı?" diye sorarak boşa harcar. CPU ısınıp enerji tüketirken hiçbir faydalı iş yapılmaz.

Semaforun Çözümü:

Dijkstra'nın icat ettiği Semafor yapısı, işletim sisteminin Çekirdek (Kernel) mimarisiyle entegredir. Bir thread semafor üzerinde Wait() (veya P) fonksiyonunu çağırdığında ve kaynak yoksa, işletim sistemi o thread'i uyutur (Blocked State). Thread CPU'dan atılır ve işlemci gücü boşa harcanmaz. Kaynak serbest kaldığında işletim sistemi uyuyan thread'i uyandırır (Wake up).

Semaforun Doğuş Sebebi: Üretici-Tüketici (Producer-Consumer) Problemi

Mülakatlarda "Semafor hangi problemi çözer?" sorusunun en uygulamalı cevabı Üretici-Tüketici (Bounded-Buffer) Problemidir.

Diyelim ki sabit boyutlu bir veri kuyruğumuz (Buffer) var.

- **Üretici (Producer):** Kuyruğa veri ekler. Kuyruk doluysa beklemek zorundadır.
- **Tüketici (Consumer):** Kuyruktan veri alır. Kuyruk boşsa veri gelene kadar beklemek zorundadır.

Eğer bu işi sadece Mutex ile çözmeye kalkarsan, sadece aynı anda kuyruğa yazmalarını engellersin (Race condition'ı çözersin). Ama "Kuyruk boşsa Tüketici uyusun, Üretici veri ekleyince Tüketiciyi uyandırsın" mantığını (Sinyalleşmeyi) sadece Mutex ile kuramazsın.

İşte Semafor bu koordinasyon problemini zarifçe çözer:

- Semafor_BosYerler = Kuyruk Boyutu (Örn: 10)
- Semafor_DoluYerler = 0
- Üretici veri üretince Semafor_BosYerler'i 1 azaltır, Semafor_DoluYerler'i 1 artırır (Signal).
- Tüketici veri okumak için Semafor_DoluYerler'i bekler (Wait). Eğer değer 0 ise, tüketici otomatik olarak uykuya dalar. Üretici ilk veriyi koyup semaforu 1 yaptığında tüketici uyanır ve veriyi alır.

İşletim Sistemi ve Donanım Desteği (Atomicity)

Mülakatçı şu zorlayıcı soruyu sorabilir: "Peki Semaforun içindeki o 'sayaç' değişkenini artırıp azaltırken Race Condition (Yarış durumu) oluşmuyor mu?"

Kritik Cevap: "Oluşmaz, çünkü Semaforun içindeki Wait() ve Signal() işlemleri işletim sistemi ve CPU (donanım) tarafından Atomik (Bölünemez) olarak garanti edilir."

Eğer sen normal bir sayac++ işlemi yaparsan bu 3 makine komutudur ve araya başka thread girebilir. Ancak semafor işlemleri, CPU üzerindeki özel Test-and-Set veya Compare-and-Swap (CAS) gibi donanım komutlarıyla desteklenir. Yani bir semaforun değeri düşürülürken (Wait), işletim sistemi o ufacık an için tüm kesmeleri (interrupts) kapatır veya donanımsal kilit atar. Bu da semaforun kendisinin asla bozulmamasını sağlar.

Producer-Consumer Kod Örneği

› *Spinlock vs Mutex/Semaphore*

› *Dining Philosophers (Yemek Yiyen Filozoflar)*

Atomik Değişken

> **SORU:** *atomik değişken nedir? hangi senaryolarda kullanılır neden?*

1. Mülakat Cevabı

> *"Atomik değişken, üzerindeki işlemlerin (okuma, değiştirme ve yazma) başka hiçbir iş parçacığı (thread) veya işletim sistemi tarafından kesintiye uğratılamadan, bölünemez tek bir adımda (indivisible) gerçekleştiği özel bir veri tipidir.*
>
> *Normal bir değişkende sayac++ işlemi arka planda üç adımdan oluşur (Oku, Artır, Yaz) ve bu adımlar arasına başka bir thread sızarak veriyi bozabilir (Race Condition). Atomik değişkenlerde ise bu üç adım, CPU'nun donanımsal yetenekleri (örneğin Compare-And-Swap) kullanılarak tek bir saat vuruşunda (clock cycle) güvenle halledilir.*
>
> *En yaygın kullanım senaryoları; yüksek performans gerektiren eşzamanlı (multithreaded) sayıcılar (counters), thread'ler arası sinyalleşme bayrakları (flags) ve Mutex gibi kilitlerin yarattığı performans kaybını (context switch) ortadan kaldırmak için yazılan Lock-Free (Kilitsiz) veri yapılarının inşasıdır."*

2. Detaylı Arka Plan

Multithreading (çoklu iş parçacığı) mülakatlarında, adayın işletim sistemi ile donanım (CPU) arasındaki köprüyü ne kadar anladığını ölçmek için Mutex ve Atomik değişkenler sıkça karşılaştırılır.

Problemin Kaynağı: x++ Neden Tehlikeli?

Diyelim ki bir web sunucusu yazdın ve gelen ziyaretçi sayısını tutmak istiyorsun: int ziyaretci = 0;

İki farklı ziyaretçi aynı milisaniyede siteye girdiğinde ve iki thread aynı anda ziyaretci++ kodunu çalıştırdığında sayacın 2 olması beklenir. Ancak normal bir değişkende bu işlem CPU seviyesinde 3 adımdır:

- **Read (Oku):** Ziyaretçi sayısını (0) CPU register'ına (yazmacına) al.
- **Modify (Değiştir):** Değeri 1 artır.
- **Write (Yaz):** Yeni değeri (1) RAM'e geri yaz.

Eğer Thread-A 1. adımı yapıp 2. adıma geçerken işletim sistemi onu durdurur (Context Switch) ve Thread-B'yi çalıştırırsa, Thread-B de eski değeri (0) okur. Sonuç olarak iki thread de sırayla RAM'e "1" yazar. Bir ziyaretçi kaybolmuştur.

Bunu çözmek için std::atomic<int> ziyaretci; (C++) veya AtomicInteger (Java) kullandığınızda, işlemciye şu donanımsal emri verirsiniz: "Bu 3 adımı benim için tek bir atomik işlem olarak yap. Araya kimse girmesin."

Hangi Senaryolarda Kullanılır? Neden?

Atomik değişkenler her derde deva değildir; ancak doğru senaryoda Mutex'e göre devasa bir performans artışı sağlarlar.

1. Yüksek Performanslı Sayıcılar (Counters) ve Metrikler

- **Senaryo:** Saniyede milyonlarca isteğin geldiği bir sistemde (örneğin bir oyun sunucusunda) anlık bağlı oyuncu sayısını tutmak.
- Neden Atomik? Bunu bir Mutex ile korursanız, sayacı her artırmak isteyen thread kilit sırasına girer. Bekleyen thread'ler işletim sistemi tarafından uyutulur (Sleep) ve uyandırılır. Bu (Context Switch) inanılmaz maliyetlidir. Atomik değişken CPU seviyesinde çözüldüğü için thread'ler uyutulmaz, işlemler mikrosaniyeler içinde biter.

2. Durum Bayrakları (State Flags / Cancellation Tokens)

- **Senaryo:** Arka planda sonsuz bir döngüde (while(is_running)) çalışan bir Worker Thread'i, ana thread'den (Main Thread) güvenli bir şekilde durdurmak istiyorsunuz.
- Neden Atomik? is_running değişkenini boolean (bool) bir atomik bayrak (std::atomic<bool>) yaparsınız. Main thread bu bayrağı false yaptığı anda, Worker thread bunu anında ve güvenle okuyup işini bitirir. Bunun için ağır bir Mutex mekanizması kurmak gereksiz bir israftır.

3. Lock-Free (Kilitsiz) Veri Yapıları Tasarlamak

- **Senaryo:** Mutex kullanmadan, tamamen thread-safe (iş parçacığı güvenli) olan bir Kuyruk (Queue) veya Yığın (Stack) mimarisi tasarlamak.
- Neden Atomik? Finans (Borsa/HFT - High Frequency Trading) uygulamaları gibi mikrosaniyelerin bile önemli olduğu yerlerde Mutex (kilit) kabul edilemez çünkü bir thread'in uyuması veya kilitlenmesi (Deadlock) felakettir. Bu sistemler, atomik pointer'lar kullanılarak tasarlanır.

Kıyaslama: Mutex vs Atomik Değişken

Özellik	Mutex (Kilit)	Atomik Değişken
Kapsam	Geniş. Onlarca satırlık bir kod bloğunu koruyabilir.	Dar. Sadece tek bir değişkeni (int, bool, pointer) korur.
Mekanizma	İşletim Sistemi (OS) seviyesinde çalışır. Bekleyen thread'i uyutur.	Donanım (CPU) seviyesinde çalışır. Thread uyumaz.
Performans	Daha yavaştır (Context Switch maliyeti vardır).	Çok hızlıdır (Doğrudan işlemci komutudur).
Deadlock Riski	Yanlış kullanımda sistem kilitlenebilir (Deadlock).	Kilit olmadığı için Deadlock riski yoktur.

CAS (Compare-And-Swap) Nedir?
› Memory Ordering (Bellek Sıralaması)
› Lock-Free Programlama Zorlukları

Deadlock (Kilitlenme)

> **SORU:** *Deadlock nedir ? Oluşma durumları nelerdir ? Deadlock önüne geçmek için ne yapmalıyız*

1. Mülakat Cevabı

> *"Deadlock (Kilitlenme), eşzamanlı çalışan iki veya daha fazla thread'in (veya process'in) çalışmaya devam edebilmek için birbirlerinin elinde tuttukları kaynakları (kilitleri) sonsuza kadar beklemesi durumudur. Sistem bir tür 'Meksika Açmazı'na girer ve hiçbir işlem ilerleyemez.*
>
> *Deadlock oluşması için dört şartın (Coffman Koşulları) aynı anda gerçekleşmesi gerekir:*
>
> *Mutual Exclusion (Karşılıklı Dışlama): Kaynakların paylaşılamaz olması (aynı anda sadece 1 thread kullanabilir).*
>
> *Hold and Wait (Tut ve Bekle): Bir thread'in elinde en az bir kaynak tutarken, başkasının elindeki başka bir kaynağı beklemesi.*
>
> *No Preemption (Zorla Almama): Bir kaynağın, onu tutan thread işini bitirene kadar elinden zorla (OS tarafından) alınamaması.*
>
> *Circular Wait (Döngüsel Bekleme): Thread'lerin dairesel bir zincir şeklinde birbirlerini beklemesi (A -> B'yi, B -> C'yi, C -> A'yı bekler).*
>
> *Deadlock'u önlemek için bu dört şarttan en az birini kırmamız gerekir. Mülakatlarda en sık sorulan ve en pratik yöntem, kilitleri her zaman aynı sırayla (aynı hiyerarşide) alarak 'Döngüsel Bekleme' (Circular Wait) şartını kırmaktır."*

2. Detaylı Arka Plan

Deadlock konusu mülakatlarda bir senaryo üzerinden sorulur. Mülakatçı sana iki fonksiyon verir ve "Bu kod neden kilitleniyor ve nasıl düzeltirsin?" der.

Deadlock Senaryosu (Klasik Banka Transferi)

İki farklı müşterinin banka hesapları arasında aynı anda para transferi yaptığımızı düşünelim. İki hesabın da bakiyesini (ortak kaynak) Mutex ile koruyoruz.

```c
// Thread 1: Hesab A'dan Hesap B'ye para transferi
transfer(Account* A, Account* B, int miktar) {
    lock(A->mutex);       // 1. Thread 1, A'nın kilidini aldı
    lock(B->mutex);       // 2. Thread 1, B'nin kilidini bekliyor
    // İşlemler...
    unlock(B->mutex);
    unlock(A->mutex);
}

// Thread 2: Hesap B'den Hesap A'ya aynı anda para transferi
transfer(Account* B, Account* A, int miktar) {
    lock(B->mutex);       // 3. Thread 2, B'nin kilidini aldı
    lock(A->mutex);       // 4. Thread 2, A'nın kilidini bekliyor
    // İşlemler...
    unlock(A->mutex);
    unlock(B->mutex);
}
```

Ne oldu? Thread 1 A'yı kilitledi, Thread 2 B'yi kilitledi. Şimdi Thread 1 B'yi bekliyor, Thread 2 ise A'yı bekliyor. İşletim sistemi bu iki thread'i de sonsuza kadar uyutur. Al sana Deadlock.

Deadlock Önleme (Prevention) ve Kaçınma (Avoidance) Stratejileri

İşletim sistemleri ve programcılar deadlock ile başa çıkmak için çeşitli yöntemler kullanır. Mülakatlarda bu yöntemlerin uygulanabilirliğini tartışman beklenir.

1. Coffman Şartlarını Kırmak (Deadlock Prevention)

Programcı olarak Deadlock'u önlemenin en garanti yolu kodun yapısını değiştirmektir.

- Döngüsel Beklemeyi (Circular Wait) Kırmak (En popüler çözüm): Kilitlere bir hiyerarşi veya sıra numarası verirsiniz. Bütün thread'ler kilitleri her zaman aynı sırayla (örneğin hafıza adreslerine göre küçükten büyüğe) almak zorundadır. Yukarıdaki örnekte, kilitleri her zaman id'si küçük olandan büyük olana doğru alsaydık:
- Thread 1: Önce A, sonra B'yi isterdi.
- Thread 2: Önce A, sonra B'yi isterdi.
- Sonuç: Döngüsel bekleme bozulur, deadlock asla oluşmaz.
- **Tut ve Bekle (Hold and Wait) Şartını Kırmak:** Bir thread çalışmaya başlamadan önce ihtiyacı olan tüm kilitleri aynı anda ister. Hepsini alabiliyorsa çalışır, birini bile alamıyorsa diğerlerini de bırakır ve en baştan dener. (Verimsizdir, performansı düşürür).
- **Zorla Almama (No Preemption) Şartını Kırmak:** Eğer kilitleri alırken bir süre bekleyip alamazsanız (trylock() gibi fonksiyonlarla), elinizdeki mevcut kilitleri de bırakıp işlemi iptal eder ve rastgele bir süre sonra tekrar denersiniz (Livelock riskine dikkat!).

2. İşletim Sistemi Seviyesinde Kaçınma (Deadlock Avoidance)

İşletim sistemleri bazen kaynakları tahsis etmeden önce durumun güvenli olup olmadığını kontrol eder.

- **Banker's Algorithm (Banker Algoritması):** Mülakatlarda ismi sıkça geçen bu algoritma, Dijkstra tarafından geliştirilmiştir. OS, bir thread kaynak (kilit, bellek vs.) istediğinde sistemi simüle eder. Eğer bu kaynağı verirsem gelecekte bir deadlock oluşma riski var mı diye bakar. Eğer sistem Güvenli Durumda (Safe State) kalacaksa kaynağı verir, Güvensiz Duruma (Unsafe State) geçecekse thread'i bekletir. (Gerçek hayatta implemente etmesi çok maliyetli olduğu için modern işletim sistemleri tarafından pek kullanılmaz).

3. Ostrich Algorithm (Devekuşu Algoritması)

Mülakatçı gülümseyerek "Windows veya Linux deadlock'u çözmek için ne yapar?" diye sorabilir.

Cevap trajikomiktir: Hiçbir şey yapmazlar. Unix, Linux ve Windows genel olarak deadlock yönetimini yazılımcıya bırakır. Eğer bir sistemde deadlock olursa sistem kitlenir (veya o process kitlenir), kullanıcı Task Manager (Görev Yöneticisi) veya kill komutu ile süreci sonlandırır. Bu duruma başını kuma gömen devekuşundan esinlenilerek Ostrich Algorithm denir. İşletim sistemleri, deadlock'u tespit etmenin getireceği performans maliyetine katlanmaktansa, nadiren olan bu durumu görmezden gelmeyi seçer.

Starvation (Açlık)

> **SORU:** *Starvation nedir açıkla*

1. Mülakat Cevabı

> *"Starvation (Açlık), eşzamanlı (concurrent) çalışan bir sistemde, bir thread veya sürecin (process) çalışmak için ihtiyaç duyduğu kaynaklara (örneğin CPU, bellek veya kilitlere) sürekli olarak diğer thread'lerin erişmesi nedeniyle asla erişememesi ve sonsuza kadar beklemek zorunda kalması durumudur.*
>
> *Deadlock'tan (kilitlenme) farkı şudur: Deadlock'ta sistemdeki hiçbir thread ilerleyemez, sistem tamamen kilitlenir. Starvation durumunda ise sistem çalışmaya devam eder, genel bir kilitlenme yoktur, ancak sadece bir veya birkaç şanssız (genellikle düşük öncelikli) thread sürekli göz ardı edildiği için görevini yerine getiremez."*

2. Detaylı Arka Plan

Mülakatlarda sadece tanım vermek yeterli değildir. Mülakatçı bu sorunu OS Scheduler (İşletim Sistemi Çizelgeleyici) mantığıyla bağdaştırmanı ve çözüm yöntemini (Aging) duymayı bekler.

Starvation Neden ve Nasıl Oluşur?

Starvation genellikle sistemde adil olmayan (unfair) bir kaynak dağılımı politikası uygulandığında ortaya çıkar. Bunun en klasik iki örneği şunlardır:

1. Öncelik Tabanlı Çizelgeleme (Priority Scheduling) Kaynaklı Açlık

İşletim sistemi CPU'yu thread'lere dağıtırken (Scheduling), her thread'in bir önceliği (Priority) olabilir.

- Diyelim ki sistemde bir tane çok düşük öncelikli bir yedekleme (backup) thread'i var.
- Sistem aynı zamanda sürekli olarak yüksek öncelikli kullanıcı arayüzü (UI) ve ağ işlemlerini kabul ediyor.
- OS Scheduler (Çizelgeleyici) kuralı gereği, eğer hazırda yüksek öncelikli bir iş varsa, her zaman onu CPU'ya alır.

Sonuç: Yüksek öncelikli işler sisteme gelmeye devam ettiği sürece, düşük öncelikli yedekleme thread'i sürekli kuyruğun arkasına itilir. Sistem tıkır tıkır çalışır ama yedekleme işlemi asla başlayamaz. Bu bir Starvation (Açlık) durumudur.

2. Kısa İşler Önce (Shortest Job First - SJF) Kaynaklı Açlık

Bir başka çizelgeleme algoritması da CPU'yu önce en kısa sürede bitecek olan işe vermektir.

- Elinizde çok uzun sürecek bir işlem (Örn: Devasa bir veritabanı raporu) var.
- Sisteme sürekli olarak milisaniyeler süren küçük sorgular geliyor.

Sonuç: Scheduler sürekli "Kısa işleri aradan çıkarayım" mantığıyla çalıştığı için, uzun raporlama süreci sürekli ertelenir ve starvation'a uğrar.

Starvation Nasıl Çözülür? (Çözüm: Aging - Yaşlandırma)

Mülakatın kilit sorusu: "Peki düşük öncelikli bu sürecin sonsuza kadar beklemesini nasıl engellersin?"

Cevap: Aging (Yaşlandırma) Tekniği.

İşletim sistemleri starvation sorununu önlemek için "Yaşlandırma" adı verilen bir mekanizma kullanır. Bu mekanizmada, bir süreç CPU kuyruğunda beklediği süre boyunca (veya beklediği her turda), sistem o sürecin önceliğini (Priority) kademeli olarak artırır.

- Başlangıçta yedekleme thread'i Priority: 10 (En düşük).

- Kuyrukta 10 saniye bekler. Sistem önceliğini Priority: 9 yapar.
- Beklemeye devam ettikçe önceliği 8, 7, 6 diye artar.

Yeterince uzun süre beklediğinde, yedekleme thread'inin önceliği o kadar yüksek bir seviyeye ulaşır ki, sisteme yeni gelen ve normalde öncelikli olan işlerin bile önüne geçerek CPU'yu zorla alır. Görevini bitirdikten sonra (veya bir kısmını yaptıktan sonra) önceliği tekrar başlangıç seviyesine düşürülür.

(Windows ve Linux'taki modern scheduler'lar (örneğin Linux'taki CFS - Completely Fair Scheduler) arka planda tam olarak bu adaleti sağlamak üzerine tasarlanmıştır.)

Readers-Writers Problemindeki Starvation

Bir diğer mülakat senaryosu, kilit mekanizmalarındaki (Locking) açlıktır.

Örneğin, bir veritabanı dosyasını aynı anda okuyan yüzlerce kullanıcı (Readers) var ve bu dosyayı güncelleyecek tek bir yazar (Writer) var. Eğer sistem, "Biri okurken başka okuyucular da gelebilir ama yazar beklemelidir" (Reader-Preference) mantığıyla çalışıyorsa, yazar güncelleme yapmak için ilk okuyucunun işini bitirmesini bekler. Ancak o bitmeden başka okuyucular da dosyaya erişmeye devam ederse, yazar sürekli gelen okuyuculardan dolayı dosyayı kilitleme fırsatı bulamaz ve starvation yaşar. Bu sorunu çözmek için kilit mekanizmalarında okuyucu ve yazar önceliklerinin dengelenmesi (Fair Locking / Writer-Preference) gerekir.

IPC (Süreçler Arası İletişim) Yöntemleri

> **SORU:** *IPC yöntemleri nelerdir*

1. Mülakat Cevabı

> *"Farklı süreçlerin (process) birbirlerinin bellek alanlarına doğrudan erişememesi nedeniyle işletim sisteminin sunduğu iletişim mekanizmalarına IPC (Inter-Process Communication - Süreçler Arası İletişim) denir. Başlıca yöntemler şunlardır:*
>
> *Pipes (Borular): Verinin tek yönde (Half-duplex) aktığı, genellikle birbiriyle ilişkili (ebeveyn-çocuk / parent-child) süreçler arasında kullanılan FIFO (İlk Giren İlk Çıkar) yapısındaki basit iletişim araçlarıdır.*
>
> *Message Queues (Mesaj Kuyrukları): Süreçlerin birbirlerine veri blokları (mesajlar) gönderdiği ve bu mesajların işletim sistemi tarafından bir kuyrukta tutulduğu sistemlerdir. Asenkrondur; gönderici mesajı bırakır ve alıcının onu hemen okumasını beklemez.*
>
> *Shared Memory (Paylaşılan Bellek): İşletim sisteminin, iki farklı sürecin sanal adres alanına (virtual address space) aynı fiziksel bellek bloğunu haritalamasıdır (mapping). En hızlı IPC yöntemidir, ancak erişimi Mutex/Semaphore gibi kilitlerle senkronize etmek programcının sorumluluğundadır.*
>
> *Sockets (Soketler): Sadece aynı makinedeki değil, bir ağ (network) üzerindeki farklı bilgisayarlarda çalışan süreçlerin (istemci-sunucu / client-server) haberleşmesini sağlayan çift yönlü iletişim yöntemidir."*

2. Detaylı Arka Plan

IPC mülakatlarında sadece isim saymak yetmez, mülakatçı "Senkronizasyon ihtiyacı hangisinde en fazladır?", "Hangisi en hızlıdır?" veya "Hangi senaryoda hangisini seçersin?" gibi karar verme (system design) soruları soracaktır.

1. Pipes (Borular) ve Named Pipes (FIFO)

Pipe'lar Unix sistemlerinin temel felsefesidir: "Bir programın çıktısı, diğerinin girdisi olsun." Terminalde kullandığın | karakteri aslında isimsiz bir Pipe (Anonymous Pipe) yaratır (Örnek: ls | grep txt).

- **Anonymous Pipes (İsimsiz Borular):** Bellekte yaşarlar. Sadece ortak bir atası olan (örneğin fork() ile yaratılmış Parent ve Child) süreçler arasında kullanılabilirler. Tek yönlüdür (Byte stream).
- **Named Pipes (İsimli Borular - FIFO):** Dosya sisteminde fiziksel bir isimle (mkfifo komutuyla) oluşturulurlar. Bu sayede birbiriyle hiçbir akrabalığı olmayan (farklı zamanlarda başlatılmış) süreçler dosyaya yazar veya okur gibi haberleşebilirler. Çift yönlü haberleşme isteniyorsa iki ayrı Pipe açmak gerekir.

2. Message Queues (Mesaj Kuyrukları)

Pipe'larda veriler bir su akıntısı (byte stream) gibi akar, nerede başlayıp bittiğini ayıklamak sana kalır. Message Queue'larda ise veriler ayrık paketler (mesajlar) halindedir.

- **Avantajı:** Asenkrondur (Decoupled). Süreç A, bir mesajı kuyruğa bırakıp kendi işine devam eder. Süreç B o sırada çökmüş olsa bile ayağa kalktığında kuyruktan mesajı okuyabilir.
- **Dezavantajı:** Mesajlar kopyalandığı için (Süreç A'dan İşletim Sistemine, oradan Süreç B'ye) Shared Memory kadar hızlı değildir.
- **Modern Karşılığı:** Bugün mikroservis mimarilerinde duyduğun RabbitMQ veya Apache Kafka, işletim sistemi seviyesindeki bu Message Queue mantığının dağıtık sistemler için devasa ölçekteki modern halleridir.

3. Shared Memory (Paylaşılan Bellek) - Mülakatların Favorisi

Mülakatçı sorar: "Süreçler (Process) birbirinden izole dedik, nasıl oluyor da aynı belleği paylaşıyorlar?"

- **Nasıl Çalışır:** İşletim sisteminde bellek yönetimi (Memory Management Unit - MMU) sayfalama (paging) ile yapılır. Sen shmget (C'de) gibi bir çağrı yaptığında, işletim sistemi RAM'de bir sayfa ayırır. Sonra Süreç A'nın ve Süreç B'nin kendi sanal belleklerindeki (Virtual Memory) belirli bir adresi aynı fiziksel RAM sayfasına işaret edecek şekilde ayarlar (Memory Mapping - mmap).
- **En Hızlısıdır Çünkü:** Veri kopyalaması (Context Switch / Kernel intervention) sıfırdır. Süreç A bir değişkene yazar, Süreç B anında okur. Sanki thread'lermiş gibi iletişim kurarlar.
- **En Tehlikelisidir Çünkü:** İşletim sistemi aradan çekildiği için "Race Condition" koruması sağlamaz. Eğer iki süreç aynı anda Shared Memory'ye yazmaya kalkarsa veri bozulur. Bunu önlemek için Semafor (Semaphore) kullanmak zorundasındır.

4. Sockets (Soketler)

Diğer IPC yöntemleri aynı makine (Localhost) içindeki iletişim içindir. Eğer Süreç A İstanbul'daki bir sunucuda, Süreç B Londra'daki bir sunucudaysa Socket kullanmak zorundasın.

- TCP/IP (güvenilir) veya UDP (hızlı/kayıplı) protokollerini kullanarak çalışır.
- Bazen aynı makine içindeki süreçler de (örneğin Web Sunucusu ile Veritabanı) esneklik sağlaması açısından Local Domain Sockets (UNIX Domain Sockets) kullanırlar. Çünkü yarın veritabanını başka bir sunucuya taşıdığında sadece IP adresini değiştirerek mimariyi bozmadan çalışmaya devam edebilirsin.

C Programlama

static Değişken

> **SORU:** *static değişken nedir, ne için kullanılır*

1. Mülakat Cevabı

> *"C ve C++ dillerinde static anahtar kelimesi, bir değişkenin ömrünü (lifetime) ve görünürlüğünü (scope/visibility) değiştirmek için kullanılır. Kullanıldığı yere göre iki farklı temel amacı vardır:*
>
> *Fonksiyon içinde (Local Static): Normal yerel değişkenler (stack'te tutulanlar) fonksiyon bitince yok olurken, static olarak tanımlanan bir yerel değişken belleğin Data/BSS segmentinde tutulur. Fonksiyon bitse bile değerini korur ve fonksiyona bir sonraki çağrıda kaldığı yerden devam eder. Fonksiyonun kaç kez çağrıldığını saymak gibi durumlar için idealdir.*
>
> *Global alanda (Global Static): Bir dosyanın (translation unit) en üstünde global bir değişken static olarak tanımlanırsa, bu değişkenin görünürlüğü (scope) sadece o C dosyasıyla sınırlandırılır. Başka bir .c dosyası extern anahtar kelimesini kullansa bile bu değişkene erişemez. Bu, isim çakışmalarını (name clashing) önlemek ve veriyi kapsüllemek (encapsulation) için kullanılır."*

2. Detaylı Arka Plan

static kelimesi mülakatların en sevilen tuzaklarından biridir çünkü bağlama göre tamamen farklı iki davranış sergiler. Mülakatçı bu konuyu büyük ihtimalle bellek yönetimi (Memory Layout) üzerinden deşecektir.

1. Local Static Variables (Fonksiyon İçi Statik Değişkenler)

Bir fonksiyonun içinde tanımlanan değişkenlerin nerede yaşadığını anlamak kilit noktadır.

```c
#include <stdio.h>

void counter() {
    int normal_var = 0;         // Stack'te yaratılır
    static int static_var = 0;  // Data/BSS Segment'te yaratılır (Sadece BİR KERE çalışır)

    normal_var++;
    static_var++;

    printf("Normal: %d, Static: %d\n", normal_var, static_var);
}

int main() {
    counter(); // Çıktı: Normal: 1, Static: 1
    counter(); // Çıktı: Normal: 1, Static: 2
    counter(); // Çıktı: Normal: 1, Static: 3
    return 0;
}
```

Arka Planda Neler Oluyor?

- normal_var, fonksiyon her çağrıldığında Stack belleğinde yeniden yaratılır ve fonksiyon bitince yok edilir.
- static_var ise program ilk yüklendiğinde Data Segment (eğer ilk değer atanmışsa) veya BSS Segment (eğer sıfırsa veya değer atanmamışsa) adı verilen bellek bölgesinde yaratılır. Programın çalışması bitene (termination) kadar bellekte kalır. static int static_var = 0; satırı aslında bir atama işlemi değil, derleyiciye verilen bir direktiftir; fonksiyon ikinci kez çağrıldığında bu satır es geçilir.

Kullanım Amacı: Bir fonksiyonun kendi iç durumunu (state) global bir değişken kullanmadan, sadece kendi içinde koruması gerektiğinde kullanılır.

2. Global Static Variables (Global Statik Değişkenler)

Bu kullanım şekli, C dilinde Nesne Yönelimli Programlamanın (OOP) temel direklerinden biri olan Kapsülleme (Encapsulation) işlemini taklit etmek için kullanılır.

Diyelim ki iki farklı dosyan var: database.c ve main.c.

database.c:

```c
int connection_count = 0;        // Herkes görebilir
static int db_password = 1234;   // SADECE database.c görebilir

void connect() {
    connection_count++;
}
```

main.c:

```c
extern int connection_count;     // Geçerli. Linker bunu database.c'de bulur.
extern int db_password;          // HATA! Linker db_password'u bulamaz.
```

Arka Planda Neler Oluyor? (Linker Aşaması)

C programları derlenirken her .c dosyası ayrı ayrı derlenerek birer Object (.o) dosyasına dönüştürülür. Daha sonra Linker (Bağlayıcı) bu dosyaları birleştirir.

Normal global değişkenler .o dosyasının sembol tablosuna (symbol table) "External (Dışarıya açık)" olarak yazılır. Ancak static olarak işaretlenmiş bir global değişken, sembol tablosuna "Internal (İçsel)" olarak yazılır. Linker dosyaları birleştirirken static değişkenleri diğer dosyalardan gizler.

Kullanım Amacı: C'de devasa projeler yazarken (örneğin Linux Kernel'i) binlerce dosya vardır. temp, index, count gibi genel isimli değişkenlerin veya yardımcı fonksiyonların başka dosyalardaki aynı isimli değişkenlerle çakışmasını (Name Collision) engellemek için mecburi bir standarttır. (Aynı kural static fonksiyonlar için de geçerlidir).

Mülakat Tuzağı: Multi-Threading ve static

Mülakatçı sorar: "Bir fonksiyonun içinde static bir sayaç değişkeni tanımladın. Bu fonksiyonu aynı anda iki farklı Thread çağırırsa ne olur?"

Cevap: Çok büyük ihtimalle bir Race Condition (Yarış Durumu) olur!

Hatırlarsan, Thread'lerin Stack alanları ayrıydı ancak Data/BSS alanları ortaktı. Local static değişkenler Data segmentinde yaşadığı için, tüm thread'ler aynı fiziksel hafıza adresine erişmeye çalışacaktır. Eğer bu değişkeni korumak için Mutex kullanmazsan sayaç değeri yanlış hesaplanır.

static Fonksiyon

> **SORU:** *static fonksiyon ne işe yarar*

1. Mülakat Cevabı

> *"C ve C++ dillerinde static anahtar kelimesinin fonksiyonlar için, kullanıldığı bağlama göre birbirinden tamamen farklı iki temel amacı vardır:*
>
> *Global Alanda (C ve C++): Bir dosyanın içinde tanımlanan fonksiyon static yapılırsa, o fonksiyonun görünürlüğü (scope) sadece o dosyayla sınırlandırılır. (Buna Internal Linkage denir). Projedeki diğer dosyalar bu fonksiyonu göremez ve çağıramaz. Temel amacı dosya seviyesinde kapsülleme (encapsulation) yapmak ve linker (bağlayıcı) aşamasında isim çakışmalarını (name collision) önlemektir.*
>
> *Sınıf İçinde (Sadece C++ ve OOP dilleri): Bir sınıfın (class) içindeki fonksiyon static yapılırsa, o fonksiyon artık yaratılan nesnelere (objelere) değil, doğrudan sınıfın kendisine ait olur. Fonksiyonu çağırmak için sınıftan bir nesne (new) yaratmanıza gerek yoktur. Ancak bu fonksiyonlar, arka planda bir this pointer'ına sahip olmadıkları için, sınıfın normal (static olmayan) değişkenlerine erişemezler."*

2. Detaylı Arka Plan

Mülakatçı sen bu iki farkı ortaya koyduğunda, mülakatın yönünü senin başvurduğun pozisyona göre seçecektir. Gömülü Sistemler/C mülakatındaysan birinci maddeyi, C++ veya Genel Yazılım Mühendisliği mülakatındaysan ikinci maddeyi deşer.

1. C Dilinde Static Fonksiyonlar (Dosya Seviyesi Kapsülleme)

Linux Kernel'i gibi on binlerce .c dosyasından oluşan devasa bir proje yazdığını düşün.

- **Sorun:** Farklı geliştiriciler kendi dosyalarında init(), calculate(), reset() gibi çok genel isimli yardımcı fonksiyonlar yazabilirler. C'de fonksiyonlar varsayılan olarak dışarıya açıktır (External Linkage). Derleyici (Compiler) bunları derler ama Linker (Bağlayıcı) bu dosyaları birleştirmeye çalıştığında "Aynı isimde iki tane init() fonksiyonu buldum, hangisini kullanacağım?" diyerek "Multiple Definition" (Çoklu Tanımlama) hatası vererek patlar.
- **Çözüm:** Eğer yazdığın fonksiyon sadece o .c dosyasının içindeki işlemler için bir yardımcı fonksiyonsa (helper function) ve dışarıdaki kimsenin onu kullanmasını istemiyorsan, başına static koyarsın.
- Arka Planda Ne Olur? Linker, static ile işaretlenmiş fonksiyonları "Global Sembol Tablosundan" gizler. Böylece başka bir dosya extern yazarak bile senin fonksiyonuna ulaşamaz.

Mülakat Tavsiyesi: Eğer bir C mülakatındaysan şu cümleyi kurman seni çok öne çıkarır: "C'de sınıf (class) ve 'private' anahtar kelimesi yoktur. C dilinde nesne yönelimli programlamadaki 'private' (gizli) metot mantığını taklit etmenin tek yolu, yardımcı fonksiyonları static yapmaktır."

2. C++ (ve Java/C#) Dilinde Static Sınıf Fonksiyonları

Bir C++ sınıfında (class) static metot tanımladığında, mimari tamamen değişir.

```
C++
class Matematik {
private:
    int normalCarpan = 5;
public:
    // Static fonksiyon
    static int kareAl(int sayi) {
        return sayi * sayi;
    }

    // Normal fonksiyon
```

```
    void ekranaYazdir() { ... }
};
```

- **Nesne Yaratmaya Gerek Yoktur:** Bu metodu çağırmak için Matematik mat; diye bir obje yaratmana gerek yoktur. Doğrudan sınıf ismiyle Matematik::kareAl(4); şeklinde çağırırsın.
- **Kullanım Alanları:** Genellikle duruma (state) bağlı olmayan, sadece aldığı parametreye göre matematiksel işlem yapan Utility/Helper (Yardımcı) sınıflarda kullanılır. Ayrıca mülakatların favorisi olan Singleton Pattern (Tekillik Kalıbı) ve Factory Pattern (Fabrika Kalıbı) gibi tasarım kalıplarının omurgasıdır.

Mülakat Tuzağı: this Pointer Yokluğu

Mülakatçı sana şu kodu verir ve "Bu kod derlenir mi? Neden?" diye sorar:

```C++
class Test {
    int deger = 10;
public:
    static void yazdir() {
        cout << deger; // HATA!
    }
};
```

Kritik Cevap: "Kod derlenmez (Compile Error). Çünkü static fonksiyonların arka planda gizli bir this pointer'ı yoktur. deger değişkeni sınıfa değil, yaratılacak olan nesneye aittir. static fonksiyon, 'Ben bu değeri yazdıracağım ama hangi nesnenin (hangi this'in) değerini yazdıracağım?' bilemez ve patlar. Static fonksiyonlar, sadece diğer static değişkenlere ve static fonksiyonlara erişebilir."

Singleton Pattern (Tekillik Kalıbı) nasıl yazılır?

› *C'de 'extern' anahtar kelimesi nedir?*

› *C'de Inline Fonksiyonlar*

struct ve union Farkı

> **SORU:** *struct union farkı*

1. Mülakat Cevabı

> *"Struct (Yapı) ve Union (Birlik) arasındaki temel fark, belleğin nasıl tahsis edildiği ve yönetildiğidir.*
>
> *Struct: İçerisindeki her bir eleman için bellekte ayrı ayrı yer ayrılır. Bir struct'ın toplam boyutu, içindeki elemanların boyutlarının toplamına eşittir (derleyici tarafından eklenen padding/hizalama boşlukları da dahil). İçindeki tüm değişkenlere aynı anda, birbirini etkilemeden erişebilir ve değer atayabilirsiniz.*
>
> *Union: İçerisindeki tüm elemanlar aynı bellek adresini (ortak alanı) paylaşır. Bir union'ın toplam boyutu, içindeki en büyük elemanın boyutu kadardır. Aynı bellek alanını paylaştıkları için, union içinde aynı anda sadece bir değişkende geçerli bir değer tutulabilir. Bir değişkene değer atadığınızda, bellekteki veri ezileceği için diğer değişkenlerin değerleri bozulur."*

2. Detaylı Arka Plan

Bu konu genellikle gömülü sistemler (embedded systems) veya sistem programlama mülakatlarında karşısına çıkar. Mülakatçı tanımı duyduktan sonra doğrudan bellek analizi yapmanı isteyecektir.

Bellek Düzeni (Memory Layout) ve Boyut Analizi

Aşağıdaki gibi bir yapı tanımladığımızı düşünelim:

```c
#include <stdio.h>

struct VeriStruct {
    int id;        // 4 byte
    double maas;   // 8 byte
    char seviye;   // 1 byte
};

union VeriUnion {
    int id;        // 4 byte
    double maas;   // 8 byte
    char seviye;   // 1 byte
};
```

- **Struct Boyutu:** 4 + 8 + 1 = 13 byte olmasını beklersin. Ancak derleyici, işlemcinin belleği daha hızlı okuyabilmesi için aralara boş baytlar (Padding) ekler. Bu yüzden sizeof(struct VeriStruct) genelde bellekte 16 byte veya 24 byte (sistemin mimarisine göre) yer kaplar. Bütün değişkenler bağımsızdır.
- **Union Boyutu:** En büyük eleman olan double 8 byte olduğu için sizeof(union VeriUnion) tam olarak 8 byte yer kaplar. id, maas ve seviye değişkenlerinin üçü de bellekte aynı başlangıç adresini (offset 0) gösterir.

Union'ın "Tehlikeli" Doğası (Type Punning)

Mülakatlarda sıkça sorulan tuzak sorulardan biri şudur: "Union içindeki bir int değişkene değer atayıp, onu char olarak okursam ne olur?"

```c
union Test {
    int sayi;
    char harf;
};

union Test t;
```

```
t.sayi = 65;
printf("%c\n", t.harf); // Çıktı: 'A' (ASCII 65)
```

Bu işleme Type Punning (Tür Uydurma) denir. Aynı bit dizilimini farklı bir veri tipiymiş gibi yorumlamanı sağlar. Gömülü sistemlerde veya ağ (network) programlamasında veriyi bit düzeyinde manipüle etmek için çok faydalıdır ancak C++'ın modern standartlarında "Strict Aliasing" (Sıkı Takma Ad) kurallarını ihlal edebileceği için Undefined Behavior (Tanımsız Davranış) riski taşır ve dikkatli kullanılmalıdır.

Neden Union Kullanırız? (Gerçek Hayat Senaryoları)

Mülakatçı "Madem içindeki veriler bozuluyor, neden her şeyi Struct yapmıyoruz?" diyecektir.

- **Bellek Optimizasyonu (Memory Optimization):** Sensörlerden veri okuyan bir mikrodenetleyicide (örneğin 2KB RAM'i olan bir sistemde) çalışıyorsun. Sensör bazen sıcaklık (float), bazen hata kodu (int), bazen durum bilgisi (char) gönderiyor. Aynı anda sadece bir veri türü geleceği kesinse, bunları struct ile tanımlayıp boşuna bellek harcamak yerine union ile aynı bellek bloğunu farklı durumlarda farklı amaçlar için geri dönüştürürsün.
- **Variant Types (Değişken Tipleri):** Bir nesnenin farklı türlerde veri tutabilmesi gerektiğinde (Python veya JavaScript'teki dinamik değişkenlerin arka planındaki mantık) Union yapısı ve yanına verinin o anki türünü tutan bir "Tag" (etiket) eklenerek (Tagged Union) kullanılır.

enum

> **SORU:** *enum nedir ve nasıl kullanılır*

1. Mülakat Cevabı

> *"Enum (Enumeration - Numaralandırma), birbiriyle ilişkili sabit tam sayı (integer) değerlerine anlamlı isimler vermek için kullanılan kullanıcı tanımlı bir veri tipidir.*
>
> *Temel amacı kodun okunabilirliğini ve yönetilebilirliğini artırmaktır. Kod içinde ne anlama geldiği belli olmayan 'sihirli sayılar' (magic numbers) kullanmak yerine (örneğin durumu belirtmek için 0, 1, 2 kullanmak), bu değerlere BASLADI, DURDU, HATA gibi isimler vererek kodu kendi kendini açıklayan (self-documenting) bir hale getiririz. Genellikle Durum Makinelerinde (State Machines), gün/ay isimlerinde, konfigürasyon bayraklarında (flags) ve hata kodlarında kullanılır."*

2. Detaylı Arka Plan

Mülakatlarda enum konusu açıldığında, mülakatçı genellikle bunun bellekte nasıl tutulduğunu ve C/C++ dillerindeki makrolarla (#define) arasındaki farkı bilip bilmediğini test eder.

Nasıl Kullanılır ve Arka Planda Ne Olur?

Standart bir enum tanımı şu şekildedir:

```c
#include <stdio.h>

// Bir enum tanımlayalım
enum MakineDurumu {
    KAPALI,      // Varsayılan olarak 0 değerini alır
    CALISIYOR,   // 1 değerini alır
    BEKLEMEDE,   // 2 değerini alır
    ARIZA = 100, // Manuel olarak 100 atadık
    BAKIM        // Kendinden öncekini 1 artırır, yani 101 olur
};

int main() {
    enum MakineDurumu mevcutDurum = CALISIYOR;

    if (mevcutDurum == CALISIYOR) {
        printf("Makine şu an çalışıyor, kod: %d\n", mevcutDurum);
    }
    return 0;
}
```

Derleyici (Compiler) bu kodu derlediğinde, KAPALI veya CALISIYOR gibi kelimeleri doğrudan tam sayılarla (0, 1, vb.) değiştirir. Yani enum, bellekte normal bir int (genellikle 4 byte) kadar yer kaplar.

Mülakat Tuzağı: #define vs enum

Mülakatçı sorar: "Madem sabit değerler tanımlıyoruz, neden #define DURUM_CALISIYOR 1 şeklinde Makro kullanmak yerine enum kullanıyoruz?"

Bu soruya vereceğin cevap senin kıdemini (seniority) belirler:

- **Hata Ayıklama (Debugging):** #define bir ön işlemci (preprocessor) direktifidir. Derleme başlamadan önce kodda bir metin bul-değiştir işlemi yapar. Bu yüzden Debugger (örneğin GDB) çalıştırırken DURUM_CALISIYOR ismini göremezsin, sadece 1 sayısını görürsün. enum ise derleyici (compiler) tarafından işlenir, sembol tablosunda tutulur ve debug yaparken değişkenin adını açıkça görebilirsin.

- **Kapsam (Scope) Kuralları:** #define bütün dosyayı etkiler, globaldir ve scope (süslü parantezler {} arası alan) kurallarına uymaz. Ancak enum normal bir değişken gibi scope kurallarına uyar, örneğin sadece bir sınıfın veya fonksiyonun içinde geçerli olacak bir enum tanımlayabilirsin.
- **Otomatik Değer Atama:** #define ile yüzlerce hata kodu yazacaksan hepsine tek tek 1, 2, 3... diye sayı ataman gerekir. enum bunu otomatik yapar.

C++ Mülakatları İçin Bonus: enum class (Scoped Enums)

Eğer C++ mülakatındaysan, geleneksel C stili enum'ların çok tehlikeli bir dezavantajından bahsetmen ekstra puan getirir: Tür Güvenliği (Type Safety).

Geleneksel C enum'ları integer'lara üstü kapalı (implicit) olarak dönüştürülebilir. Ayrıca isimleri global isim alanına (namespace) sızar.

```
C++
enum Renk { KIRMIZI, MAVI };
enum Meyve { ELMA, ARMUT };

int sayi = KIRMIZI; // C'de ve eski C++'ta GEÇERLİDİR. (Tür güvenliği yok)
// KIRMIZI == ELMA (İkisi de 0 olduğu için eşittir! Çok tehlikeli bir mantık hatası)
```

C++11 ile gelen enum class bu sorunu çözer:

```
C++
enum class Renk { KIRMIZI, MAVI };
enum class Meyve { ELMA, ARMUT };

// int sayi = Renk::KIRMIZI; // HATA! Otomatik dönüştürme yapmaz.
// if(Renk::KIRMIZI == Meyve::ELMA) // HATA! Farklı türleri kıyaslatmaz.
```

enum class, değerlerin isim alanına sızmasını engeller ve programcının yanlışlıkla bir rengi bir meyveyle veya bir integer ile kıyaslamasının önüne geçer.

Pointer'lar (Göstericiler)

> **SORU:** *pointerlar ile mülakatta sorulması muhtemel soruları cevaplandırabilir misin?*

Pointer'lar (Göstericiler), C ve C++ mülakatlarının "patron son canavarıdır". Mülakatçı senin bellek mimarisine ne kadar hakim olduğunu, C dilinin tehlikelerini ne kadar anladığını sadece pointer sorularıyla ölçebilir.

İşte FAANG ve köklü teknoloji şirketlerinin mülakatlarında en çok sorulan 3 kritik pointer sorusu ve bunları nasıl cevaplaman gerektiği:

Soru 1: Bellek Sızıntısı (Memory Leak) ve Sarkan Gösterici (Dangling Pointer) arasındaki fark nedir?

1. Mülakat Cevabı

> *"İkisi de dinamik bellek yönetimi (Heap) hatalarıdır, ancak birbirinin zıttı yönünde çalışırlar.*
>
> *Memory Leak (Bellek Sızıntısı): Dinamik olarak tahsis edilen (malloc veya new ile) bir bellek bloğunun işi bittiğinde işletim sistemine iade edilmemesidir (free veya delete çağrılmaması). Pointer'ın kendisi (Stack'teki değişken) yok olsa bile, Heap'teki veri asılı kalır ve program çalıştıkça RAM tüketimi sürekli artar.*
>
> *Dangling Pointer (Sarkan Gösterici): Tahsis edilen bellek free() ile temizlenip işletim sistemine geri verilmesine rağmen, pointer değişkeninin hala o eski bellek adresini işaret etmeye devam etmesidir. Eğer program bu pointer'ı kullanarak o adrese tekrar okuma veya yazma yapmaya kalkarsa (işletim sistemi o adresi başka bir sürece vermiş olabileceğinden) Segmentation Fault hatası alırız veya veri bozulması yaşarız."*

2. Detaylı Arka Plan

Mülakatçı senden Dangling Pointer'ı kod üzerinde nasıl önleyeceğini soracaktır.

Kritik Çözüm: Bir pointer'ı free() ettikten HEMEN SONRA onu NULL (veya C++'ta nullptr) değerine eşitlemektir.

```c
C
int *ptr = (int *)malloc(sizeof(int));
*ptr = 10;
free(ptr); // Bellek iade edildi.
// Şu an 'ptr' bir Dangling Pointer!
ptr = NULL; // Çözüm: Artık güvenli.
```

C ve C++'ta NULL bir pointer'ı free() etmeye çalışmak veya kontrol etmek (if (ptr != NULL)) güvenlidir, ancak Dangling bir pointer'ı kontrol edemezsin çünkü adresi hala geçerli bir sayı gibi görünür.

Soru 2: Dizi (Array) ve Pointer aynı şey midir? (Array Decay)

1. Mülakat Cevabı

> *"Hayır, kesinlikle aynı şey değillerdir. Bir dizi (Array), bellekte yan yana duran aynı tipteki verilerin tamamını temsil ederken; bir Pointer sadece tek bir bellek adresini tutan bir değişkendir.*
>
> *Mülakatlarda sıkça karıştırılmalarının sebebi 'Array Decay' (Dizinin Çürümesi) adı verilen kuraldır. C dilinde bir diziyi bir fonksiyona parametre olarak gönderdiğinizde, dizi bütünlüğünü (boyut bilgisini) kaybeder ve dizinin sadece ilk elemanını gösteren bir pointer'a dönüşür (decay). Bu yüzden fonksiyon içindeki bir dizinin boyutunu sizeof() ile ölçemeyiz."*

2. Detaylı Arka Plan

Bu, mülakatların en eleyici tuzak sorusudur. Kod üzerinde kanıtlaman istenir.

```c
void diziYazdir(int arr[10]) {
    // BURASI TUZAK!
    // arr artık 10 elemanlı bir dizi değil, sadece bir (int *) pointer'dır.
    printf("%lu\n", sizeof(arr)); // Çıktı: 8 (64-bit sistemde bir pointer'ın boyutu)
}

int main() {
    int sayilar[10];
    printf("%lu\n", sizeof(sayilar)); // Çıktı: 40 (10 eleman * 4 byte)

    diziYazdir(sayilar);
    return 0;
}
```

İkinci Fark (Adres Farkı):

- Pointer'ın bellekte kendine ait bir adresi vardır (&ptr farklı bir adrestir).
- Dizinin adresini aldığında (&sayilar), yine dizinin başladığı adresi verir. Dizi isminin kendisi bellekte yer kaplayan ayrı bir değişken değildir, derleyici seviyesinde bir etikettir.

Soru 3: Bir fonksiyona Pointer geçmek (*p) ile Double Pointer (p) geçmek arasındaki fark nedir?**

1. Mülakat Cevabı

> *"C dilinde her şey (pointer'lar dahil) fonksiyona değer kopyası (pass by value) ile geçirilir.*
>
> *Eğer bir fonksiyonun, orijinal değişkenin içindeki değeri değiştirmesini istiyorsak, o değişkenin adresini (tek yıldızlı pointer, *p) göndeririz.*
>
> *Ancak, fonksiyonun orijinal pointer'ın nereyi işaret ettiğini (adresini) değiştirmesini istiyorsak (örneğin fonksiyon içinde yeni bir bellek tahsis edip (malloc) bunu ana programa döndüreceksek), o zaman pointer'ın da adresini, yani bir Double Pointer (**p) göndermek zorundayız."*

2. Detaylı Arka Plan

Bağlı Liste (Linked List) mülakat sorularında adayların %90'ı bu yüzden elenir.

Yanlış Kullanım (Memory Leak Yaratır):

```c
void bellekAyir(int *p) {
    p = (int *)malloc(sizeof(int)); // p, main'deki ptr'nin kopyasıdır!
    *p = 50;
} // Fonksiyon bittiğinde kopyalanan 'p' silinir, ayrılan bellek sızar (Leak).

int main() {
    int *ptr = NULL;
    bellekAyir(ptr);
    // ptr HALA NULL'dır! Patlar.
}
```

Doğru Kullanım (Double Pointer):

```c
void bellekAyirDogru(int **p) {
    *p = (int *)malloc(sizeof(int)); // Orijinal ptr'nin içindeki adresi değiştir!
    **p = 50;
}

int main() {
    int *ptr = NULL;
    bellekAyirDogru(&ptr); // Pointer'ın kendi adresini gönderiyoruz
```

```
    // ptr artık tahsis edilen belleği gösteriyor.
}
```

Özellikle bir Linked List'in başını (Head) değiştiren bir insertAtHead() veya deleteNode() fonksiyonu yazıyorsan, listeyi dışarıdan Node **head olarak almak zorundasın.

Function Pointers (Fonksiyon Göstericileri)

› *C++'ta Smart Pointers ile Çözüm*

› *Pointer Aritmetiği Mülakat Sorusu*

Makrolar (#define)

> **SORU:** *makrolar ile ilgili mülakatta sorulması muhtemel soruları cevaplandırabilir misin?*

Makrolar (#define), C ve C++ mülakatlarında genellikle adayın Önişlemci (Preprocessor) ile Derleyici (Compiler) arasındaki farkı ne kadar anladığını test etmek için "tuzak sorular" üzerinden sorulur.

İşte FAANG ve köklü sistem programlama mülakatlarında karşına çıkacak en kritik 3 makro sorusu ve cevapları:

Soru 1: Makro (Macro) ile Inline Fonksiyon arasındaki fark nedir? Hangi durumda hangisini tercih edersin?

1. Mülakat Cevabı

> *"Makrolar, derleme başlamadan önce Önişlemci (Preprocessor) tarafından çalıştırılan saf bir metin bul-değiştir (text substitution) işlemidir. Inline fonksiyonlar ise doğrudan Derleyici (Compiler) tarafından işlenen gerçek fonksiyonlardır.*
>
> *En büyük fark Tür Güvenliği (Type Safety) ve Hata Ayıklama (Debugging) konularındadır. Makrolar parametrelerin veri tipini kontrol etmez (int mi, float mı bakmaz) ve debugger (hata ayıklayıcı) sembollerinde görünmezler, bu yüzden hatayı bulmak çok zordur. Inline fonksiyonlar ise tür kontrolü yapar ve güvenlidir. Modern C++'ta (ve hatta C99 sonrası C'de) sabit değerler için const/constexpr, küçük fonksiyonlar için ise inline kullanmayı her zaman makrolara tercih ederim."*

2. Detaylı Arka Plan

Mülakatçı burada "Madem inline fonksiyon bu kadar iyi, makroyu ne zaman kullanırsın?" diyecektir.

Ne Zaman Makro Kullanılır?

Makroların hala rakipsiz olduğu yerler Koşullu Derleme (Conditional Compilation) ve Kod Üretimi (Code Generation) aşamalarıdır.

Eğer Windows için farklı, Linux için farklı bir kod bloğu derlemek istiyorsan bunu fonksiyonla yapamazsın. #ifdef _WIN32 gibi makrolar kullanmak zorundasındır.

Loglama sistemlerinde hatanın hangi dosyanın kaçıncı satırında olduğunu yazdıran __FILE__ ve __LINE__ gibi önişlemci yetenekleri sadece makrolarla çalışır.

Soru 2: Makrolardaki klasik "İşlem Önceliği" ve "Yan Etki (Side Effect)" hatalarını açıklar mısın?

1. Mülakat Cevabı

> *"Makrolar metin değiştirme mantığıyla çalıştığı için, programcılar onlara fonksiyonmuş gibi davrandığında iki büyük hata ortaya çıkar.*
>
> *Birincisi Parantez Eksikliği hatasıdır. Parametreler parantez içine alınmazsa, işlem önceliği bozulur ve matematiksel sonuçlar yanlış çıkar. İkincisi ise Yan Etki (Side Effect) hatasıdır. Bir makroya parametre olarak ++x (artırma) gibi bir işlem gönderdiğinizde, makronun içinde o parametre birden fazla kez kullanılıyorsa, değişken istemeden iki veya üç kez artırılır."*

2. Detaylı Arka Plan

Bu soruda mülakatçı sana bir kağıt veya beyaz tahta verir ve "Şu kodun çıktısı nedir?" der.

Tuzak 1: Parantez Eksikliği

```c
#define KARE_AL(x) x * x
```

```
int sonuc = KARE_AL(3 + 2);
```

İlk bakışta 25 olmasını beklersin. Ancak önişlemci bunu şu metne çevirir: 3 + 2 * 3 + 2. Matematiksel işlem önceliği (çarpma önce yapılır) nedeniyle sonuç 3 + 6 + 2 = 11 çıkar!

Doğrusu: #define KARE_AL(x) ((x) * (x)) olmalıdır.

Tuzak 2: Yan Etki (Side Effect)

```
C
#define MAX(a, b) ((a) > (b) ? (a) : (b))
int i = 5, j = 3;
int enBuyuk = MAX(i++, j++);
```

İlk bakışta en büyük 5 olarak atanır, sonra i 6 olur diye beklersin. Ancak önişlemci kodu şu hale getirir:

```
C
((i++) > (j++) ? (i++) : (j++))
```

i, j'den büyük olduğu için koşul doğru (true) çıkar ve i++ kısmı bir kez daha çalışır. Yani i değişkeni arka planda bir kez değil, iki kez artırılarak 7 değerini alır. Bu, bulması en zor bug'lardan (hatalardan) biridir. (Bu yüzden Inline fonksiyonlar hayat kurtarır).

Soru 3: C kodlarında sıkça gördüğümüz çok satırlı makrolardaki do { ... } while(0) yapısı ne işe yarar?

1. Mülakat Cevabı

> *"Çok satırlı bir makroyu süslü parantezi olmayan bir if-else bloğunun içinde kullandığımızda, makronun metin değiştirmesi sonucu kodun mantığı bozulur ve derleyici 'Dangling Else' (sahipsiz else) hatası verir.*
>
> *do { ... } while(0) yapısı, makronun içindeki tüm kod satırlarını tek bir güvenli işlem (statement) bloğuna hapseder. Üstelik sonuna noktalı virgül (;) konulmasını zorunlu kıldığı için, geliştirici makroyu normal bir fonksiyon gibi çağırabilir ve kodun sözdizimi (syntax) hiçbir şekilde bozulmaz."*

2. Detaylı Arka Plan

Bu, kıdemli (senior) sistem programcısı mülakatlarının çok popüler bir sorusudur. Neden normal bir süslü parantez { } kullanmıyoruz da do-while(0) kullanıyoruz?

Diyelim ki iki sayıyı değiştiren bir makro yazdık:

```
C
// YANLIŞ MAKRO KULLANIMI
#define SWAP(a, b)  int temp = a; a = b; b = temp;
if (kosul)
    SWAP(x, y);
else
    printf("Kosul saglanmadi");
```

Önişlemci bu kodu açtığında şuna dönüşür:

```
C
if (kosul)
    int temp = a;
    a = b;
    b = temp;; // <--- Buradaki noktalı virgüle dikkat
else // HATA: Derleyici burada patlar! Çünkü if bloğu "int temp = a;" ile kapandı.
    printf("Kosul saglanmadi");
```

Sadece { } koysaydık ne olurdu?

```c
#define SWAP(a, b) { int temp = a; a = b; b = temp; }
if (kosul)
    SWAP(x, y); // Bu satırın sonundaki ';' makrodaki '}' ile birleşir -> }; olur.
else // HATA: Derleyici yine patlar! Çünkü if bloğundan sonra gelen }; işlemi bitirir, else sahipsiz kalır.
```

Mükemmel Çözüm (do-while(0)):

```c
#define SWAP(a, b) do { int temp = a; a = b; b = temp; } while(0)
if (kosul)
    SWAP(x, y); // do { ... } while(0); şekline dönüşür ve if-else yapısı mükemmel çalışır.
else
    printf("Kosul saglanmadi");
```

Bu yöntem Linux Kernel'i dahil tüm büyük C projelerinde endüstri standardıdır.

C'de Volatile Anahtar Kelimesi

› Fonksiyon Çağrı Yığını (Call Stack)

› Veri Yapıları: Hash Table

C Kodunun Executable Olma ve Çalıştırılma Süreci

> **SORU:** *C ile yazılmış bir kod executable olup ve sonrasında çalıştırılana kadarki geçen bütün işlemleri anlat*

1. Mülakat Cevabı

> *"Bir C kaynak kodunun (.c), çalıştırılabilir bir dosyaya (executable) dönüşmesi 4 aşamalı bir derleme zincirinden (Compilation Pipeline) oluşur. Dosya oluştuktan sonra ise işletim sistemi tarafından belleğe yüklenme (Loading) süreci başlar.*
>
> *Derleme Aşamaları:*
>
> *Preprocessing (Önişlemci): Kod derlenmeden önce #include ile belirtilen kütüphaneler koda dahil edilir, #define makroları yerlerine yazılır ve yorum satırları silinir.*
>
> *Compilation (Derleme): Önişlemciden çıkan saf C kodu, işlemcinin mimarisine uygun Assembly diline çevrilir. (Aynı zamanda sözdizimi ve mantık hataları burada yakalanır).*
>
> *Assembly (Çevirici): Assembly kodu, makine diline (0'lar ve 1'ler) çevrilerek Object (Nesne) dosyası (.o) oluşturulur.*
>
> *Linking (Bağlama): Projedeki birden fazla nesne dosyası ve dış kütüphaneler (örneğin printf fonksiyonunu barındıran standart C kütüphanesi) birleştirilerek tek bir Executable (Çalıştırılabilir) dosya (Linux'ta ELF, Windows'ta PE formatında) elde edilir.*
>
> *Çalıştırma Aşaması (Execution):*
>
> *Kullanıcı dosyayı çalıştırdığında, işletim sisteminin Loader (Yükleyici) bileşeni devreye girer. Program için sanal bellek (Virtual Memory) ayırır, kodu ve verileri belleğe (Text, Data, BSS segmentlerine) yerleştirir, Stack (Yığın) alanını hazırlar ve CPU'nun kontrolünü programın başlangıç noktasına (_start ve ardından main fonksiyonuna) verir."*

2. Detaylı Arka Plan

Bu soru, sistem programlama ve derleyici mimarisi mülakatlarının (özellikle Gömülü Sistemler ve C/C++ pozisyonlarının) favori sorusudur. Mülakatçı bu zincirin her bir halkasındaki hataları ve arka plan detaylarını sorabilir.

A. Derleme Zinciri (Compilation Pipeline) Detayları

Mülakatlarda "Bir fonksiyonu deklare ettin ama gövdesini yazmadın. Kod hangi aşamada patlar?" gibi sorular gelir. Süreci bilmek bu yüzden hayatidir.

1. Preprocessor (Önişlemci)

Bu aşama tamamen bir "metin bul ve değiştir" (text substitution) işlemidir. Derleyici henüz C dilinin kurallarını bilmez.

- **Ne Yapar:** #include <stdio.h> gördüğünde, gider o dosyanın içindeki metni kopyalar ve senin kodunun en üstüne yapıştırır. Yorum satırlarını (//) siler, #define PI 3.14 gördüğünde kodda PI yazan her yere 3.14 yazar.
- **Çıktı:** .i uzantılı, oldukça şişkin bir C dosyasıdır. (GCC'de gcc -E ile görülebilir).

2. Compiler (Derleyici)

Burası işin beynidir. C kodu, donanıma özel (örneğin x86 veya ARM) Assembly diline çevrilir.

- **Ne Yapar:** Lexical (Kelime) ve Syntax (Sözdizimi) analizleri yapılır. Eksik noktalı virgül burada hata verir. Ayrıca kod optimizasyonları (kullanılmayan değişkenleri silme, döngüleri hızlandırma) bu aşamada yapılır (Örn: -O2, -O3 bayrakları).
- **Çıktı:** .s uzantılı Assembly dosyasıdır. (GCC'de gcc -S ile görülebilir).

3. Assembler (Çevirici)

İnsanın okuyabildiği Assembly komutları (örneğin MOV EAX, 1), işlemcinin anlayacağı saf ikilik sisteme (binary makine kodu) çevrilir.

- **Çıktı:** .o (Object) dosyasıdır. (GCC'de gcc -c ile oluşturulur).
- **Kritik Detay:** Bu dosya henüz çalıştırılamaz! Neden? Çünkü senin kodunda çağırdığın printf fonksiyonunun bellekte tam olarak nerede (hangi adreste) olduğunu Assembler bilemez. İçinde sadece boş adres referansları vardır. (Buna Relocatable Object File denir).

4. Linker (Bağlayıcı)

Bütün .o dosyalarını ve dış kütüphaneleri birleştirip o boş adresleri dolduran (Symbol Resolution) son aşamadır.

- **Mülakat Sorusu:** "Undefined reference to 'hesapla' hatası nedir?"
- **Cevap:** Bu bir Derleyici (Compiler) hatası değil, Linker hatasıdır. Derleyici fonksiyonun imzasını header dosyasında görmüş ve sorun etmemiştir, ama Linker birleştirmeye çalışırken fonksiyonun gövdesini (implementasyonunu) içeren .o dosyasını bulamamıştır.
- **Statik vs Dinamik Linkleme:** Eğer kütüphaneler (.lib, .a) doğrudan exe'nin içine kopyalanırsa Statik; exe'nin içine sadece bir referans koyulup, çalışırken dışarıdan (.dll, .so) okunması istenirse Dinamik linkleme yapılmış olur.

B. Çalıştırma Aşaması (Execution / Loading)

Exe'ye çift tıkladın (veya terminalde ./program yazdın). Artık iş Derleyiciden çıktı, tamamen İşletim Sisteminin (OS) elinde.

- **OS Loader (Yükleyici) Devreye Girer:** OS, executable dosyanın başlığını (Linux'ta ELF - Executable and Linkable Format, Windows'ta PE) okur. Bu başlıkta programın ne kadar belleğe ihtiyacı olduğu ve nerede başlayacağı yazar.
- **Sanal Bellek (Virtual Memory) Oluşturulur:** OS, bu yeni Process için izole bir bellek alanı (Memory Layout) kurar:
- Text Segment: Makine kodları buraya yüklenir (Sadece okunabilirdir, değiştirilemez).
- Data Segment: İlk değeri atanmış global ve static değişkenler buraya konur.
- BSS Segment: İlk değeri atanmamış (sıfır olan) global/static değişkenler için yer ayrılır.
- Heap ve Stack: Dinamik bellek tahsisi ve yerel değişkenler için sınırları belirlenir.
- **Dinamik Kütüphanelerin Yüklenmesi:** Eğer program paylaşımlı kütüphaneler (Shared Libraries - .so veya .dll) kullanıyorsa, Dynamic Linker bunları RAM'e yükler ve programın bunlara erişebilmesi için gerekli adres bağlantılarını kurar.
- **Kontrolün Devredilmesi (Entry Point):** İşletim sistemi CPU'nun Program Counter register'ını, uygulamanın başlangıç noktasına ayarlar. Bu nokta genellikle doğrudan senin yazdığın main() fonksiyonu değildir. C'nin kendi başlama rutini olan _start veya crt0 (C Runtime Zero) fonksiyonudur. Bu fonksiyon işletim sisteminden argümanları (argc, argv) alır, ortam değişkenlerini ayarlar ve en sonunda main() fonksiyonunu çağırır.

Program Çalışır.

Nesne Yönelimli Programlama (OOP) ve C++

Encapsulation, Abstraction, Inheritance ve Polymorphism

> **SORU:** *Encapsulation, Abstraction, Inheritance ve Polymorphism*

1. Mülakat Cevabı

> *"Bu dört kavram, Nesne Yönelimli Programlamanın (OOP) temel yapıtaşlarıdır.*
>
> *Encapsulation (Kapsülleme): Veriyi (değişkenleri) ve bu veriyi işleyen fonksiyonları (metotları) tek bir birim (sınıf) içinde gruplamak ve veriyi dış dünyanın doğrudan erişimine kapatmaktır. Değişkenler private yapılarak gizlenir, kontrollü erişim için public metotlar (Getter/Setter) sunulur. Bu, verinin bütünlüğünü ve güvenliğini sağlar.*
>
> *Abstraction (Soyutlama): Karmaşık bir sistemin sadece kullanıcının ihtiyaç duyduğu önemli özelliklerini dışarıya sunup, gereksiz arka plan detaylarını gizlemektir. Abstract Class'lar veya Interface'ler kullanılarak 'Nasıl yapıldığı' (implementasyon) değil, 'Ne yapıldığı' öne çıkarılır. (Örneğin bir arabanın motorunun nasıl çalıştığını bilmeden sadece direksiyon ve pedallarla arabayı kullanabilmek gibi).*
>
> *Inheritance (Kalıtım/Miras): Bir sınıfın (alt sınıf/subclass), başka bir sınıfın (üst sınıf/base class) özelliklerini (değişkenler) ve yeteneklerini (metotlar) miras almasıdır. Ortak kodların tekrar tekrar yazılmasını engeller (Code Reusability) ve sınıflar arasında hiyerarşik bir 'Is-A' (bir türüdür) ilişkisi kurar.*
>
> *Polymorphism (Çok Biçimlilik): Aynı isimdeki bir yeteneğin veya metodun, farklı nesneler (sınıflar) tarafından farklı şekillerde uygulanabilmesidir. Örneğin ciz() komutu verildiğinde, bu komutu alan Çember nesnesinin yuvarlak, Kare nesnesinin ise köşeli çizim yapmasıdır. Ortak bir arayüz (Interface veya Base Class) üzerinden farklı alt sınıfları aynı tipmiş gibi yönetebilmemizi sağlar."*

2. Detaylı Arka Plan

Mülakatlarda bu kavramları saymak yetmez, mülakatçı "Peki Abstraction ile Encapsulation arasındaki fark nedir?" veya "Polymorphism arka planda nasıl çalışır?" gibi sorularla konuyu derinleştirecektir.

1. Encapsulation (Veri Gizleme)

- **Sorun:** Bir sınıftaki verilere herkesin müdahale edebilmesi sistemi kararsız hale getirir.
- **Çözüm:** Veriyi sarmalamak (kapsüllemek).
- **Mülakat Detayı:** Encapsulation sadece private yapmak demek değildir. Aynı zamanda Information Hiding (Bilgi Gizleme) prensibidir. Bir sınıfın iç state'i (durumu) tamamen o sınıfın sorumluluğunda olmalıdır. Sınıfın dışı, sınıfın içine güvenmemelidir, sınıfın içi de dışarıdan gelen veriye güvenmemelidir.

2. Abstraction (Tasarım Gizleme)

Mülakatların klasik kafa karıştırıcı sorusu: "Encapsulation ile Abstraction aynı şey değil mi? İkisi de bir şeyleri gizliyor."

Kritik Ayrım:

- Encapsulation veriyi (değişkenleri ve state'i) gizler. Amaç güvenliği ve bütünlüğü sağlamaktır.
- Abstraction ise karmaşıklığı (tasarımı ve kodun nasıl çalıştığını) gizler. Amaç sistemi basitleştirmek ve kullanım kolaylığı sağlamaktır.

Örnek: Bir Veritabani sınıfın olsun. private String dbSifresi; yapmak Encapsulation'dır (veriyi korudun). Dışarıya sadece baglan() diye tek bir public metot sunup, içerideki o karmaşık soket bağlantılarını, retry mekanizmalarını dış dünyadan gizlemek Abstraction'dır.

3. Inheritance (Kalıtım)

Inheritance, kod tekrarını önlemenin en güçlü ama en tehlikeli yoludur.

Mülakat Tuzağı: "Her ortak kod gördüğümüzde kalıtım mı kullanmalıyız?"

Cevap: Hayır. Kalıtım sadece gerçek bir "Is-A" (bir türüdür) ilişkisi varsa kullanılmalıdır. Kedi bir Hayvandır. Ancak araba bir motor değildir, arabanın bir motoru vardır (Has-A ilişkisi). "Has-A" ilişkilerinde kalıtım değil, Composition (Bileşim) kullanılmalıdır. Modern yazılım mimarisinde "Favor Composition over Inheritance" (Kalıtım yerine bileşimi tercih et) altın bir kuraldır, çünkü derin kalıtım ağaçları (örneğin A'dan B türer, B'den C, C'den D) sistemi çok kırılgan yapar (Fragile Base Class Problem).

4. Polymorphism (Çok Biçimlilik)

Polymorphism, sistemin esnek (flexible) ve genişletilebilir (extensible) olmasını sağlayan temel kavramdır. İkiye ayrılır:

- **Compile-time Polymorphism (Static Binding / Overloading):** Aynı sınıf içinde, metodun isminin aynı ancak aldığı parametrelerin (imzasının) farklı olmasıdır. Hangi metodun çağrılacağına derleme (compile) aşamasında karar verilir.
- Örnek: topla(int a, int b) ve topla(double a, double b)
- **Run-time Polymorphism (Dynamic Binding / Overriding):** Mülakatların asıl odak noktası budur. Bir alt sınıfın, üst sınıftan miras aldığı bir metodu kendi ihtiyacına göre yeniden yazmasıdır (ezmesi).

Arka Plandaki C++ / Vtable Detayı:

Mülakatçı sorar: "Program çalışırken, hafızadaki nesnenin Kedi mi yoksa Köpek mi olduğunu ve hangi sesCikar() metodunu çağıracağını nasıl biliyor?"

Cevap (C++ gibi diller için) Virtual Table (Vtable) mekanizmasıdır.

Bir sınıfta sanal (virtual) bir metot tanımlandığında, derleyici o sınıf için gizli bir tablo (vtable) oluşturur. Bu tablo, o sınıfa ait override edilmiş fonksiyonların bellek adreslerini (function pointers) tutar. Obje yaratıldığında, objenin içine bu tabloyu gösteren gizli bir pointer (vptr) eklenir. Çalışma zamanında (runtime) bir metot çağrıldığında, program önce objenin vptr'sine bakar, doğru vtable'a gider ve çağrılacak doğru metodun adresini oradan bularak çalıştırır. (Bu ekstra pointer okuması (indirection) nedeniyle Run-time polymorphism ufak bir performans maliyeti getirir).

Abstract Class ve Interface

> **SORU:** *abstract ve interference*

1. Mülakat Cevabı

> *"Nesne Yönelimli Programlamada (OOP) Abstract Class (Soyut Sınıf) ve Interface (Arayüz), soyutlama (abstraction) ve kalıtım (inheritance) mekanizmalarının iki temel aracıdır.*
>
> *Abstract Class, 'Bu nesne nedir?' (Is-A ilişkisi) sorusuna cevap verir. Ortak bir kökenden gelen nesneler için temel bir şablon sunar. İçerisinde hem tamamlanmış (gövdeli) metotlar hem de sadece imzası bulunan (gövdesiz) soyut metotlar barındırabilir. Durum (state/değişken) tutabilir. Bir sınıf yalnızca tek bir Abstract Class'ı miras alabilir.*
>
> *Interface ise, 'Bu nesne ne yapabilir?' (Can-Do ilişkisi) sorusuna cevap verir. Sınıfların dış dünyaya sunduğu bir sözleşmedir (contract). Geleneksel olarak sadece metot imzalarını barındırır, gövdeli metot veya durum (state) barındırmaz. En büyük avantajı, bir sınıfın birden fazla Interface'i implement (uygulama) edebilmesidir; bu da çoklu kalıtım (multiple inheritance) sorunlarını çözer."*

2. Detaylı Arka Plan

(Not: Başlığında "interference" (girişim/müdahale) yazmışsın, ancak OOP bağlamında mülakatlarda sorulan ikili kesinlikle Interface (Arayüz) kavramıdır. Bu nedenle cevabı Interface üzerinden kurguladım. Eğer fizik veya network bağlamındaki "interference"ı sorduysan lütfen belirt.)

Mülakatlarda "Abstract class vs Interface" sorusu, OOP bilginin derinliğini ölçmek için kullanılan en klasik sorulardan biridir. Sadece tanımlarını bilmek yetmez, hangi durumda hangisini kullanacağını savunman gerekir.

1. Abstract Class (Soyut Sınıf)

Soyut sınıflar, birbiriyle genetik bir bağı olan nesneler için ortak bir taban (base) oluşturur.

Özellikleri:

- Doğrudan örneği (instance) oluşturulamaz (new AbstractClass() yapılamaz).
- İçinde normal (gövdeli) fonksiyonlar ve değişkenler (state) bulunabilir.
- Sadece gövdesi olmayan abstract metotların alt sınıflar (subclasses) tarafından ezilmesi (override) zorunludur.

Gerçek Hayat Mülakat Örneği:

Bir oyun yapıyorsun. Dusman adında bir soyut sınıfın var.

```Java
abstract class Dusman {
    int canPointi = 100; // State (Durum) tutabilir

    void hareketEt() { // Gövdeli metot: Bütün düşmanlar aynı şekilde yürür
        System.out.println("İleri gidiyor...");
    }

    abstract void saldir(); // Gövdesiz metot: Her düşmanın saldırı şekli farklıdır!
}
```

Zombi sınıfı Dusmandan türerse, hareketEt metodunu bedavaya alır ama saldir metodunu kendine göre yazmak zorundadır (örneğin ısırarak). Burada bir "Is-A" ilişkisi vardır: Zombi bir Düşmandır.

2. Interface (Arayüz)

Interface, sınıflar arası bir yetenek sözleşmesidir. Birbiriyle genetik hiçbir bağı olmayan sınıflara aynı davranışı kazandırmak için kullanılır.

Özellikleri:

- Geleneksel olarak içinde durum (state/değişken) tutulamaz (sadece static final sabitler olabilir).
- Tüm metotlar varsayılan olarak public ve abstract'tır (gövdesizdir). (Not: Java 8+ ile birlikte default anahtar kelimesiyle gövdeli metotlar eklenebilmiştir, ancak mülakatlarda temel tanım her zaman gövdesiz olması üzerinedir).
- Bir sınıf birden fazla interface'i uygulayabilir (Multiple Implementation).

Gerçek Hayat Mülakat Örneği:

Sisteminde bir Ucak sınıfı ve bir Kus sınıfı var. Bunların ortak bir atası yoktur (Biri makine, biri hayvan). Ama ikisi de uçabilir.

```Java
interface Ucabilir {
    void uc();
}

class Kus implements Ucabilir {
    public void uc() { System.out.println("Kanat çırparak uçuyor."); }
}

class Ucak implements Ucabilir {
    public void uc() { System.out.println("Motor çalıştırarak uçuyor."); }
}
```

Burada "Can-Do" ilişkisi vardır: Uçak uçma yeteneğine sahiptir.

Mülakatın Kırılma Noktası: Ne Zaman Hangisi?

Mülakatçı şu soruyu soracaktır: "İkisi de metot imzası tutabiliyor ve override edilmeye zorluyor. O zaman ben neden Abstract Class kullanayım, her şeyi Interface yapsam ya?"

Kritik Cevapların:

- **Kod Tekrarı (Code Duplication):** Eğer alt sınıfların ortaklaşa kullanacağı bir kod bloğu veya durum (değişken) varsa, Interface kullanamazsın. Abstract class, bu ortak kodu tek bir yerde (base class'ta) yazıp tüm alt sınıflara miras bırakmanı sağlar.
- **Çoklu Kalıtım (Diamond Problem):** Çoğu modern dil (Java, C#) iki farklı sınıftan miras almayı (multiple inheritance) yasaklar çünkü iki sınıfta aynı isimli metot varsa hangisinin çağrılacağı karışır. Ancak bir sınıf istediği kadar Interface uygulayabilir, çünkü Interface'lerde sadece imza vardır (gövde yoktur), bu yüzden çakışma riski bulunmaz.
- **Versiyonlama ve Geriye Dönük Uyumluluk:** Zaten yayında olan (production) bir Interface'e yeni bir metot eklersen, o Interface'i uygulayan tüm sınıflar patlar (çünkü yeni metodu override etmemişlerdir). Ancak Abstract Class'a gövdeli yeni bir normal metot eklersen, alt sınıflar bozulmaz, bu metodu doğrudan miras alırlar.

public, private, protected (Erişim Belirleyiciler)

> **SORU:** *public, private, protected değişkenleri açıkla*

1. Mülakat Cevabı

> *"Nesne Yönelimli Programlamada (OOP) public, private ve protected, sınıfların (class) içindeki üyelere (değişkenler ve metotlar) kimlerin erişebileceğini belirleyen Erişim Belirleyicilerdir (Access Modifiers). Bu kavramlar, OOP'nin en temel prensibi olan Kapsülleme (Encapsulation) ve Veri Gizleme (Data Hiding) işlemlerini sağlar.*
>
> *Public (Genel): Hiçbir erişim kısıtlaması yoktur. public olarak tanımlanmış bir değişkene veya metoda, sınıfın içinden, alt sınıflardan (subclasses) veya tamamen dışarıdaki herhangi bir sınıftan doğrudan erişilebilir.*
>
> *Private (Özel): En sıkı erişim seviyesidir. private bir değişkene sadece tanımlandığı sınıfın kendi içinden erişilebilir. Ne alt sınıflar (miras alanlar) ne de dışarıdaki nesneler bu veriyi göremez.*
>
> *Protected (Korumalı): private ile public arasında bir köprüdür. protected üyelere, tanımlandığı sınıfın içinden ve o sınıftan miras alan alt sınıfların (subclasses) içinden erişilebilir. Ancak dışarıdaki alakasız sınıflar için bu üyeler private gibi davranır ve gizlidir."*

(Not: Java ve C# gibi dillerde 'package/internal' gibi ekstra belirleyiciler de vardır, ancak temel OOP konsepti bu üçü üzerine kuruludur).

2. Detaylı Arka Plan

Mülakatlarda sadece bu üç terimin tanımını yapmak yeterli değildir. Mülakatçı, bu belirleyicileri neden kullandığını, mimari kararları nasıl etkilediklerini ve pratikteki sonuçlarını görmek ister.

Neden Her Şeyi Public Yapmıyoruz? (Kapsülleme - Encapsulation)

Junior geliştiriciler genellikle işler kolay olsun diye tüm değişkenleri public yapar. Mülakatçı "Neden sınıf değişkenlerini private yapıp, onlara ulaşmak için Getter/Setter (veya Property) yazıyoruz?" diye soracaktır.

Kritik Cevap: Kontrol ve Güvenlik.

Eğer bir BankaHesabi sınıfında bakiye değişkenini public yaparsan, dışarıdan herhangi biri hesap.bakiye = -5000; yazabilir.

Ancak değişkeni private yapıp, dışarıya sadece bir paraCek(int miktar) adlı public metot sunarsan (Kapsülleme), kontrolü eline alırsın. Metodun içine if (miktar > bakiye) gibi iş kuralları (business logic) ekleyerek verinin bozulmasını (invalid state) engellersin.

Protected'ın Gücü ve Riski

Mülakatların en can alıcı noktalarından biri protected kullanımıdır.

Diyelim ki bir Karakter (Base) sınıfın ve ondan türeyen Savasci (Derived) sınıfın var. canPointi değişkenini nerede tutacaksın?

- private yaparsan, Savasci sınıfı bu canı doğrudan azaltamaz, Base sınıfa yalvarması (onun metotlarını çağırması) gerekir.
- protected yaparsan, Savasci sınıfı canPointi değişkenini sanki kendi değişkeniymiş gibi doğrudan okuyup değiştirebilir (örneğin this.canPointi -= 10;). Dışarıdaki bir Dusman sınıfı ise Savasci'nın canına doğrudan müdahale edemez.

Risk: protected kullanımı, Kalıtım (Inheritance) yoluyla kapsüllemeyi kırar. Eğer Karakter sınıfını yazan kişi canPointi değişkeninin adını saglikDurumu olarak değiştirirse, ondan türeyen ve o protected değişkene doğrudan erişen yüzlerce alt sınıf aynı anda patlar (Compile error). Bu yüzden modern yazılım mimarisinde,

zorunlu kalmadıkça değişkenleri private tutup, alt sınıflara bile protected metotlar (getter/setter) üzerinden erişim vermek daha güvenli kabul edilir.

Özet Karşılaştırma Tablosu (Genel OOP Yaklaşımı)

Erişim Belirleyici	Kendi Sınıfı İçinden (Same Class)	Alt Sınıftan (Subclass / Derived)	Dışarıdan (Alakasız Sınıflar)
Public	Evet	Evet	Evet
Protected	Evet	Evet	Hayır
Private	Evet	Hayır	Hayır

(Not: C++'ta protected veya private miras alma (inheritance) gibi daha spesifik kavramlar da vardır, ancak bunlar dil spesifik sorulardır, genel OOP mimarisi yukarıdaki gibidir).

C++'ta struct ve class Farkı

> **SORU:** *Cpp structer ve class farkı*

1. Mülakat Cevabı

> *"C++ dilinde struct (yapı) ve class (sınıf) arasında teknik (derleyici seviyesinde) sadece tek bir fark vardır: Varsayılan (default) erişim ve kalıtım belirleyicileri.*
>
> *Bir struct içinde tanımlanan tüm değişkenler, metotlar ve kalıtımlar (inheritance) siz aksini belirtmedikçe varsayılan olarak public (herkese açık) kabul edilir. Bir class içinde ise her şey varsayılan olarak private (gizli) kabul edilir.*
>
> *Bunun dışındaki her şey (Constructor/Destructor tanımlama, metot ekleme, virtual fonksiyon kullanımı vb.) her ikisi için de tamamen aynıdır. Ancak endüstri standardı (convention) olarak; sadece birbirine bağlı verileri (POD - Plain Old Data) bir arada tutmak için struct kullanırken, veriyi gizleme (encapsulation) ve davranış (metot) barındıran tam teşekküllü Nesne Yönelimli Programlama (OOP) yapıları için class kullanmayı tercih ederiz."*

2. Detaylı Arka Plan

Bu soru, Junior/Mid-level C++ mülakatlarının en meşhur "şaşırtmaca" sorularından biridir. Mülakatçı, senin C dili ile C++ dili arasındaki tarihsel geçişi ve nesne yönelimli programlama felsefesini bilip bilmediğini test eder.

Mülakat Tuzağı: "Struct İçine Metot Yazılır Mı?"

Eğer geçmişinde saf C programlama varsa, mülakatçının şu sorusuna düşebilirsin: "C++'ta bir struct'ın içine Constructor (Yapıcı) veya bir fonksiyon yazabilir miyim?"

C dilinde struct sadece değişken (veri) tutabilir, içine fonksiyon yazılamaz. Ancak C++'ta yazılır!

C++, C dilinin üzerine inşa edildiği için geriye dönük uyumluluğu (backward compatibility) korumak zorundaydı. Bu yüzden eski C kodlarının bozulmaması için struct yapısını korudu, ancak ona sınıfların (class) tüm yeteneklerini verdi. Ardından OOP felsefesindeki "veri gizliliği" (Encapsulation) kuralına uymak için, her şeyi varsayılan olarak gizli (private) yapan class anahtar kelimesini dile ekledi.

Örnek Kod Üzerinde Kanıt:

```C++
// STRUCT Örneği
struct KediStruct {
    // Burada public: yazmamıza gerek yoktur, varsayılan olarak public'tir.
    int yas;
    void miyavla() { cout << "Miyav!"; }
};

// CLASS Örneği
class KediClass {
    // Burada private: yazmamıza gerek yoktur, varsayılan olarak private'tir.
    int yas;
    void miyavla() { cout << "Miyav!"; }
};

int main() {
    KediStruct kedi1;
    kedi1.yas = 2;       // BAŞARILI! Çünkü struct'ta yas public'tir.
    kedi1.miyavla();     // BAŞARILI!

    KediClass kedi2;
    // kedi2.yas = 2;    // HATA (Compile Error)! Çünkü class'ta yas private'tir.
    // kedi2.miyavla();  // HATA! Metot da private'tir.
```

```
    return 0;
}
```

Eğer KediClass içindeki int yas; satırının hemen üstüne public: yazarsanız, bu iki yapı derleyici (compiler) gözünde %100 aynı makine koduna dönüşür.

Kalıtım (Inheritance) Farkı

Mülakatlarda detaycı bir mülakatçı kalıtım farkını da sorabilir. Tıpkı değişkenlerde olduğu gibi, sınıflar arası miras alırken de erişim belirleyicileri farklı çalışır:

```C++
class Base {};

// Eğer bir STRUCT miras alırsa, varsayılan olarak PUBLIC kalıtım yapar.
// struct Derived : public Base ile aynıdır.
struct DerivedStruct : Base {};

// Eğer bir CLASS miras alırsa, varsayılan olarak PRIVATE kalıtım yapar.
// class Derived : private Base ile aynıdır.
class DerivedClass : Base {};
```

Karşılaştırma Özeti

Özellik	struct	class
Varsayılan Üye Erişimi	public	private
Varsayılan Kalıtım Türü	public	private
Metot / Constructor / Virtual	Destekler (C'nin aksine)	Destekler
Kullanım Felsefesi (Convention)	Metodu olmayan, sadece veri (Data) taşıyan pasif veri paketleri (Örn: X,Y koordinatları)	Kapsülleme (Encapsulation) gerektiren, mantık barındıran aktif nesneler

C++'ta Encapsulation (Kapsülleme)

› Virtual Destructor Neden Gerekli?

› C++11: struct padding ve bellek

C++'ta malloc ve new Farkı

> **SORU:** *cpp malloc ve new farkı nedir?*

1. Mülakat Cevabı

> *"Temel fark, Nesne Yönelimli Programlama (OOP) desteği ve tür güvenliğidir (type safety).*
>
> *malloc, C dilinden miras kalan bir standart kütüphane fonksiyonudur. Yalnızca belirttiğiniz byte miktarı kadar ham (raw) bellek tahsis eder, objenin ne olduğuyla ilgilenmez ve dönüş tipi void* olduğu için tür dönüşümü (casting) gerektirir.*
>
> *new ise C++'a özgü bir operatördür. En büyük gücü, belleği tahsis ettikten hemen sonra o sınıfın Constructor'ını (Yapıcı Metot) otomatik olarak çağırmasıdır. Doğrudan ilgili sınıfın pointer tipini döndürür. Ayrıca, temizleme aşamasında free sadece belleği silerken, new'un karşılığı olan delete operatörü belleği silmeden önce nesnenin Destructor'ını (Yıkıcı Metot) çağırarak nesnenin içindeki kaynakların (dosyalar, soketler vb.) güvenle kapatılmasını sağlar."*

2. Detaylı Arka Plan

C++ mülakatlarında bu soru bir "ısınma" sorusudur. Ancak mülakatçı, cevabının ardından belleğin arka planında neler döndüğünü ve ikisini birbirine karıştırırsan ne tür felaketler yaşanacağını soracaktır.

1. Nesne Yaratımı (Constructor ve Destructor Tuzağı)

Mülakatlarda en sık karşılaşılan tuzak soru şudur: "C++'ta bir sınıfım (Class) var. Onu malloc ile yaratırsam ne olur?"

```C++
class Veritabani {
public:
    Veritabani() { cout << "Bağlantı Açıldı!"; }
    ~Veritabani() { cout << "Bağlantı Kapatıldı!"; }
};

// Durum 1: malloc kullanımı
Veritabani* db1 = (Veritabani*)malloc(sizeof(Veritabani));

// Durum 2: new kullanımı
Veritabani* db2 = new Veritabani();
```

Cevap: malloc kullanıldığında kod derlenir, bellekte o nesne kadar yer ayrılır ama ekranda "Bağlantı Açıldı!" yazısını göremezsin. Çünkü malloc aptaldır; Constructor'ı çağırmaz. Nesne bellekte vardır ama başlatılmamıştır (uninitialized), içindeki pointer'lar çöp (garbage) değerlerle doludur. new ise belleği ayırdıktan sonra arka planda Constructor'ı tetikler ve nesneyi güvenli bir duruma getirir (Valid state).

2. Bellek İade Edilirken Çapraz Kullanım Tehlikesi

Mülakatçı sorar: "Bir nesneyi new ile yaratıp, free() ile silersem ne olur?"

Kritik Cevap: "Bu işlem Undefined Behavior (Tanımsız Davranış) yaratır ve çok büyük ihtimalle bir Kaynak Sızıntısına (Resource Leak) yol açar."

new ile yaratılan bir nesne silinirken delete kullanılmalıdır çünkü delete önce Destructor'ı (Yıkıcı Metot) çağırır, sonra belleği iade eder. Eğer free() kullanırsan, bellek işletim sistemine iade edilir ancak Destructor asla çalışmaz. Eğer o nesne kendi içinde new ile başka bellekler ayırdıysa veya bir veritabanı bağlantısı açtıysa, o bağlantılar sonsuza kadar açık ve ulaşılamaz kalır. (Aynı kural malloc ile ayrılan belleği delete ile silmeye çalışmak için de geçerlidir).

3. Hata Yönetimi (Error Handling)

Sistemde tahsis edilecek yeterli RAM kalmadığında (Out of Memory) ikisi tamamen farklı davranır:

- **malloc:** Eğer bellek ayıramazsa geriye NULL (sıfır) döndürür. Eski usul C programcıları her malloc çağrısından sonra if (ptr == NULL) kontrolü yapmak zorundadır.
- **new:** Modern C++ felsefesine uygun olarak NULL döndürmez, doğrudan std::bad_alloc hatası (exception) fırlatır. Programcı bunu try/catch blokları ile yakalamalıdır. (İstisna: Eğer C++'ta new'un exception fırlatmasını istemiyorsan, mülakatçıya ekstra puan getirecek şu syntax'ı söyleyebilirsin: new (std::nothrow) Obje(); Bu durumda aynı malloc gibi NULL döndürür).

Özet Karşılaştırma Tablosu

Özellik	malloc	new
Türü	Fonksiyon (Function)	Operatör (Operator)
Dönüş Tipi	void* (Type casting gerekir)	İlgili sınıfın pointer'ı (Type-safe)
Boyut Hesaplama	Manuel (Örn: 10 * sizeof(int))	Otomatik (Örn: new int[10])
Constructor/Destructor	Çağırmaz	Çağırır
Hata Durumu	NULL döndürür	Exception (istisna) fırlatır
Yeniden Boyutlandırma	realloc() ile genişletilebilir	Doğrudan karşılığı yoktur (Vektör kullanılır)
Overloading (Aşırı Yükleme)	Sınıf bazında yapılamaz	Sınıflara özel yeniden yazılabilir (Overloadable)

C++'ta delete ve delete[] farkı

› *Modern C++: Smart Pointers*

› *C++'ta RAII Prensibi*

Smart Pointers (Akıllı Göstericiler)

> **SORU:** *smart pointerları açıkla*

1. Mülakat Cevabı

> *"Smart Pointers (Akıllı Göstericiler), C++11 ile birlikte standart kütüphaneye (<memory>) eklenen, dinamik bellek yönetimini otomatikleştirerek bellek sızıntılarını (Memory Leak) ve sarkan göstericileri (Dangling Pointer) önleyen araçlardır.*
>
> *Geleneksel (Raw) pointer'larda, new ile alınan belleği işimiz bittiğinde delete ile manuel olarak iade etmemiz gerekir; aksi takdirde sızıntı olur. Smart Pointer'lar ise RAII (Resource Acquisition Is Initialization) prensibiyle çalışır. Arka planda bir sınıf gibi davranırlar ve kapsam (scope) dışına çıktıklarında Yıkıcıları (Destructor) otomatik olarak çağrılarak içerdikleri belleği güvenle silerler.*
>
> *C++'ta üç temel Smart Pointer vardır:*
>
> *std::unique_ptr: Sahipliği tekildir (Exclusive Ownership). Kopyalanamaz, sadece taşınabilir (move). En hafif ve en çok kullanılandır.*
>
> *std::shared_ptr: Sahipliği paylaşılabilir. Arka planda bir referans sayacı (Reference Counter) tutar. Sayac sıfırlandığında belleği siler.*
>
> *std::weak_ptr: shared_ptr'nin döngüsel bağımlılık (Cyclic Reference) yarattığı hatalı durumlarda referans sayacını artırmadan gözlem (observe) yapmak için kullanılır."*

2. Detaylı Arka Plan

Modern C++ mülakatlarında ham (raw) pointer ile kod yazarsanız büyük ihtimalle reddedilirsiniz. Mülakatçı bu 3 akıllı göstericinin farklarını ve nasıl çalıştıklarını detaylıca soracaktır.

1. std::unique_ptr (Tek ve Gerçek Sahip)

Kural: Bir nesnenin bellekteki adresini aynı anda sadece bir tane unique_ptr tutabilir. Kopyalanması yasaktır.

Nasıl Kullanılır? (C++14'te gelen std::make_unique ile kullanılması güvenlidir).

```C++
{
    std::unique_ptr<Kedi> kedi1 = std::make_unique<Kedi>("Tekir");
    kedi1->miyavla();
    // std::unique_ptr<Kedi> kedi2 = kedi1; // HATA! Kopyalanamaz!
    // Sahipliği devretmek istiyorsanız move kullanmalısınız:
    std::unique_ptr<Kedi> kedi2 = std::move(kedi1);
    // kedi1 artık NULL'dır, nesnenin yeni sahibi kedi2'dir.
}
// Scope bittiğinde kedi2'nin Destructor'ı çalışır ve Kedi nesnesi otomatik silinir.
```

Mülakat Notu: unique_ptr kullanmanın normal (raw) bir pointer kullanmaya göre hiçbir ekstra bellek veya CPU maliyeti yoktur (Zero-overhead abstraction). Bu yüzden her zaman varsayılan tercihiniz olmalıdır.

2. std::shared_ptr (Paylaşımlı Sahiplik)

Kural: Bir nesnenin adresini aynı anda birden fazla shared_ptr tutabilir. Kopyalandıkça arka plandaki referans sayacı (Reference Count) 1 artar. Bir pointer yok olduğunda sayaç 1 azalır. Sayaç 0 olduğunda bellek silinir.

Nasıl Kullanılır? (Performans ve güvenlik için std::make_shared ile kullanılmalıdır).

```C++
{
    std::shared_ptr<Dosya> p1 = std::make_shared<Dosya>("veri.txt"); // Sayaç = 1
    {
        std::shared_ptr<Dosya> p2 = p1; // KOPYALANDI! Sayaç = 2 oldu.
```

```
        p2->oku();
    } // p2'nin scope'u bitti. Sayaç = 1'e düştü. Dosya HALA HAYATTA!
    p1->oku();
} // p1'in scope'u bitti. Sayaç = 0 oldu. Dosya SİLİNDİ!
```

Dezavantajı: Arka planda bir sayaç tuttuğu için bellekte ekstra yer kaplar ve sayacı artırıp azaltmak (atomik işlemler gerektirdiği için) minik bir CPU maliyeti yaratır.

3. std::weak_ptr (Gözlemci / Hata Çözücü)

Mülakatın en eleyici sorusu buradan gelir: "Madem shared_ptr var, weak_ptr'ye neden ihtiyaç duyuyoruz?"

Cevap (Döngüsel Bağımlılık - Cyclic Reference Problemi):

Diyelim ki bir Anne ve Cocuk sınıfı yazdık ve birbirlerini shared_ptr ile gösteriyorlar.

- Anne, Çocuğu gösteriyor (Çocuğun sayacı = 1)
- Çocuk, Anneyi gösteriyor (Annenin sayacı = 1)

Program bittiğinde ikisinin de scope'u biter, ancak birbirlerini işaret ettikleri için sayaçları asla 0'a düşmez. Ortaya devasa bir Memory Leak (Bellek Sızıntısı) çıkar.

Çözüm: Bu referans zincirini kırmak için bir tarafı weak_ptr yaparız. weak_ptr bir nesneyi işaret eder ama sayacı artırmaz.

Genelde ebeveyn nesne çocuğu shared_ptr ile tutar, çocuk nesne ise ebeveyni weak_ptr ile tutar.

Kullanımı (Tuzak!): weak_ptr üzerindeki veriye doğrudan -> operatörüyle erişemezsiniz. Önce o nesnenin bellekte hala yaşayıp yaşamadığını kontrol etmeniz gerekir. Bunun için lock() metodu kullanılarak geçici bir shared_ptr yaratılır.

```C++
std::weak_ptr<Cocuk> zayifPtr = orjinalSharedPtr;
// Zayıf pointer doğrudan kullanılamaz, önce lock() edilir:
if (std::shared_ptr<Cocuk> geciciPtr = zayifPtr.lock()) {
    geciciPtr->oyunOyna(); // Nesne hala hayatta, güvenle kullan.
} else {
    // Nesne çoktan silinmiş!
}
```

RAII Prensibi Nedir?

› make_shared Neden Önemli?

› C++'ta L-value ve R-value

Casting (Tür Dönüşümü)

> **SORU:** *Casting nedir ?*

1. Mülakat Cevabı

> *"Casting (Tür Dönüşümü), programlamada bir değişkenin veya nesnenin mevcut veri tipini başka bir veri tipine dönüştürme işlemidir.*
>
> *Casting temelde ikiye ayrılır: Derleyicinin bizim yerimize otomatik yaptığı Implicit (Örtülü) Casting (örneğin bir int değerini float değişkene atamak) ve programcının manuel olarak zorladığı Explicit (Açık) Casting.*
>
> *Eğer bir C++ mülakatındaysak, geleneksel C-tarzı (int)degisken dönüşümleri güvensiz kabul edildiği için kullanılmamalıdır. C++ derleyicisine ne yapmak istediğimizi tam olarak belirtmek ve tip güvenliğini (type safety) sağlamak için C++'ın sunduğu 4 özel casting operatörünü kullanırız: static_cast (genel güvenli dönüşümler), dynamic_cast (polimorfik sınıflar arası güvenli alt-dönüşüm), const_cast (sabitliği kaldırmak) ve reinterpret_cast (düşük seviyeli tehlikeli bellek dönüşümleri)."*

2. Detaylı Arka Plan

C++ mülakatlarında "Casting nedir?" sorusu asla basit bir tanım sorusu değildir. Mülakatçı, eski usul C-tarzı casting'in neden C++'ta dışlandığını ve C++'ın 4 silahşörünü (cast operatörlerini) ne kadar anladığını test etmek için bu soruyu sorar.

Neden C-Tarzı Casting (Tip)Degisken C++'ta Kötüdür?

C dilinde bir pointer'ı tam sayıya çevirmek istersen (int)ptr yazarsın. Aynı sözdizimiyle bir float'ı int'e de çevirebilirsin (int)pi.

Derleyici burada kördür; sen ne yazarsan onu yapar ve hata vermez. Ayrıca binlerce satırlık bir kodda birisi nerede tip dönüşümü yapmış diye (int) araması yapamazsın, bulması imkansızdır.

C++ derleyicisi ise kodun derlenme aşamasında hataları yakalamak ister. Bu yüzden C++'ta casting, amacına göre 4 farklı spesifik araca bölünmüştür.

C++'ın 4 Casting Operatörü (Mülakatın Kalbi)

Mülakatçı büyük ihtimalle "Peki bu 4 cast operatörünün farkları nelerdir ve hangisini ne zaman kullanırsın?" diyecektir.

1. static_cast (Çalışkan Araç)

Günlük hayatta C-tarzı casting'in yerini alan, en çok kullanacağın operatördür.

- Ne yapar? Derleme zamanında (compile-time) dönüşümün mantıklı olup olmadığını kontrol eder. float'ı int'e çevirir, void*'u int*'a çevirir. Birbirine dönüşmesi imkansız iki sınıfı dönüştürmeye çalışırsan kod derlenmez, seni hatadan kurtarır.
- **Örnek:** int x = static_cast<int>(3.14f);

2. dynamic_cast (Polimorfizm ve Güvenlik Uzmanı)

En çok mülakat sorusu buradan gelir. Sadece Kalıtım (Inheritance) olan ve içinde en az bir tane Virtual (Sanal) fonksiyon barındıran (Polimorfik) sınıflar arasında dönüşüm yapmak için kullanılır.

- Ne yapar? Dönüşümün güvenliğini derleme zamanında değil, Çalışma Zamanında (Run-time) kontrol eder. Arka planda RTTI (Run-Time Type Information) adı verilen bir sistem kullanır.
- **Tuzak:** Eğer bir Base (Temel) sınıf pointer'ını, aslında işaret etmediği bir Derived (Alt) sınıf pointer'ına çevirmeye (Downcasting) çalışırsan, programı çökertmek yerine sana NULL (veya

nullptr) döndürür. Böylece if(ptr != nullptr) diyerek güvenli bir şekilde nesnenin o sınıfa ait olup olmadığını kontrol edebilirsin.

3. const_cast (Kural Bozan)

Adından da anlaşılacağı gibi, bir değişkenin üzerindeki const (değiştirilemez) veya volatile etiketini kaldırmak (veya eklemek) için kullanılan tek casting yöntemidir.

Mülakat Tuzağı: "Madem bir değişkeni const tanımladın, neden const_cast ile bu kuralı deliyorsun?"

Cevap: Yeni yazdığımız C++ kodlarında bunu kullanmak kötü bir tasarımdır (Code Smell). Ancak eski bir C kütüphanesini veya dışarıdan aldığımız bir API'yi kullanmak zorundaysak ve o API bizden char* istiyorsa, ancak bizim elimizde const char* varsa, sadece o fonksiyona parametre geçebilmek için mecburen const_cast kullanırız.

4. reinterpret_cast (Tehlikeli ve Güçlü)

Donanım, işletim sistemi çekirdeği (Kernel) veya oyun motoru yazmıyorsan, normal bir uygulamada bunu kullanıyorsan muhtemelen bir şeyleri yanlış yapıyorsundur.

- Ne yapar? Derleyiciye "Sus ve bana güven, bu bellekteki bitleri (0 ve 1'leri) benim istediğim tipe çevir" der. Bir pointer'ı bir tam sayıya (int) veya bir Kedi* pointer'ını Araba* pointer'ına çevirebilir.
- **Kullanım Alanı:** Örneğin ağ üzerinden (Network) bir byte dizisi (char*) geldi. Sen o byte'ların aslında bir OyunKarakteri struct'ı olduğunu biliyorsun. O belleği doğrudan struct'a çevirmek (Serialization/Deserialization) için reinterpret_cast kullanırsın.

dynamic_cast ve RTTI Nasıl Çalışır?

› Upcasting ve Downcasting Farkı

› C++'ta Polimorfizm ve Virtual Fonksiyonlar

C++'ta extern

> **SORU:** *cpp extern nedir*

1. Mülakat Cevabı

> *"extern (external - harici) anahtar kelimesi, derleyiciye (compiler) 'Bu değişkenin veya fonksiyonun adını ve tipini bil, ancak bellekteki asıl yeri (tanımı) başka bir dosyada bulunuyor' demek için kullanılır.*
>
> *C ve C++'ta en temel amacı, Bildirim (Declaration) ile Tanımlama (Definition) işlemlerini birbirinden ayırmaktır. extern kullandığınızda işletim sisteminden bellek tahsis edilmez; derleyiciye sadece bir söz vermiş olursunuz.*
>
> *Mülakatlarda extern iki farklı senaryoda karşımıza çıkar:*
>
> *Global Değişken Paylaşımı: Birden fazla .cpp dosyasının aynı global değişkeni kullanabilmesi için Header (.h) dosyasında extern ile bildirim yapılır, değişken sadece tek bir .cpp dosyasında tanımlanır.*
>
> *extern "C" Kullanımı (Sadece C++): C++ derleyicisinin, C diliyle yazılmış kütüphaneleri (veya fonksiyonları) tanıyabilmesi ve Linker (Bağlayıcı) aşamasında İsim Bozma (Name Mangling) krizlerini önlemesi için kullanılır."*

2. Detaylı Arka Plan

Mülakatçı extern kelimesini sorduğunda, aslında senin derleme sürecini (Compilation Pipeline) ve Linker'ın (Bağlayıcı) nasıl çalıştığını bilip bilmediğini test ediyordur.

1. Global Değişken Paylaşımı ve Linker Hatası

Mülakatlarda genellikle şu senaryo verilir: "Projede bir veritabani.h dosyam var. İçine int port = 8080; yazdım. Bu başlık dosyasını main.cpp ve kullanici.cpp dosyalarına #include ettim. Kodu derlersem ne olur?"

Kritik Cevap: "Linker (Bağlayıcı) aşamasında 'Multiple Definition' (Çoklu Tanımlama) hatası alarak patlar."

Çünkü #include sadece metin kopyalama işlemidir. Hem main.cpp hem de kullanici.cpp dosyası derlendiğinde, her ikisinin de kendi .o (object) dosyasının içinde birer tane port değişkeni oluşur. Linker bu iki dosyayı birleştirip tek bir .exe yapmaya çalıştığında "Aynı isimde iki tane port değişkeni var, hangisini kullanacağım?" diyerek çöker.

Çözüm extern kullanmaktır:

veritabani.h (Sadece Bildirim):

```C++
// Derleyiciye söz veriyoruz: "port diye bir int var, ama belleği burada ayırma"
extern int port;
```

veritabani.cpp (Gerçek Tanımlama ve Bellek Tahsisi):

```C++
#include "veritabani.h"
int port = 8080; // Bellek sadece burada 1 kez ayrıldı.
```

main.cpp (Kullanım):

```C++
#include "veritabani.h"
// Derleyici extern'ü görür ve "Tamam, port'un adresi Linker aşamasında bulunacak" der.
cout << port;
```

2. C++ Mülakatlarının Yıldızı: extern "C"

Eğer başvurduğun pozisyon C++ ise, mülakatçı kesinlikle extern "C" yapısını soracaktır.

C++ nesne yönelimli bir dildir ve Function Overloading (Fonksiyon Aşırı Yükleme) destekler (Yani aynı isimde iki farklı fonksiyon yazabilirsin: yazdir(int) ve yazdir(double)).

Linker'ın bu iki fonksiyonu ayırt edebilmesi için C++ derleyicisi Name Mangling (İsim Bozma) denen bir işlem yapar. yazdir(int) fonksiyonunun makine kodundaki adını örneğin _Z6yazdiri yapar.

Ancak C dili overloading desteklemez, bu yüzden isim bozma da yapmaz. C dilinde hesapla() fonksiyonu makine kodunda da hesapla() olarak kalır.

Mülakat Tuzağı:

"C++ projemin içine, yıllar önce C ile yazılmış bir kütüphaneyi dahil ettim ve oradaki hesapla() fonksiyonunu çağırdım. Derleyici hata vermedi ama Linker 'Undefined Reference' hatası verdi. Neden?"

Cevap:

Çünkü sen C++ kodundan hesapla() çağırdığında, C++ derleyicisi onun ismini bozup Linker'a "Bana _Z7hesaplav fonksiyonunu bul" dedi. Linker, C kütüphanesine baktı ama orada sadece orijinal hesapla var, bulamadı ve çöktü.

Bunu çözmek için C++ kodumuzda o fonksiyonu extern "C" bloğu içine alırız:

```C++
extern "C" {
    void hesapla(); // Derleyiciye talimat: "Bu bir C fonksiyonu, ismini (Name Mangling) BOZMA!"
}
```

Bu sayede C++ derleyicisi fonksiyon ismine dokunmaz ve Linker C kütüphanesindeki doğru adresi bulur.

Compile-Time vs Link-Time Hataları

› *Header Guards (#pragma once) Nedir?*

› *C++'ta Static vs Extern Farkı*

C++'ta virtual Kullanımı

> **SORU:** *virtual kullanıyor musun? hangi senaryolarda neden?*

Bir yapay zeka asistanı olarak ben kod yazmıyorum (benim kendi bir vtable'ım veya nesne yönelimli bir mimarim yok), ancak C++ (ve C# gibi OOP dillerinde) kod ürettiğimde veya mimari tasarladığımda virtual anahtar kelimesini kesinlikle ve stratejik olarak kullanıyorum.

Bir C++ geliştiricisi gibi düşünecek olursak; virtual kullanımının "zorunlu" olduğu, "opsiyonel" olduğu ve "yasak" olduğu senaryolar vardır.

İşte benim virtual kelimesini kullandığım kritik senaryolar ve nedenleri:

1. Polimorfizm İstediğim Her Yerde (Çalışma Zamanı Esnekliği)

Senaryo: Birbiriyle akraba (kalıtım almış) farklı nesneleri tek bir listede (array veya vector) toplayıp, hepsine aynı komutu vermek istediğimde.

Neden?

Bir oyun motoru yazdığımızı düşünelim. Ekranda Dusman, Oyuncu, ve Mermi var. Hepsi ortak bir OyunNesnesi sınıfından (Base Class) türüyor.

Oyun döngüsü (Game Loop) içinde, hepsini tek tek sınıfına göre ayırıp oyuncu.ciz(), dusman.ciz() yazmak felakettir (Spaghetti kod).

Bunun yerine hepsini std::vector<OyunNesnesi*> içinde toplarım. Base sınıftaki ciz() fonksiyonunu virtual yaparım.

Artık sadece şunu yazmam yeterlidir:

```C++
for (auto* nesne : nesneListesi) {
    nesne->ciz(); // Virtual sayesinde mermi kendi çizimini, düşman kendi çizimini yapar.
}
```

virtual olmasaydı: Derleyici (Compiler) bu listedeki her şeyi saf OyunNesnesi sanacağı için (Statik Bağlama), ekranda mermi veya düşman değil, sadece boş base class nesneleri çizilirdi.

2. Yıkıcılarda (Destructors) - (Zorunlu Kullanım)

Senaryo: Bir sınıfı Base (Temel/Ebeveyn) sınıf olarak tasarlıyorsam ve o sınıftan miras (kalıtım) alınacaksa.

Neden?

Mülakatların en klasik sorusudur. Eğer ebeveyn sınıfın yıkıcısı (Destructor) virtual değilse ve biz onu bir pointer üzerinden silmeye kalkarsak Memory Leak (Bellek Sızıntısı) yaşanır.

```C++
class Base {
public:
    // Eğer buraya virtual YAZMAZSAM büyük hata olur!
    virtual ~Base() { cout << "Base silindi"; }
};

class Derived : public Base {
    int* array;
public:
    Derived() { array = new int[1000]; }
    ~Derived() { delete[] array; cout << "Derived silindi"; }
};

int main() {
```

```
    Base* ptr = new Derived();
    delete ptr;
}
```

Eğer Base sınıfın yıkıcısına virtual yazmazsam, delete ptr dediğimde derleyici sadece Base kısmı siler. Derived sınıfın yıkıcısı asla çağrılmaz ve o 1000 elemanlık dizi bellekte sonsuza kadar sızar. virtual yazdığımda ise önce Derived silinir, sonra Base silinir (tam temizlik).

3. Arayüz (Interface / Pure Virtual) Oluştururken

Senaryo: "Bu sınıftan doğrudan nesne üretilemesin, bu sadece alt sınıflar için bir şablon (kural seti) olsun" dediğimde.

Neden?

C++'ta Java'daki gibi interface anahtar kelimesi yoktur. Bir sınıfı soyut (Abstract) yapmak için Pure Virtual (Saf Sanal) fonksiyonlar kullanırız.

```
C++
class IVeritabani {
public:
    // Fonksiyonun sonuna = 0 koymak, bunu "Pure Virtual" yapar.
    virtual void baglan() = 0;
};
```

Bunu yaptığımda:

- Hiç kimse yanlışlıkla IVeritabani db; diyerek nesne üretemez (Derleyici hata verir).
- Bu sınıftan miras alan MySQL veya Oracle gibi sınıflar, baglan() metodunu yazmaya (override etmeye) mecbur kalır. Böylece mimaride bir sözleşme (contract) oluşturmuş olurum.

Ne Zaman KULLANMAM? (Maliyet ve Tasarım Sınırları)

Mülakatlarda sadece nerede kullanacağını bilmek yetmez, nerede kullanmayacağını bilmek de önemlidir.

- **Performansın Çok Kritik Olduğu Yerler:** Bir fonksiyonu virtual yaptığınızda, çağrılması normal bir fonksiyona göre bir tık daha yavaştır. Çünkü çalışma zamanında (Run-time) önce gidip vtable (Virtual Table) adlı bir tabloya bakması, oradan fonksiyonun gerçek adresini bulup sıçraması gerekir (Indirection). Çok sığ bir döngüde saniyede milyonlarca kez çağrılacak bir metotsa (örneğin donanım seviyesi gömülü sistemler) virtual kullanmaktan kaçınırım.
- **Veri Sınıfları (POD - Plain Old Data):** Sadece 3 Boyutlu bir X, Y, Z vektörünü tutan küçük bir sınıf yazıyorsam virtual kullanmam. Çünkü bir sınıfa bir tane bile virtual fonksiyon eklerseniz, derleyici o sınıfa gizli bir vptr (Virtual Pointer) ekler. Bu da 64-bit bir sistemde o sınıfın bellekteki boyutunun aniden 8 byte büyümesine sebep olur.

vtable ve vptr Tam Olarak Nasıl Çalışır?

› C++'ta Diamond Problem (Elmas Problemi)

› Virtual Constructor Mümkün Mü?

Egzersiz: Çıktı Ne Olur, Neden? (virtual & Object Slicing)

> **SORU:** *S3.2 Egzersiz — çıktı ne olur, neden?*

```C++
struct Base {
    void f() { std::cout << "Base::f\n"; }
    virtual void g() { std::cout << "Base::g\n"; }
};

struct Derived : Base {
    void f() { std::cout << "Derived::f\n"; }
    void g() override { std::cout << "Derived::g\n"; }
};

Derived d;
Base* pb = &d;
Base& rb = d;
Base bv = d;

pb->f(); pb->g();
rb.f(); rb.g();
bv.f(); bv.g();
```

1. Çıktı (Output)

Kod çalıştırıldığında ekrana tam olarak şu çıktı yazdırılır:

```Çıktı
Base::f
Derived::g
Base::f
Derived::g
Base::f
Base::g
```

2. Neden? (Arka Plan Analizi)

Bu kod parçası C++ mülakatlarının en klasik üç kavramını aynı anda test eder: Statik Bağlama (Static Binding), Dinamik Bağlama (Dynamic/Late Binding) ve Nesne Kırpılması (Object Slicing).

Süreci satır satır inceleyelim:

A. Pointer (İşaretçi) Kullanımı

```C++
pb->f(); // Çıktı: Base::f
pb->g(); // Çıktı: Derived::g
```

f() fonksiyonu virtual (sanal) değildir. Bu yüzden derleyici, kod derlenirken (compile-time) karar verir (Static Binding). İşaretçinin tipi Base* olduğu için, işaret ettiği nesne ne olursa olsun derleyici doğrudan Base::f fonksiyonunu çağırır.

g() fonksiyonu virtual tanımlanmıştır. Bu yüzden karar çalışma zamanında (run-time) verilir (Dynamic Binding). İşaretçinin tipi Base* olsa da, gösterdiği gerçek nesne bir Derived nesnesidir. Bu nedenle polimorfizm devreye girer ve Derived::g çağrılır.

B. Referans Kullanımı

```C++
rb.f(); // Çıktı: Base::f
```

```
rb.g(); // Çıktı: Derived::g
```

Referanslar (&), arka planda işaretçilerle (*) aynı polimorfik kurallara tabidir.

- Yine f() sanal olmadığı için referansın statik tipine (Base&) bakılarak Base::f çağrılır.
- g() sanal olduğu için referansın bağlandığı gerçek nesneye (d) bakılır ve Derived::g çağrılır.

C. Değer Ataması ve Nesne Kırpılması (Object Slicing) - En Önemli Kısım

```
C++
bv.f(); // Çıktı: Base::f
bv.g(); // Çıktı: Base::g
```

Buradaki kritik olay Base bv = d; satırıdır.

- bv bir işaretçi (*) veya referans (&) değildir. Bellekte yeni oluşturulan saf bir Base nesnesidir.

Siz büyük bir sınıfı (Derived), küçük bir sınıfın (Base) içine kopyalamaya çalıştığınızda Object Slicing (Nesne Kırpılması) yaşanır. C++ derleyicisi, d nesnesinin sadece Base sınıfına ait olan temel parçalarını kopyalar, Derived sınıfına ait olan tüm ekstra özellikleri kesip çöpe atar.

Artık bv %100 saf bir Base nesnesidir. İçinde Derived sınıfına dair hiçbir iz (veya ona ait bir vtable işaretçisi) kalmamıştır.

Bu yüzden g() sanal bir fonksiyon olsa bile, ortada bir Derived nesnesi kalmadığı için kendi fonksiyonu olan Base::g'yi çağırır.

vtable (Virtual Table) Nasıl Çalışır?

› *Sanal Yıkıcı (Virtual Destructor)*

› *Override ve Final anahtar kelimeleri*

C++ Kodunun Executable Olma ve Çalıştırılma Süreci

> **SORU:** *C++ ile yazılmış bir kod executable olup ve sonrasında çalıştırılana kadarki geçen bütün işlemleri anlat*

1. Mülakat Cevabı

> *"C++ kodunun çalıştırılabilir (executable) hale gelme süreci, temelde C ile aynı 4 aşamadan (Preprocessing, Compilation, Assembly, Linking) oluşur. Ancak C++'ın nesne yönelimli (OOP) ve jenerik programlama yetenekleri, bu sürece üç kritik farklılık katar:*
>
> *Derleme (Compilation) Aşamasında C++ Farkı: Derleyici, fonksiyon aşırı yüklemelerini (overloading) ve sınıfları desteklemek için Name Mangling (İsim Bozma/Dekorasyonu) işlemi yapar. Ayrıca Template (Şablon) kodları kullanıldıkları tipe göre bu aşamada fiziksel koda dönüştürülür (Instantiation) ve polimorfizm için Virtual Table'lar (vtable) oluşturulur.*
>
> *Bağlama (Linking) Aşamasında C++ Farkı: Bağlayıcı, bozulan (mangled) isimleri çözer, C++ standart kütüphanesini (libstdc++ veya libc++) projeye dahil eder ve One Definition Rule (ODR) (Tek Tanım Kuralı) ihlallerini denetler.*
>
> *Çalıştırma (Execution) Aşamasında C++ Farkı: Program belleğe yüklendiğinde işletim sistemi kontrolü main fonksiyonuna bırakmadan önce, C++ Runtime'ı devreye girer. Global ve statik sınıf nesnelerinin Constructor'ları (Yapıcıları) main'den önce çalıştırılır. Program biterken de main'den sonra Destructor'lar (Yıkıcılar) tetiklenir."*

2. Detaylı Arka Plan

Mülakatçı sen C sürecini bildiğin için bu soruyu sorduğunda, aslında C ile C++ arasındaki mimari farkları test ediyordur. Mülakatlarda bu farklar genellikle şu üç ana başlık üzerinden tuzak sorularla gelir:

1. Name Mangling (İsim Bozma) ve C Entegrasyonu

C dilinde aynı isimde iki fonksiyon yazamazsın. C++'ta ise Overloading (Aşırı Yükleme) sayesinde yazabilirsin. Peki Linker (Bağlayıcı) bu ikisini nasıl ayırt eder?

```C++
void yazdir(int x);
void yazdir(double x);
```

Mülakat Detayı: C++ derleyicisi bu kodu Assembly'ye çevirirken fonksiyon isimlerini değiştirir (Name Mangling). yazdir(int) fonksiyonu Linker'ın gözünde _Z6yazdiri (örneğin) gibi bir isme, yazdir(double) ise _Z6yazdird gibi bir isme dönüşür.

Tuzak Soru: "Bir C++ projesinde, C ile yazılmış eski bir kütüphanedeki hesapla() fonksiyonunu çağırırsan Linker neden hata verir?"

Cevap: C++ derleyicisi hesapla ismini mangle edip (bozup) Linker'a gönderir. Ama C kütüphanesi derlenirken ismi bozulmadan kalmıştır. Bu sorunu çözmek için C++'ta extern "C" bloğu kullanılır; bu, derleyiciye "Bu fonksiyonun ismini bozma, o bir C fonksiyonu" talimatı verir.

2. Template'ler ve Bağlayıcı (Linker) Hataları

C++ mülakatlarının en eleyici noktalarından biri şablonlardır (Templates).

Nasıl Çalışır: Template'ler kendi başlarına derlenmiş bir kod üretmezler, onlar derleyici için birer reçetedir. Sen bir <int> listesi istediğinde, derleme aşamasında derleyici senin için o an arka planda sadece int ile çalışan yeni bir sınıf kodu (instantiation) yazar.

Tuzak Soru: "Neden Template implementasyonlarını (gövdelerini) bir .cpp dosyasına değil de, .h (header) dosyasının içine yazarız?"

Cevap: Eğer implementasyonu .cpp dosyasına koyarsan, o dosya ayrı olarak derlendiğinde derleyici henüz hangi tiplerin (int, string vs.) kullanılacağını bilmediği için o template'i derlemez ve boş geçer. Başka bir dosyada o template'i kullanmaya kalktığında, Linker (Bağlayıcı) aradığında fonksiyonun makine kodunu hiçbir .o (object) dosyasında bulamaz ve "Undefined reference" hatası verir.

3. Çalıştırma Aşaması (Execution): C++'ta Hayat main'den Önce Başlar

C mülakatında "Program main'de başlar" diyebilirsin ama C++ mülakatında bu çok büyük bir eksiktir.

```
C++
class Veritabani {
public:
    Veritabani() { cout << "Veritabanına bağlanıldı!"; }
    ~Veritabani() { cout << "Bağlantı koptu!"; }
};

Veritabani globalDb; // Global nesne

int main() {
    cout << "Program başladı!";
    return 0;
}
```

Arka Planda Ne Olur? (OS Loader ve C++ Runtime)

- OS programı belleğe (Virtual Memory) yükler.
- _start fonksiyonu çağrılır. Bu, C ve C++ Runtime ortamını hazırlar.
- Kritik Adım: C++ Runtime, BSS ve Data segmentlerindeki tüm global nesnelerin ve static nesnelerin belleğini ayarladıktan sonra, onların Constructor'larını (Yapıcı Metotlarını) çalıştırır. Yani sen ekranda main'in içindeki "Program başladı!" yazısından önce, "Veritabanına bağlanıldı!" yazısını görürsün.
- Daha sonra main() çağrılır.
- main() return ile bittiğinde (veya exit() çağrıldığında), C++ Runtime tekrar kontrolü ele alır ve bu global nesnelerin Destructor'larını (Yıkıcı Metotlarını) çalıştırır. (Ekranda "Bağlantı koptu!" yazar).
- İşletim sistemi süreci (process) temizler.

Veri Yapıları

Array, Dinamik Array ve Linked List Karşılaştırması

> **SORU:** *Array, dinamik array ve linkli liste yapılarını kıyasla. Kullanım alanlarından bahset*

Bu üç temel veri yapısı, verileri bellekte (RAM) nasıl tuttuklarına ve üzerlerinde yapılan işlemlere (ekleme, silme, okuma) verdikleri tepkilere (zaman karmaşıklığına) göre birbirinden ayrılır.

[Görsel] Bellek Mimarisi: Bitişik Bloklar vs Dağınık Düğümler, yapay zekayla üretilmiş

› *Bellek Mimarisi: Bitişik Bloklar vs Dağınık Düğümler. Kaynak: Medium*

Görselde de görebileceğiniz gibi; Array'ler verileri bellekte sıralı ve yan yana kutular halinde tutarken, Linked List'ler (Bağlı Listeler) belleğin farklı yerlerine dağılmış verileri oklar (pointer/işaretçi) yardımıyla birbirine bağlar. Bu mimari fark, kullanım alanlarını doğrudan belirler.

Detaylı Kıyaslama

Özellik	Statik Array	Dinamik Array	Linked List (Bağlı Liste)
Boyut (Size)	Sabit (Baştan belirlenir)	Dinamik (Doldukça otomatik büyür)	Dinamik (Düğüm eklendikçe büyür)
Bellek Düzeni	Bitişik (Contiguous)	Bitişik (Contiguous)	Dağınık (Non-contiguous)
İndeksle Erişim	O(1) (Çok hızlı)	O(1) (Çok hızlı)	O(N) (Yavaş, baştan sayılır)
Sona Ekleme	O(1) (Eğer yer varsa)	O(1) (Amortized - Genelde hızlı)	O(1) (Kuyruk pointer'ı varsa)
Başa/Ortaya Ekleme	O(N) (Diğerleri kaydırılır)	O(N) (Diğerleri kaydırılır)	O(1) (Sadece oklar değişir)
Ekstra Bellek Yükü	Yok	Boş kalan kapasite kadar israf	Her veri için ekstra Pointer boyutu

1. Array (Statik Dizi)

- Nasıl Çalışır? Bellekte yan yana dizilmiş aynı tip elemanlar topluluğudur. Kod yazarken boyutunu (örneğin 100 eleman) baştan belirtmek zorundasınızdır ve sonradan genişletilemez.
- **Avantajı:** Veriler yan yana olduğu için işlemci önbelleği (CPU Cache) dostudur ve rastgele erişim (örneğin doğrudan 54. elemanı okumak) anında gerçekleşir.
- **Dezavantajı:** Boyutu sabittir. Başına veya ortasına yeni bir eleman eklemek isterseniz, sağında kalan tüm elemanları bellekte birer adım sağa kaydırmak zorundasınızdır ki bu çok yavaştır.
- **Kullanım Alanları:** Maksimum eleman sayısının kesin olarak bilindiği ve sürekli arama/okuma yapılan senaryolar. (Örn: Yılın aylarını tutmak, Satranç tahtası kurgulamak, Piksel dizileri ile görüntü işlemek).

2. Dinamik Array (Vector, ArrayList, List)

- Nasıl Çalışır? Modern dillerde en çok kullandığımız (Python'daki list, Java'daki ArrayList, C++'taki std::vector) yapıdır. Arka planda aslında bildiğimiz statik array'i kullanır. Ancak dizi tamamen dolduğunda arka planda otomatik olarak kapasitesi daha büyük (genelde 2 katı) yepyeni bir statik dizi oluşturur, eski verileri oraya kopyalar ve eskisini siler.
- **Avantajı:** Array'in müthiş okuma hızını (O(1)) korurken, boyut kısıtlamasından da kurtarır.

- **Dezavantajı:** Dizi her tam dolduğunda gerçekleşen "yeni diziye taşıma" işlemi o anlık performansı düşürür. Ayrıca her zaman fazladan kapasite ayırdığı için bir miktar bellek israfı (memory overhead) yapar.
- **Kullanım Alanları:** Veri boyutunun net bilinemediği ama rastgele okumanın (indekslemenin) hala sıkça kullanıldığı her türlü modern uygulama. (Örn: Veritabanı sorgu sonuçlarını ön bellekte tutmak, bir oyun içindeki dinamik oyuncu listesi).

3. Linked List (Bağlı Liste)

- Nasıl Çalışır? Veriler bellekte ardışık durmaz. Her bir eleman (Node/Düğüm), hem kendi verisini hem de bir sonraki elemanın bellek adresini (Pointer) bir arada tutar. Bir trenin vagonları gibi birbirlerine kanca ile bağlıdırlar.
- **Avantajı:** Araya veya başa eleman eklemek/çıkarmak inanılmaz hızlıdır (O(1)). Diğer verileri sağa sola kaydırmanıza gerek kalmaz, sadece aradaki kancaları (okları) söküp yeni vagona bağlarsınız. Ayrıca başlangıçta kapasite belirtmenize gerek yoktur, tam ihtiyacınız kadar bellek kullanırsınız.
- **Dezavantajı:** İndeksleme yoktur. 54. elemana ulaşmak için en baştaki vagondan başlayıp tek tek 54 kere okları takip etmek zorundasınızdır (O(N)). Ayrıca her veri bir sonraki adresi tutmak zorunda olduğu için ekstra hafıza kullanır.
- **Kullanım Alanları:** Veri sayısının öngörülemediği ve listenin başına/sonuna sürekli ekleme-çıkarma yapılan durumlar. (Örn: Tarayıcılardaki İleri/Geri buton geçmişi, İşletim sistemlerindeki görev kuyrukları, Müzik çalma listeleri, Hash Table çatışma (collision) çözümleri).

map vs unordered_map (Red-Black Tree vs Hash Map)

> **SORU:** *map vs unordered_map (RB tree vs hash map)*

1. Mülakat Cevabı

> *"C++ mülakatlarında veri yapıları kısmının en banko sorularından biridir. Seçim yaparken temel kural şudur: Sıralama (Ordering) mi istiyorsun, yoksa Mutlak Hız (Speed) mi?*
>
> *std::map (Arka plan: Red-Black Tree): Verileri anahtarlarına (key) göre her zaman sıralı (sorted) tutar. Ekleme, silme ve arama işlemleri ağaç üzerinde yapıldığı için zaman karmaşıklığı O(log N)'dir. Verileri küçükten büyüğe gezmek veya belirli bir aralıktaki (range) verileri çekmek istiyorsak bunu kullanırız.*
>
> *std::unordered_map (Arka plan: Hash Table): Verileri rastgele (sırasız) tutar. Bir Hash fonksiyonu kullanarak anahtarı (key) bir bellek indeksine çevirir. Bu sayede ekleme, silme ve arama işlemleri ortalamada O(1) (mükemmel hız) ile çalışır. Ancak sıralama garantisi yoktur ve Hash çakışmaları (collision) yaşanırsa en kötü senaryoda (worst-case) hızı O(N)'e kadar düşebilir.*
>
> *Endüstri standardı olarak; eğer verilerin sıralı durmasına özel bir ihtiyacımız yoksa, varsayılan (default) olarak her zaman unordered_map kullanırız çünkü çok daha hızlıdır."*

2. Detaylı Arka Plan ve Mülakat Tuzakları

Mülakatçı bu temel farkı duyduktan sonra işin mimari tarafına (Red-Black Tree ve Hash Map'in iç yapısına) ve C++'a özel kullanım detaylarına inecektir.

Tuzak Soru 1: Özel Sınıfları (Custom Objects) Key Olarak Kullanmak

C++ mülakatlarının en eleyici sorusudur: "Kendi yazdığım bir Kullanici struct'ını map veya unordered_map içinde Key (Anahtar) olarak kullanmak istersem ne yapmam gerekir?"

std::map İçin:

Sadece küçüktür operator< metodunu aşırı yüklemeniz (overload) veya bir karşılaştırıcı (comparator) yazmanız yeterlidir. Çünkü Red-Black Tree'nin tek bilmesi gereken şey, yeni gelen nesnenin ağacın sağına mı yoksa soluna mı (büyük mü küçük mü) gideceğidir.

std::unordered_map İçin:

İşler burada zorlaşır. Sadece büyüklük-küçüklük yetmez. İki şeye ihtiyacınız vardır:

- **operator== (Eşitlik Operatörü):** İki nesnenin aynı olup olmadığını anlamak için (Hash çakışmalarında kontrol etmek için).
- **Custom Hash Function (Özel Hash Fonksiyonu):** C++, sizin kendi objenizi bellekte nasıl bir sayıya (hash) çevireceğini bilemez. Sizin std::hash şablonunu (template) kendi sınıfınız için özelleştirmeniz (specialize) gerekir. (Bu detayı mülakatta söylemek senior seviyesinde bir bilgidir).

Tuzak Soru 2: Hash Çakışması (Collision) ve O(N) Çöküşü

Mülakatçı sorar: "unordered_map O(1) ile çalışıyor diyorsun ama her zaman böyle midir?"

Kritik Cevap: "Hayır, O(1) sadece ortalama (average) durumdur. Eğer kullandığımız Hash Fonksiyonu kötü tasarlanmışsa ve farklı anahtarlar (keys) sürekli aynı indeksi (bucket) üretirse, buna Hash Collision (Çakışma) denir. C++ unordered_map, çakışmaları genellikle 'Separate Chaining' (Bağlı Liste) ile çözer. Aynı indekse denk gelen elemanlar bir Linked List oluşturur. Tüm elemanlar aynı indekse yığılırsa, aradığımızı bulmak için o Linked List'i baştan sona gezmemiz gerekir ki bu da bizi en kötü senaryoda (worst-case) O(N) zaman karmaşıklığına mahkum eder. std::map (O(log N)) bu yüzden daha istikrarlı bir performans (guaranteed bounds) sunar."

Mimari Kıyaslama Tablosu

Özellik	std::map	std::unordered_map
Arka Plan Yapısı	Red-Black Tree (Dengeli İkili Arama Ağacı)	Hash Table (Karma Tablosu)
Zaman Karmaşıklığı (Arama, Ekleme)	O(log N) (Garantili / Stabil)	O(1) Ortalama, O(N) En Kötü
Veri Sıralaması	Anahtara göre daima sıralı tutulur.	Karışık/Rastgele tutulur.
Gereksinim	Key için operator< (Küçüktür) gerekir.	Key için Hash Function ve operator== gerekir.
Bellek Tüketimi (Overhead)	Her düğüm (Node) sağ/sol okları (pointer) ve renk bilgisi tuttuğu için yüksektir.	Array (Bucket) ve Link (Pointer) tuttuğu için yüksektir ama ağaca göre bir tık daha iyidir.
İterasyon (Döngüye Sokma)	Sıralı iterasyon yapmak yavaştır (Ağaçta in-order traversal yapar).	Bellekte array/bucket üzerinde gezdiği için cache-friendly ve hızlıdır.

Ne Zaman Hangisini Seçmeliyiz?

- **std:**:unordered_map Kullan: Oyun motorlarında obje ID'lerini eşleştirirken, web sunucularında kullanıcı oturum (session) bilgilerini tutarken veya bir metindeki kelimelerin frekansını sayarken (mutlak hız gereken her yer).
- **std:**:map Kullan: Verileri ekranda alfabetik veya tarih sırasına göre listelemen gerektiğinde. Veya "Fiyatı 50 TL ile 100 TL arasındaki ürünleri getir" (Range query - lower_bound, upper_bound) gibi operasyonlara ihtiyacın olduğunda.

Red-Black Tree vs AVL Tree

› Hash Collision Çözüm Yöntemleri

› Big O: O(1) vs O(log n)

Heap Veri Yapısı

> **SORU:** *heap veri yapısı hakkında ne biliyorsun, kullanım alanları nelerdir*

1. Mülakat Cevabı

> *"Heap (Yığın), bellekte arka planda bir Array (Dizi) olarak tutulan, ancak mantıksal olarak Complete Binary Tree (Eksiksiz İkili Ağaç) yapısında çalışan özel bir veri yapısıdır.*
>
> *Temel amacı bir veri setindeki en büyük (Max-Heap) veya en küçük (Min-Heap) elemana anında (O(1) zaman karmaşıklığında) erişmektir. Yeni bir eleman eklendiğinde veya kökteki eleman silindiğinde, ağaç kendi kendini yukarı veya aşağı doğru yeniden düzenler (Heapify işlemi) ve bu düzenleme her zaman O(logN) hızında gerçekleşir.*
>
> *En yaygın kullanım alanı Priority Queue (Öncelikli Kuyruk) sistemleridir. İşletim sistemlerindeki görev zamanlamasından tutun da, Dijkstra gibi en kısa yol algoritmalarına kadar, 'sıradaki en önemli/en öncelikli veriyi hızlıca ver' dediğimiz her yerde Heap kullanırız."*

2. Detaylı Arka Plan

Bir yazılım mülakatında "Heap" denildiğinde, senden sadece bir ağaç (tree) anlatman beklenmez; bu ağacın neden Array içinde tutulduğunu ve zaman karmaşıklıklarını mimari boyutta tartışman istenir.

Neden Ağaç (Tree) Görünümlü Bir Array (Dizi)?

Heap, şekil olarak bir ağaçtır ama Pointer'lar (sağ-sol okları) kullanarak bir Linked List (Bağlı Liste) mantığıyla tutulmaz. Düz bir Array (dizi) içine yerleştirilir.

Bunun sebebi "Complete (Eksiksiz)" olma kuralıdır. Heap'te yeni gelen bir veri her zaman en alt seviyede, soldan sağa doğru ilk boşluğa yerleşir. Ağaçta hiç boşluk (null node) bırakılmadığı için indeksleme ile matematiksel olarak parent-child (ebeveyn-çocuk) ilişkisi kurulabilir:

Eğer bir eleman Array'in i indeksindeyse:

- Sol çocuğu: 2i+1 indeksindedir.
- Sağ çocuğu: 2i+2 indeksindedir.
- Ebeveyni (Parent): (i−1)/2 indeksindedir (tam sayı bölmesi).

Avantajı: Pointer'lar (oklar) için ekstra bellek (RAM) harcanmaz ve veriler bellekte bitişik (contiguous) durduğu için işlemci önbelleği (CPU Cache) müthiş verimli çalışır.

Heap Türleri ve Heapify İşlemi

- **Max-Heap:** Her ebeveyn düğümün değeri, kendi çocuklarından büyük veya eşit olmak zorundadır. Bu yüzden en büyük eleman her zaman en tepededir (Kök/Root).
- **Min-Heap:** Her ebeveyn düğümün değeri, kendi çocuklarından küçük veya eşit olmak zorundadır. En küçük eleman her zaman en tepededir.

Yeni Eleman Nasıl Eklenir? (Heapify-Up)

Diyelim ki bir Max-Heap'e yeni bir değer ekliyorsun.

- Değeri dizinin en sonuna (ağacın en alt, en sağındaki boşluğa) koyarsın.
- Değer kendi ebeveyni ile kıyaslanır. Eğer ebeveyninden büyükse, ikisi yer değiştirir (Swap).
- Bu işlem, eleman doğru yerine yerleşene kadar (ya da köke ulaşana kadar) yukarı doğru tekrarlanır. (Buna Bubble-up veya Heapify-up denir).

Önemli Not: Extract (Çıkarma) işlemi sadece en tepedeki kök (Root) için yapılır. Heap'in ortasından rastgele bir eleman aramak veya silmek O(N) sürer, çünkü Heap bir Binary Search Tree (İkili Arama Ağacı) değildir. Sadece kök düğümü garanti altına alır.

Temel Kullanım Alanları (Mülakat Senaryoları)

- **Priority Queue (Öncelikli Kuyruklar):** Normal kuyruklarda (Queue) "ilk giren ilk çıkar (FIFO)". Öncelikli kuyruklarda ise "önceliği en yüksek olan ilk çıkar". İşletim sistemlerinde CPU'nun hangi sürece (process) öncelik vereceğini seçmesi arka planda Max-Heap ile yönetilir.
- **Dijkstra'nın En Kısa Yol Algoritması:** Harita/Navigasyon uygulamalarında bir noktadan diğerine giderken sürekli "Şu ana kadarki en kısa/en ucuz yol hangisi?" diye sormamız gerekir. Min-Heap bu soruyu O(1)'de cevaplayarak algoritmayı devasa ölçüde hızlandırır.
- **Top-K Problemleri (Mülakatların Gözdesi):** "Sana 1 Milyar tane sayı verdik, bana en büyük 5 sayıyı bul" gibi bir soruda diziyi komple sıralamak (Sort) intihardır. Bunun yerine boyutu sadece 5 olan bir Min-Heap oluşturursun. Sayılar geldikçe Heap'e atarsın, Heap dolduğunda yeni gelen sayı köktekinden (en küçükten) büyükse, kökü silip yeni sayıyı eklersin. İşlem bittiğinde Heap'in içinde en büyük 5 sayı kalmış olur.
- **Heap Sort Algoritması:** Bütün elemanları bir Heap'e atıp, sonra tek tek kökten çekerek diziyi sıralama mantığıdır. Zaman karmaşıklığı O(NlogN)'dir ve ekstra bellek kullanmadığı (In-place) için değerlidir.

Binary Search Tree vs Heap

› Top-K Problemi Kod Mantığı

› Dijkstra Algoritması Nasıl Çalışır?

Cache ve Bellek Düzeni Karmaşıklık Analizi

> **SORU:** *Cache ve bellek düzeni karmaşıklık analizi nasıl yapılır, bir örnekle anlat*

1. Mülakat Cevabı

> *"Geleneksel algoritma analizinde (Big O Notasyonu) sadece işlem sayısına odaklanılır ve tüm bellek okumalarının aynı maliyette (O(1)) olduğu varsayılır. Ancak modern donanımlarda Cache ve Bellek Düzeni (Memory Layout) analizi, verinin bellekte nasıl dizildiğine ve işlemci önbelleğine (CPU Cache) ne kadar verimli taşındığına bakar.*
>
> *Bunun analizini yaparken Cache Miss (Önbellek Iskalama) oranını hesaplarız. Veriler bellekte bitişik (contiguous) bir şekilde dizilmişse (örneğin bir Array), CPU bir veriyi okuduğunda peşinden gelecek olanları da topluca Cache'e alır (Cache Line). Buna Spatial Locality (Mekansal Yakınlık) denir. Eğer veriler dağınıksa (örneğin Linked List) CPU sürekli ana belleğe (RAM) gitmek zorunda kalır ve ciddi bir performans düşüşü (Cache Thrashing) yaşanır.*
>
> *Kısacası, karmaşıklık analizi yaparken sadece adım sayısını değil, Memory Access Pattern (Bellek Erişim Dokusu)'nun doğrusal mı (predictable) yoksa rastgele mi (random) olduğunu analiz ederiz."*

2. Detaylı Arka Plan ve Analiz Örneği

Mülakatçı bu teorik cevabı aldıktan sonra, "Bunu bana somut bir kod örneği üzerinden, Cache Line mantığıyla açıkla" diyecektir. En iyi örnek Matris (2 Boyutlu Dizi) dolaşma problemidir.

Örnek: Matris Gezinme (Row-Major vs Column-Major)

Elimizde 1000×1000 boyutlarında bir 2 Boyutlu tam sayı (int) dizisi (Matris) olsun. Bu dizinin içindeki tüm sayıları toplamak istiyoruz. İki farklı döngü yazalım:

Algoritma A (Satır Satır Gezmek - Row-Major):

```C++
int toplam = 0;
for(int i = 0; i < 1000; i++) {       // Satır (Row)
    for(int j = 0; j < 1000; j++) {   // Sütun (Column)
        toplam += matris[i][j];       // Yan yana okuma
    }
}
```

Algoritma B (Sütun Sütun Gezmek - Column-Major):

```C++
int toplam = 0;
for(int j = 0; j < 1000; j++) {       // Sütun (Column)
    for(int i = 0; i < 1000; i++) {   // Satır (Row)
        toplam += matris[i][j];       // Atlaya atlaya okuma
    }
}
```

1. Geleneksel Big O Analizi (Teori)

Klasik algoritma analizine göre her iki algoritma da tam olarak 1000×1000=1.000.000 adet okuma ve toplama işlemi yapar. Yani ikisinin de zaman karmaşıklığı $O(N^2)$'dir. İkisi de "teoride" aynı hızda çalışmalıdır.

2. Cache (Önbellek) Karmaşıklık Analizi (Gerçeklik)

C ve C++ dillerinde 2 boyutlu bir matris, RAM'de fiziksel olarak kare şeklinde değil, tek bir uzun satır (Row-Major) şeklinde ardışık (yan yana) dizilir.

CPU, RAM'den veri okurken sadece istediğiniz 1 adet int'i (4 byte) almaz. Donanım seviyesinde veriler Cache Line adı verilen bloklar halinde taşınır (Genellikle 64 Byte = 16 adet int).

Algoritma A'nın Analizi (Mükemmel):

CPU matris[0][0] elemanını okumak için RAM'e gider (Bu bir Cache Miss'tir). Ancak dönerken [0][0] ile birlikte sonraki 15 elemanı da Cache'e kopyalar.

İçteki döngü devam ettiğinde [0][1], [0][2]... için RAM'e gidilmez, zaten Cache'te oldukları için nanosaniyeler içinde okunurlar (Cache Hit). Yani her 16 elemanda sadece 1 kez yavaş okuma yapılır. Bellek erişim dokusu doğrusaldır (Linear Access).

Algoritma B'nin Analizi (Felaket):

Döngü matris[0][0]'ı okur, CPU 16 elemanı Cache'e getirir. Ancak döngünün bir sonraki adımı yanındaki veriyi değil, bir alt satırdaki matris[1][0]'ı ister! O veri fiziksel olarak 1000 int (4000 byte) ileridedir ve Cache'te yoktur (Cache Miss). CPU tekrar RAM'e gider.

Bir sonraki adımda matris[2][0] istenir, yine yoktur, yine RAM'e gidilir. Algoritma B, her bir okuma için Cache'i çöpe atıp RAM'e gitmek zorunda kalır. Bellek erişim dokusu rastgeledir (Strided Access).

Sonuç: Pratikte Algoritma A, Algoritma B'den donanıma bağlı olarak 10 ila 30 kat daha hızlı çalışır. İşte Cache-Aware (Önbellek Farkındalıklı) analiz budur.

3. Data-Oriented Design (Veri Odaklı Tasarım)

Oyun motoru geliştirme (örneğin Unity DOTS veya Unreal Engine) ve yüksek frekanslı ticaret (HFT) mülakatlarında, bu analizin mimariye yansıması sorulur.

OOP (Nesne Yönelimli) Yaklaşım Neden Yavaştır?

Geleneksel OOP'de bir OyunKarakteri sınıfı yazarsın. İçinde karakterin Adı, Rengi, Sağlık Puanı, X-Y-Z Konumu gibi birçok farklı veri (struct of arrays) bulunur. Sen sadece 10.000 karakterin Konumunu güncelleyen bir döngü çalıştırdığında, CPU karakteri okurken ihtiyacın olmayan 'Adı' ve 'Rengi' gibi verileri de Cache'e doldurur. Cache gereksiz çöple dolar, asıl veriye yer kalmaz.

DOD (Veri Odaklı Tasarım) Çözümü:

Verileri nesne nesne değil, özellik özellik ayırırsın (Array of Structs).

Sistemde sadece KarakterKonumları[10000] diye bitişik bir dizi olur. Döngü sadece bu diziyi günceller. Bellekte araya hiçbir gereksiz çöp veri girmediği için Cache %100 verimle çalışır. Buna Data Locality (Veri Yöreselliği) denir.

Bilgisayar Ağları (Networking)

TCP ve UDP

> **SORU:** *Tcp ve udp nedir ? Farklarıyla beraber açıkla*

1. Mülakat Cevabı

> *"TCP (Transmission Control Protocol) ve UDP (User Datagram Protocol), ağ üzerindeki cihazların birbiriyle nasıl iletişim kuracağını belirleyen Taşıma Katmanı (Transport Layer) protokolleridir. Temel farkları, güvenilirlik ile hız arasındaki tercihtir.*
>
> *TCP, bağlantı odaklı (connection-oriented) ve güvenilir bir protokoldür. Veri aktarımına başlamadan önce cihazlar arasında sanal bir bağlantı kurar (3-Way Handshake). Giden her veri paketinin ulaştığını teyit eder (Acknowledgment), kaybolan paketleri tekrar gönderir ve paketleri doğru sırayla birleştirir. Ancak bu garanti mekanizmaları onu 'ağır' ve yavaş yapar. Web sayfaları, e-postalar ve dosya transferleri gibi veri kaybının kabul edilemeyeceği yerlerde kullanılır.*
>
> *UDP ise bağlantısız (connectionless) ve 'ateşle ve unut' mantığıyla çalışan bir protokoldür. Bağlantı kurmakla veya paketlerin ulaşıp ulaşmadığıyla ilgilenmez; veriyi sadece ağa fırlatır. Kaybolan paketler tekrar istenmez, paketlerin sırası garanti edilmez. Hata kontrolü yapmadığı için başlık (header) boyutu çok küçüktür ve TCP'ye göre inanılmaz derecede hızlıdır. Anlık video/ses görüşmeleri (Zoom, Skype), çevrimiçi rekabetçi oyunlar ve DNS sorguları gibi hızın ve düşük gecikmenin (latency) verinin doğruluğundan daha önemli olduğu yerlerde kullanılır."*

2. Detaylı Arka Plan

Bir Sistem Tasarımı (System Design) veya Network mülakatında sadece bu farkları sayman yetmez. Mülakatçı, "TCP bu güvenilirliği arka planda donanımsal/yazılımsal olarak nasıl sağlıyor?" veya "Oyun yaparken neden TCP kullanmak istemezsin?" gibi senaryo sorularıyla konuyu derinleştirecektir.

TCP'nin Güvenilirlik Mekanizmaları (Nasıl Çalışır?)

TCP'nin güvenilirliği "sihirli" değildir; arkasında katı algoritmalar yatar:

- **3-Way Handshake (3 Yönlü El Sıkışma):** İletişim başlamadan önce istemci ve sunucu anlaşıp bağlantı kurar.
- İstemci: "Seninle konuşmak istiyorum" (SYN paketi).
- Sunucu: "Tamam, konuşalım. Ben de hazırım" (SYN-ACK paketi).
- İstemci: "Harika, veriyi göndermeye başlıyorum" (ACK paketi).
- **Sequence Numbers (Sıra Numaraları):** İnternet, paketlerin farklı yollardan (farklı router'lar üzerinden) gidip farklı zamanlarda ulaşabildiği bir ağdır. TCP, her pakete bir sıra numarası verir. Paketler karşıya karışık sırayla (örneğin 1, 3, 2, 4) ulaşsa bile, alıcı bilgisayarın TCP katmanı bu sıra numaralarına bakarak dosyayı doğru şekilde birleştirir.
- Acknowledgments (ACK - Teyit) ve Retransmission (Tekrar Gönderim): TCP gönderdiği her paket için karşı taraftan bir ACK (aldım) mesajı bekler. Eğer belirli bir süre (Timeout) içinde ACK gelmezse, paketin yolda kaybolduğunu varsayar ve o paketi tekrar gönderir.

Neden Her Yerde TCP Kullanmıyoruz? (UDP'nin Gücü)

Mülakatın kırılma noktası burasıdır: "Madem TCP bu kadar güvenli, neden UDP diye bir şeye ihtiyacımız var?"

Cevap: "Eski veri, çöp veridir" prensibi.

Çevrimiçi bir FPS oyunu (örneğin CS:GO veya Valorant) oynadığını düşün. Sunucu sana düşmanın koordinatlarını saniyede 60 kez yolluyor.

Eğer TCP kullansaydın ve 14. saniyedeki paket yolda kaybolsaydı, TCP sistemi durdurur, kaybolan o paketi tekrar ister ve gelene kadar sonraki tüm güncel paketleri (15., 16. saniyeyi) bekletirdi (Buna Head-of-line blocking denir). Sonuç: Oyunun aniden donar, paket gelince her şey ışınlanır.

UDP kullanıldığında ise, 14. saniyedeki paket kaybolursa, oyun bunu umursamaz. "Zaten birazdan 15. saniyenin yeni paketi gelecek, eskisini ne yapayım?" der ve devam eder. Oyun anlık olarak milisaniyelik takılır ama akmaya devam eder. Hız ve akıcılık (düşük gecikme), 1-2 kayıp paketten daha değerlidir.

Karşılaştırma Tablosu

Özellik	TCP	UDP
Bağlantı Kurulumu	3-Way Handshake (Bağlantı odaklı)	Yok (Bağlantısız)
Paket Garantisi	Var (Ulaşana kadar tekrar dener)	Yok (Fire and forget)
Sıralama	Paketleri doğru sırayla birleştirir	Geldiği gibi uygulamaya verir
Akış/Tıkanıklık Kontrolü	Var (Ağ yoğunsa yavaşlar)	Yok (Ağ dursa bile aynı hızda gönderir)
Başlık (Header) Boyutu	Büyük (Minimum 20 byte)	Çok küçük (8 byte)
Kullanım Alanları	Web (HTTP), E-posta (SMTP), Dosya (FTP)	Video Call, Oyunlar, DNS, IoT sensörleri

Mülakat Tuzağı: "UDP güvenilir yapılabilir mi?"

Mülakatçı gülümseyerek sorar: "UDP'nin hızlı olmasını istiyorum ama veri kaybı da yaşamak istemiyorum. Ne yapmalıyım?"

Kritik Cevap: "Güvenilirliği Transport Layer (Taşıma Katmanı) yerine Application Layer (Uygulama Katmanı) seviyesinde kendim uygularım."

Bu günümüzde aktif olarak kullanılan bir mimaridir. Örneğin, Google tarafından geliştirilen ve bugün HTTP/3'ün temelini oluşturan QUIC protokolü, altyapıda hızlı olan UDP'yi kullanır, ancak TCP'nin hata düzeltme ve tekrar gönderim mekanizmalarını yazılım seviyesinde kendisi yönetir. Böylece TCP'nin hantal bağlantı kurma (Handshake) maliyetinden kurtulup güvenilir bir yapı sunar.

Yazılım Geliştirme Süreçleri (Agile & DevOps)

Agile (Çevik) Metodoloji

> **SORU:** *Bir mülakatta agile sistem ile ilgili ne sorular sorulabilir?*

Teknik mülakatlarda kodlama ve mimari bilgin test edildikten sonra (veya İK/Yönetici mülakatlarında), takım çalışmasına ne kadar yatkın olduğunu ve modern yazılım geliştirme döngülerini (SDLC) anlayıp anlamadığını ölçmek için Agile (Çevik) metodoloji soruları gelir.

Agile mülakatlarında mülakatçı senin sadece terimleri ezberleyip ezberlemediğine değil; esneklik, iletişim, tahmin yapabilme (estimation) ve kriz yönetimi becerilerine bakar.

İşte karşına çıkması en muhtemel 4 kritik Agile sorusu ve onlara vermen gereken ideal cevaplar:

Soru 1: Agile ile Geleneksel (Waterfall) yaklaşım arasındaki fark nedir? Neden Agile kullanıyoruz?

1. Mülakat Cevabı

> *"Temel fark esneklik ve teslimat (delivery) modelidir.*
>
> *Waterfall (Şelale) yönteminde her aşama (analiz, tasarım, kodlama, test) sırayla ve bir kere yapılır. Müşteri ürünü aylar veya yıllar süren projenin en sonunda görür. Geliştirme ortasında gelen bir değişiklik talebi tüm mimariyi bozar ve çok maliyetlidir.*
>
> *Agile (Çevik) ise iteratif (döngüsel) ve artımlıdır. Proje genellikle 2 haftalık 'Sprint' adı verilen küçük döngülere bölünür. Her döngünün sonunda müşteriye çalışan, test edilmiş küçük bir ürün parçası (MVP - Minimum Viable Product) sunulur. Böylece müşteri geri bildirimleri projenin erken aşamalarında alınır ve değişen gereksinimlere çok hızlı adapte olunur."*

2. Detaylı Arka Plan (Mülakatçı ne arıyor?)

Mülakatçı burada senin "Değişime kucak açma" felsefesini anlayıp anlamadığını test eder. Yazılım dünyasında (özellikle startup'larda) gereksinimler sürekli değişir. "Bana baştan her şeyi 100 sayfalık doküman olarak verin, ben 6 ay kodlayıp çıkayım" diyen bir mühendis, modern teknoloji şirketlerinde tercih edilmez.

Soru 2: Sprint'in ortasında Product Owner (Ürün Sahibi) gelip "Çok acil yeni bir özellik eklememiz lazım" derse ne yaparsın?

1. Mülakat Cevabı

> *"Doğrudan 'Evet' veya 'Hayır' demem. Agile'ın doğası gereği değişikliklere açığız ancak Sprint'in de bir taahhüdü (commitment) vardır.*
>
> *Öncelikle bu yeni özelliğin boyutunu (Story Point) takımca tahmin ederiz. Ardından Product Owner'a şunu söylerim: 'Bu özelliği Sprint'e alabiliriz ancak kapasitemiz (velocity) sabit olduğu için, şu an Sprint'te bulunan aynı büyüklükteki X veya Y görevlerinden birini bu Sprint'ten (Backlog'a) çıkarmak zorundayız. Hangisini çıkaralım?' Önceliklendirme kararı her zaman Product Owner'a aittir, geliştirme takımı ise kapasitesini korumaktan sorumludur."*

2. Detaylı Arka Plan (Tuzak Soru!)

Bu, en meşhur davranışsal (behavioral) tuzak sorudur.

- Eğer "Hemen yaparım, mesaiye kalırım" dersen elenirsin (Tükenmişlik/Burnout riski ve Sprint yapısına saygısızlık).
- Eğer "Asla alamayız, kurala aykırı" dersen yine elenirsin (Çevik değilsin, katısın).

Doğru cevap her zaman "Takas (Trade-off)" yapmaktır (Scope Creep'i yönetmek). Eğer baskı artarsa, takımın koruyucusu olan Scrum Master'ı devreye sokman gerektiğini belirtmek de ekstra puan getirir.

Soru 3: Daily Stand-up (Günlük) toplantılarının amacı nedir?

1. Mülakat Cevabı

> *"Daily toplantıları yöneticilere verilen bir 'Durum Raporu (Status Report)' toplantısı değildir. Geliştirme takımının kendi arasında senkronize olduğu, genellikle ayakta ve maksimum 15 dakika süren kısa planlama toplantılarıdır.*
>
> *Her üye üç temel soruya cevap verir:*
>
> *Dün Sprint hedefine ulaşmak için ne yaptım?*
>
> *Bugün ne yapacağım?*
>
> *Önümde beni engelleyen bir sorun (Blocker) var mı?*
>
> *Amacımız mikro-yönetim değil, iletişimi şeffaflaştırmak ve eğer bir arkadaşımız tıkandıysa ona hızlıca yardım etmektir."*

2. Detaylı Arka Plan

Mülakatçı senin takım içi iletişimini ölçer. Pek çok junior geliştirici, Daily toplantısında 10 dakika boyunca yazdığı kodun teknik detaylarını anlatmaya başlar. Mülakatçıya "Eğer teknik bir detay tartışılacaksa, Daily bittikten sonra sadece ilgili kişilerle ayrı bir 'Otopark (Parking Lot)' toplantısı yaparız, herkesin vaktini almayız" demen senin tecrübeli (senior) bir bakış açısına sahip olduğunu gösterir.

Soru 4: "Story Point" (Hikaye Puanı) nedir? Neden zaman (saat/gün) yerine Puan kullanıyoruz?

1. Mülakat Cevabı

> *"Story Point, bir işin zorluğunu, karmaşıklığını ve barındırdığı riski tahmin etmek için kullandığımız göreceli (relative) bir ölçü birimidir (genellikle Fibonacci dizisi 1, 2, 3, 5, 8 kullanılır).*
>
> *Saat veya gün kullanmamamızın temel sebebi, zamanın kişiden kişiye değişmesidir. Örneğin bir veritabanı taşıma işi, Senior bir mühendis için 2 saat sürerken, Junior bir mühendis için 2 gün sürebilir. Eğer zaman tahmini yaparsak işi kimin alacağına göre tahmin değişir. Ancak işin 'karmaşıklığı' (Story Point'i) kişiye göre değişmez, her ikisi için de örneğin 5 puandır. Bu sayede takımın genel kapasitesi (Velocity) çok daha tutarlı ölçülür."*

2. Detaylı Arka Plan

Planlama Pokeri (Planning Poker) mülakatlarda çok sorulur. Mülakatçı şu senaryoyu verebilir: "Sen bir işe 2 puan verdin, başka bir takım arkadaşın 8 puan verdi. Ne yaparsınız?"

Cevap: "Ortalamasını alıp 5 yazıp geçmeyiz. 2 veren kişiye 'Bu işi neden bu kadar basit gördün, bizim kaçırdığımız bir kütüphane mi biliyorsun?', 8 veren kişiye ise 'Neden bu kadar karmaşık gördün, bizim görmediğimiz bir risk mi var?' diye sorarak aramızda tartışırız ve ortak bir noktada (konsensüs) buluşana kadar yeniden oylarız."

Davranışsal (Behavioral) STAR Metodu
› *CI/CD ve DevOps Kültürü*
› *Tekniğe Dönüş: SOLID Prensipleri*

CI/CD (Continuous Integration / Continuous Deployment)

> **SORU:** *Agile takımların vazgeçilmezi olan CI/CD (Continuous Integration / Continuous Deployment) süreçleri nelerdir ve mülakatlarda nasıl sorulur?*

1. Mülakat Cevabı

> *"CI/CD (Continuous Integration / Continuous Deployment), yazılım geliştirme sürecinde kodun yazılmasından canlı ortama (production) alınmasına kadar geçen tüm test, derleme ve dağıtım adımlarının otomatikleştirilmesidir.*
>
> *CI (Sürekli Entegrasyon - Continuous Integration): Geliştiricilerin kendi bilgisayarlarında yazdıkları kodları günde birkaç kez ana koda (main/master branch) birleştirmesidir. Her commit (gönderim) sonrasında otomatik olarak bir derleme (build) işlemi başlar ve birim testleri (unit tests) çalışır. Amaç, 'Benim bilgisayarımda çalışıyordu' bahanesini ortadan kaldırmak ve yeni eklenen kodun mevcut sistemi bozup bozmadığını anında görmektir.*
>
> *CD (Sürekli Teslimat veya Sürekli Dağıtım): CI aşamasını (derleme ve testleri) başarıyla geçen kodun sunuculara yüklenmesidir. Eğer Continuous Delivery (Sürekli Teslimat) yapıyorsak, kod canlı ortama çıkmaya her an hazırdır ancak son bir yöneticinin veya ürün sahibinin manuel 'Onay' butonuna basması gerekir. Eğer Continuous Deployment (Sürekli Dağıtım) yapıyorsak, insan müdahalesi yoktur; testleri geçen kod anında ve otomatik olarak canlıdaki kullanıcılara ulaşır.*
>
> *Agile takımlar için bu süreç vazgeçilmezdir çünkü Agile'ın temel prensibi olan 'müşteriye sürekli ve sık çalışan yazılım teslim etme' hedefini, insan hatasını (human error) devreden çıkararak mümkün kılar."*

2. Detaylı Arka Plan

Bir yazılım mühendisi mülakatında CI/CD soruluyorsa, senden bir DevOps mühendisi olman beklenmez. Ancak kodunu yazıp Git'e yükledikten sonra (push) o kodun arka planda hangi fabrikadan (Pipeline) geçtiğini mimari olarak bilmen istenir.

Mülakatlarda karşına çıkacak en kritik 3 senaryo sorusu şunlardır:

Senaryo 1: Delivery ve Deployment Tuzak Sorusu

Mülakatçı sorar: "Projende Continuous Delivery mi yoksa Continuous Deployment mı tercih edersin? Neden?"

Kritik Cevap: "Bu tamamen projenin iş modeline (business logic) bağlıdır. Eğer Netflix veya Amazon gibi bir e-ticaret/medya sisteminde çalışıyorsam, günde binlerce kez canlıya kod atabilmek için Continuous Deployment kullanırım; insan müdahalesi hızı keser.

Ancak bir Bankacılık, Sağlık uygulaması veya Uzay/Havacılık yazılımı (örneğin NASA) geliştiriyorsam, regülasyonlar ve güvenlik sebebiyle Continuous Delivery kullanmak zorundayım. Testler %100 geçse bile, canlıya alma işlemini manuel bir onaya (Release Gate) bağlamak yasal bir zorunluluk olabilir."

Senaryo 2: "Build Patladı" (Broken Build) Sorusu

Mülakatçı sorar: "Cuma öğleden sonra saat 16:00. Kodunu commit ettin, CI pipeline'ı çalıştı ve Unit Testlerden biri hata verip Pipeline'ı kırmızıya (Failed) düşürdü. Ne yaparsın?"

Kritik Cevap: "Agile ve CI kültüründe bir numaralı kural şudur: Eğer Build kırmızıysa, tüm takımın önceliği onu tekrar yeşile döndürmektir. Asla o kodu o halde bırakıp yeni bir özelliğe (feature) geçmem veya hafta sonu tatiline çıkmam. Hata benim kodumdan kaynaklanıyorsa, hemen hatayı çözen yeni bir yama (hotfix/commit) atarım. Eğer hata çok karmaşıksa ve hemen çözemiyorsam, kendi commit'imi Rollback / Revert (Geri al) yaparak sistemi hızlıca eski, çalışan (yeşil) haline döndürürüm. Kırık bir ana dal (main branch) ile kimse çalışamaz."

Senaryo 3: Modern CI/CD Araçları ve Pipeline Adımları

Mülakatçı: "Bana ideal bir CI/CD Pipeline'ının adımlarını ve kullanacağın araçları sayar mısın?"

İdeal bir Pipeline (Boru Hattı) soldan sağa şu adımlarla akar:

- **Source (Kaynak Kontrolü):** Geliştirici kodu Git (GitHub, GitLab, Bitbucket) deposuna gönderir. Bu işlem Pipeline'ı tetikler (Webhook).
- **Build (Derleme):** Jenkins, GitHub Actions veya GitLab CI devreye girer. Proje derlenir (Java ise .jar, C++ ise .exe üretilir). Modern sistemlerde uygulamanın bir Docker Image'ı (Konteyner) oluşturulur.
- **Test:** Birim testleri (Unit Tests) ve entegrasyon testleri çalıştırılır. Ayrıca kod kalitesini ve güvenlik açıklarını ölçmek için SonarQube gibi statik kod analizi (SAST) araçları çalışır. Test coverage (test kapsama oranı) örneğin %80'in altındaysa süreç otomatik iptal edilir.
- **Deploy (Dağıtım):** Oluşturulan Docker imajı, test veya canlı ortamdaki Kubernetes kümelerine (cluster) veya bulut sunucularına (AWS, Azure) yüklenir.

Docker ve Konteyner Mimarisi
› Test Çeşitleri: Unit vs Integration
› Veri Yapıları: Ağaç (Tree) Mülakat Sorusu